ebook_esp32_robot

Od Podstaw C do Profesjonalnego Developera Embedded

Kompleksowy Przewodnik po Budowie Robota ESP32-CAM

Autor: Tutorial wygenerowany na podstawie projektu ESP32-CAM Robot Controller

Wersja: 1.0

Wymagania wstępne: Podstawowa znajomość języka C (zmienne, funkcje, pętle, tablice)

Spis Treści

CZĘŚĆ I: FUNDAMENTY

- Rozdział 1: Zaawansowane Koncepty Języka C
- Rozdział 2: Wprowadzenie do Programowania Mikrokontrolerów
- Rozdział 3: ESP32 - Architektura i Ekosystem
- Rozdział 4: Komponenty Sprzętowe Projektu

CZĘŚĆ II: BUDOWA PROJEKTU

- Rozdział 5: Struktura Projektu i Konfiguracja
- Rozdział 6: Komponent robot_hal - Warstwa Abstrakcji Sprzętu
- Rozdział 7: Komponent motor_control - Sterowanie Silnikami
- Rozdział 8: Komponent servo_control - Sterowanie Wieżą
- Rozdział 9: Komponent wifi_manager - Zarządzanie WiFi
- Rozdział 10: Komponent camera_stream - Streaming MJPEG
- Rozdział 11: Komponent safety_handler - Bezpieczeństwo
- Rozdział 12: Komponent robot_core - Fasada

CZĘŚĆ III: HTTP API

- Rozdział 13: Komponent http_server - Serwer REST
- Rozdział 14: API Handlers - Endpointy REST

CZĘŚĆ IV: TESTOWANIE I DEPLOYMENT

- Rozdział 15: Pisanie Testów z Unity Framework

- Rozdział 16: Budowanie, Flashowanie i Debugging
- Rozdział 17: Gotowy Projekt i Dalszy Rozwój

DODATKI

- Dodatek A: Pełna Lista Referencji do Dokumentacji
- Dodatek B: Słownik Polsko-Angielski
- Dodatek C: Diagram Zależności Komponentów

CZĘŚĆ I: FUNDAMENTY

Rozdział 1: Zaawansowane Koncepty Języka C

1.1 Wprowadzenie

Zanim przejdziemy do programowania mikrokontrolerów, musimy ugruntować wiedzę z zaawansowanych aspektów języka C. W tym rozdziale poznasz techniki, które są powszechnie używane w programowaniu embedded:

- **Struktury** - grupowanie powiązanych danych
- **Wskaźniki** - bezpośredni dostęp do pamięci
- **Dyrektywy preprocesora** - konfiguracja kompilacji
- **Słowo kluczowe** `static` - enkapsulacja w C

1.2 Struktury (struct)

1.2.1 Podstawy struktur

Struktura to sposób na grupowanie powiązanych zmiennych różnych typów pod jedną nazwą. W programowaniu embedded struktury są wszechobecne - używamy ich do konfiguracji peryferiów, przechowywania stanu urządzeń i przekazywania złożonych danych.

```
// Podstawowa deklaracja struktury
struct motor_pins {
    int in1;  // Pin kierunku 1
    int in2;  // Pin kierunku 2
};
```

1.2.2 typedef struct - Tworzenie nowych typów

W praktyce niemal zawsze używamy `typedef` do tworzenia aliasu dla struktury. Dzięki temu nie musimy pisać słowa `struct` przy każdym użyciu:

```c
// Sposób 1: typedef po deklaracji struktury
struct motor_pins {
    int in1;
    int in2;
};
typedef struct motor_pins motor_pins_t;

// Sposób 2: typedef w jednej linii (preferowany w embedded)
typedef struct {
    int in1;
    int in2;
} motor_pins_t;
```

> **Konwencja nazewnictwa:** W programowaniu embedded często dodajemy sufiks `_t` do nazw typów (np. `motor_pins_t`, `gpio_num_t`). Jest to konwencja pochodząca z POSIX, która ułatwia rozpoznawanie typów w kodzie.

1.2.3 Przykład z projektu: motor_control.h

Przyjrzyjmy się rzeczywistej strukturze z naszego projektu. Plik `motor_control.h` definiuje konfigurację sterowania silnikami:

```c
/**
 * @file motor_control.h
 * @brief DRV8833 motor driver control interface
 */

// Linia 27–30: Struktura pinów pojedynczego silnika
typedef struct {
    gpio_num_t in1; /**< Forward direction pin */
    gpio_num_t in2; /**< Backward direction pin */
} motor_pins_t;
```

Wyjaśnienie linia po linii:

Linia	Kod	Wyjaśnienie
27	`typedef struct {`	Rozpoczynamy definicję nowego typu strukturalnego
28	`gpio_num_t in1;`	Pole `in1` typu `gpio_num_t`[1] - numer pinu GPIO dla kierunku "do przodu"
29	`gpio_num_t in2;`	Pole `in2` - numer pinu GPIO dla kierunku "do tyłu"
30	`} motor_pins_t;`	Kończymy strukturę i nadajemy jej nazwę typu `motor_pins_t`

1.2.4 Struktury zagnieżdżone

Struktury mogą zawierać inne struktury jako pola. Jest to potężna technika organizacji kodu:

```
// Linia 35-42: Pełna konfiguracja sterowania silnikami
typedef struct {
    motor_pins_t left_motor;   /**< Left motor pins */
    motor_pins_t right_motor;  /**< Right motor pins */
    gpio_num_t enable_pin;     /**< Shared PWM enable pin */
    uint32_t pwm_frequency_hz; /**< PWM frequency (default: 1000) */
    uint32_t ramp_duration_ms; /**< Soft-start ramp time (default: 500) */
    uint8_t ramp_steps;        /**< Number of ramp steps (default: 25) */
} motor_control_config_t;
```

Wyjaśnienie linia po linii:

Linia	Kod	Wyjaśnienie
35	`typedef struct {`	Rozpoczęcie definicji struktury konfiguracyjnej
36	`motor_pins_t left_motor;`	Zagnieżdżona struktura - piny lewego silnika
37	`motor_pins_t right_motor;`	Zagnieżdżona struktura - piny prawego silnika
38	`gpio_num_t enable_pin;`	Pin PWM włączający oba silniki
39	`uint32_t pwm_frequency_hz;`	Częstotliwość PWM w hercach (typ 32-bitowy bez znaku)[2]
40	`uint32_t ramp_duration_ms;`	Czas trwania "soft-start" w milisekundach
41	`uint8_t ramp_steps;`	Liczba kroków rampy (typ 8-bitowy, 0-255)
42	`} motor_control_config_t;`	Zakończenie i nazwa typu

1.2.5 Inicjalizacja designowana (Designated Initializers)

C99 wprowadził "designated initializers" - możliwość inicjalizacji pól struktury po nazwie. Jest to preferowany sposób w programowaniu embedded, ponieważ:

1. Kod jest bardziej czytelny
2. Kolejność pól nie ma znaczenia
3. Pominięte pola są automatycznie zerowane

```
// Inicjalizacja klasyczna (zła praktyka w embedded)
motor_control_config_t config = {
    {12, 13},        // left_motor - co to są za liczby?
    {14, 15},        // right_motor
```

```
    2,                  // enable_pin
    1000,               // pwm_frequency_hz
    500,                // ramp_duration_ms
    25                  // ramp_steps
};

// Inicjalizacja designowana (dobra praktyka)
motor_control_config_t config = {
    .left_motor = {.in1 = 12, .in2 = 13},
    .right_motor = {.in1 = 14, .in2 = 15},
    .enable_pin = 2,
    .pwm_frequency_hz = 1000,
    .ramp_duration_ms = 500,
    .ramp_steps = 25
};
```

1.2.6 Ćwiczenie 1.1

Zadanie: Zaprojektuj strukturę `sensor_config_t` do przechowywania konfiguracji czujnika odległości HC-SR04:

- Pin TRIGGER (wyjście)
- Pin ECHO (wejście)
- Maksymalny zasięg w centymetrach
- Timeout pomiaru w mikrosekundach

Rozwiązanie:

```
typedef struct {
    gpio_num_t trigger_pin;    /**< Trigger output pin */
    gpio_num_t echo_pin;       /**< Echo input pin */
    uint16_t max_range_cm;     /**< Maximum range in centimeters */
    uint32_t timeout_us;       /**< Measurement timeout in microseconds */
} sensor_config_t;

// Przykład użycia:
sensor_config_t ultrasonic = {
    .trigger_pin = GPIO_NUM_17,
    .echo_pin = GPIO_NUM_18,
    .max_range_cm = 400,
    .timeout_us = 25000  // ~4.25m przy prędkości dźwięku 340m/s
};
```

1.3 Wskaźniki - Zaawansowane Techniki

1.3.1 Przypomnienie podstaw

Wskaźnik to zmienna przechowująca adres w pamięci. W programowaniu embedded wskaźniki są niezbędne, ponieważ:

1. Pozwalają na bezpośredni dostęp do rejestrów sprzętowych
2. Umożliwiają przekazywanie dużych struktur bez kopiowania
3. Są podstawą dynamicznej alokacji pamięci

```
int value = 42;
int *ptr = &value;  // ptr przechowuje adres zmiennej value

printf("Wartość: %d\n", *ptr);      // Dereferencja - odczyt wartości
printf("Adres: %p\n", (void*)ptr);  // Wyświetlenie adresu
```

1.3.2 Wskaźniki do struktur

Gdy pracujemy ze strukturami, często przekazujemy je przez wskaźnik zamiast przez wartość. Dzięki temu:

- Unikamy kopiowania całej struktury (oszczędność pamięci i czasu)
- Funkcja może modyfikować oryginalną strukturę

```
// Przekazanie przez wartość - kopiuje całą strukturę (ZŁE dla dużych
struktur)
void print_config_bad(motor_control_config_t config) {
    printf("PWM: %lu Hz\n", config.pwm_frequency_hz);
}

// Przekazanie przez wskaźnik - przekazuje tylko adres (DOBRE)
void print_config_good(const motor_control_config_t *config) {
    printf("PWM: %lu Hz\n", config->pwm_frequency_hz);
}
```

> **Operator `->` vs `.`:**
>
> - `struktura.pole` - dostęp do pola gdy mamy zmienną struktury
> - `wskaznik->pole` - dostęp do pola gdy mamy wskaźnik (równoważne z `(*wskaznik).pole`)

1.3.3 Wskaźnik const - ochrona danych

Słowo kluczowe `const` przy wskaźniku ma różne znaczenie w zależności od pozycji:

```
// 1. Wskaźnik na stałą wartość - nie można zmienić wartości przez wskaźnik
const int *ptr1;
```

```c
// lub równoważnie:
int const *ptr1;

// 2. Stały wskaźnik - nie można zmienić adresu wskaźnika
int *const ptr2;

// 3. Stały wskaźnik na stałą wartość - nic nie można zmienić
const int *const ptr3;
```

W API embedded bardzo często widzimy:

```c
// Funkcja przyjmuje konfigurację, ale jej NIE modyfikuje
esp_err_t motor_control_init(const motor_control_config_t *config);
```

1.3.4 Wskaźniki do funkcji (Function Pointers)

Wskaźniki do funkcji pozwalają na "przekazywanie funkcji jako argumentów". Są używane w:

- Callbackach (funkcje wywoływane po zdarzeniu)
- Tablicach handlerów
- Implementacji wzorców projektowych

Składnia:

```c
// Deklaracja typu wskaźnika do funkcji
typedef void (*event_handler_t)(void *arg, int event_id, void *event_data);

// Deklaracja zmiennej tego typu
event_handler_t my_handler;

// Przypisanie funkcji
void my_callback(void *arg, int event_id, void *event_data) {
    printf("Event: %d\n", event_id);
}

my_handler = my_callback;  // Przypisanie (bez &, nazwa funkcji = jej adres)
my_handler(NULL, 42, NULL); // Wywołanie przez wskaźnik
```

1.3.5 Przykład z projektu: Rejestracja event handlera

W pliku `wifi_manager.c` widzimy praktyczne użycie wskaźników do funkcji:

```c
// Linia 36-37: Deklaracja handlera zdarzeń
static void wifi_event_handler(void *arg, esp_event_base_t event_base,
                               int32_t event_id, void *event_data);
```

```
// Linia 123-124: Rejestracja handlera
ret = esp_event_handler_instance_register(
    WIFI_EVENT,           // Typ zdarzeń do obsługi
    ESP_EVENT_ANY_ID,     // Które ID zdarzeń (wszystkie)
    &wifi_event_handler,  // Wskaźnik do naszej funkcji
    NULL,                 // Argument użytkownika (opcjonalny)
    NULL                  // Handle instancji (opcjonalny)
);
```

Wyjaśnienie:

Element	Wyjaśnienie
`esp_event_handler_instance_register`[3]	Funkcja ESP-IDF rejestrująca callback
`&wifi_event_handler`	Adres naszej funkcji obsługi zdarzeń
`WIFI_EVENT`[4]	Kategoria zdarzeń WiFi
`ESP_EVENT_ANY_ID`	Obsłuż wszystkie zdarzenia z tej kategorii

1.4 Dyrektywy Preprocesora

1.4.1 Czym jest preprocesor?

Preprocesor C to program uruchamiany przed kompilacją właściwą. Przetwarza on dyrektywy rozpoczynające się od `#`:

```
Kod źródłowy (.c) → Preprocesor → Kod przetworzony → Kompilator → Kod obiektowy (.o)
```

1.4.2 #define - Makra i stałe

Stałe symboliczne:

```
#define HAL_PWM_MAX_CHANNELS 8
#define HAL_PWM_DUTY_MAX 100
#define AP_IP_ADDR "10.42.0.1"
```

Makra z parametrami:

```
// Makro konwertujące milisekundy na ticki FreeRTOS
#define pdMS_TO_TICKS(ms) ((TickType_t)(((TickType_t)(ms) *
configTICK_RATE_HZ) / 1000))

// Makro sprawdzające błąd i przerywające
#define ESP_ERROR_CHECK(x) do {                          \
```

```
    esp_err_t err = (x);                                \
    if (err != ESP_OK) {                                \
        ESP_LOGE(TAG, "Error: %s", esp_err_to_name(err));\
        abort();                                        \
    }                                                   \
} while(0)
```

> **Uwaga:** Makra wieloliniowe wymagają znaku `\` na końcu każdej linii oprócz ostatniej.

1.4.3 Include Guards - Ochrona przed wielokrotnym włączeniem

Problem: Jeśli ten sam plik nagłówkowy zostanie włączony więcej niż raz, kompilator zgłosi błąd ponownej definicji.

Rozwiązanie - include guards:

```
// Na początku pliku motor_control.h
#ifndef MOTOR_CONTROL_H  // Jeśli NIE zdefiniowano MOTOR_CONTROL_H
#define MOTOR_CONTROL_H  // Zdefiniuj MOTOR_CONTROL_H

// ... zawartość pliku nagłówkowego ...

typedef struct {
    // ...
} motor_control_config_t;

#endif /* MOTOR_CONTROL_H */  // Koniec bloku warunkowego
```

Jak to działa:

1. Pierwsze włączenie: `MOTOR_CONTROL_H` nie istnieje → preprocesor definiuje go i przetwarza zawartość
2. Kolejne włączenia: `MOTOR_CONTROL_H` już istnieje → preprocesor pomija całą zawartość

1.4.4 Kompilacja warunkowa - #ifdef, #if

ESP-IDF intensywnie wykorzystuje kompilację warunkową do konfiguracji:

```
// W main.c - sprawdzanie trybu mock
#if APP_MOCK_MODE
    ESP_LOGI(TAG, "Hardware initialization skipped (mock mode)");
#else
    ESP_ERROR_CHECK(init_motors());
    ESP_ERROR_CHECK(init_servo());
    ESP_ERROR_CHECK(init_camera());
#endif
```

```
// Warunkowe włączanie funkcjonalności
#ifdef CONFIG_ROBOT_ENABLE_CAMERA
    ret = camera_stream_init(&camera_config);
#endif
```

1.4.5 Konfiguracja przez Kconfig

ESP-IDF używa systemu Kconfig (pochodzącego z jądra Linux) do konfiguracji projektu. Wartości z `menuconfig` są dostępne jako makra `CONFIG_*`:

```
// Plik: main/Kconfig.projbuild (fragment)
config ROBOT_MOTOR_LEFT_IN1
    int "Left Motor IN1 GPIO"
    default 12
    range 0 39

// W kodzie C:
#include "app_config.h"
#define APP_MOTOR_LEFT_IN1 CONFIG_ROBOT_MOTOR_LEFT_IN1  // = 12
```

1.5 Statyczne Zmienne i Funkcje - Enkapsulacja w C

1.5.1 Problem: Brak private w C

Język C nie ma słów kluczowych `private` ani `public` jak C++ czy Java. Jednak możemy osiągnąć podobny efekt używając `static`.

1.5.2 static dla zmiennych globalnych

Zmienna `static` na poziomie pliku jest widoczna **tylko w tym pliku**:

```
// W pliku motor_control.c

// Ta struktura jest PRYWATNA - nie można do niej dostać się z innych plików
static struct {
    bool initialized;
    motor_control_config_t config;
    hal_pwm_channel_t pwm_channel;
} s_motor = {0};
```

Wyjaśnienie linia po linii:

Linia	Kod	Wyjaśnienie
1	`static struct {`	Anonimowa struktura, widoczna tylko w tym pliku

Linia	Kod	Wyjaśnienie
2	`bool initialized;`	Flaga inicjalizacji
3	`motor_control_config_t config;`	Kopia konfiguracji
4	`hal_pwm_channel_t pwm_channel;`	Przydzielony kanał PWM
5	`} s_motor = {0};`	Zmienna `s_motor` zainicjalizowana zerami

> **Konwencja:** Zmienne statyczne na poziomie pliku często rozpoczynamy od `s_` (np. `s_motor`, `s_wifi`, `s_servo`). Jest to konwencja ESP-IDF i ułatwia rozpoznawanie zmiennych stanu.

1.5.3 static dla funkcji

Funkcja `static` jest widoczna tylko w pliku, w którym została zdefiniowana:

```c
// W pliku motor_control.c

// Funkcja PRYWATNA - pomocnicza, nie eksportowana w .h
static void set_motor_direction(const motor_pins_t *motor, motor_mode_t
mode) {
    switch (mode) {
        case MOTOR_MODE_FORWARD:
            hal_gpio_set_level(motor->in1, 1);
            hal_gpio_set_level(motor->in2, 0);
            break;
        // ...
    }
}

// Funkcja PUBLICZNA - zadeklarowana w motor_control.h
esp_err_t motor_move_forward(uint32_t duration_ms) {
    // Używa prywatnej funkcji
    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_FORWARD);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_FORWARD);
    return ESP_OK;
}
```

1.5.4 Wzorzec enkapsulacji w C

```
┌──────────────────────────────────────────┐
│          motor_control.h                 │  ← Interfejs publiczny
│   - Deklaracje typów (typedef)           │
```

```
│  - Prototypy funkcji publicznych    │
└─────────────────────────────────────┘
                  │
                  ▼
┌─────────────────────────────────────┐
│         motor_control.c             │  ← Implementacja
│  - static zmienne stanu             │
│  - static funkcje pomocnicze        │
│  - Implementacje funkcji publicznych│
└─────────────────────────────────────┘
```

1.5.5 Ćwiczenie 1.2

Zadanie: Przeanalizuj poniższy kod i określ, które elementy są publiczne, a które prywatne:

```c
// sensor.h
#ifndef SENSOR_H
#define SENSOR_H

typedef struct {
    int pin;
    float last_reading;
} sensor_t;

int sensor_init(sensor_t *s, int pin);
float sensor_read(sensor_t *s);

#endif

// sensor.c
#include "sensor.h"

static int calibration_offset = 0;

static float apply_calibration(float raw_value) {
    return raw_value + calibration_offset;
}

int sensor_init(sensor_t *s, int pin) {
    s->pin = pin;
    s->last_reading = 0.0f;
    calibration_offset = 5;  // Przykładowa kalibracja
    return 0;
}
```

```c
float sensor_read(sensor_t *s) {
    float raw = /* odczyt z ADC */;
    s->last_reading = apply_calibration(raw);
    return s->last_reading;
}
```

Rozwiązanie:

Element	Publiczny/Prywatny	Uzasadnienie
`sensor_t`	Publiczny	Zdefiniowany w .h, dostępny wszędzie
`sensor_init()`	Publiczny	Zadeklarowany w .h
`sensor_read()`	Publiczny	Zadeklarowany w .h
`calibration_offset`	Prywatny	`static` w .c, niewidoczny na zewnątrz
`apply_calibration()`	Prywatny	`static` w .c, niewidoczny na zewnątrz

1.6 Podsumowanie rozdziału

W tym rozdziale poznałeś:

1. **Struktury** - jak grupować dane, używać `typedef`, inicjalizację designowaną
2. **Wskaźniki** - przekazywanie przez referencję, `const`, wskaźniki do funkcji
3. **Preprocesor** - `#define`, include guards, kompilacja warunkowa, Kconfig
4. **static** - enkapsulacja zmiennych i funkcji w C

Te techniki będą wykorzystywane we wszystkich komponentach naszego projektu robota.

Kluczowe punkty do zapamiętania:

- ✅ Używaj `typedef struct` dla czytelności
- ✅ Inicjalizuj struktury przez designated initializers
- ✅ Przekazuj duże struktury przez wskaźnik `const`
- ✅ Zawsze używaj include guards w plikach .h
- ✅ Ukrywaj szczegóły implementacji przez `static`

Rozdział 2: Wprowadzenie do Programowania Mikrokontrolerów

2.1 Czym jest mikrokontroler?

2.1.1 Mikrokontroler vs procesor PC

Cecha	Procesor PC	Mikrokontroler
Architektura	x86/ARM, wielordzeniowy	ARM/Xtensa, 1-2 rdzenie
Taktowanie	2-5 GHz	80-240 MHz
RAM	8-64 GB	320-520 KB
Pamięć programu	SSD/HDD (TB)	Flash (4-16 MB)
System operacyjny	Windows/Linux/macOS	RTOS lub brak
Peryferia	Zewnętrzne	Wbudowane (GPIO, ADC, PWM...)
Pobór mocy	15-150 W	10-500 mW
Cena	$100-$1000	$1-$10

2.1.2 Architektura mikrokontrolera

```
┌──────────────────────────────────────────────────────────────┐
│                        MIKROKONTROLER                        │
│  ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────────┐  │
│  │   CPU    │   │  Flash   │   │   RAM    │   │   Peryferia  │  │
│  │ (rdzeń)  │   │(program) │   │ (dane)   │   │ GPIO,PWM,ADC... │  │
│  └──────────┘   └──────────┘   └──────────┘   └──────────────┘  │
│       │              │              │                │          │
│  ┌────┴──────────────┴──────────────┴────────────────┴──────┐  │
│  │                        MAGISTRALA                         │  │
│  └──────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
                        ┌──────────────────────┐
                        │  Świat zewnętrzny    │
                        │ (sensory, silniki...) │
                        └──────────────────────┘
```

2.1.3 Pamięć w mikrokontrolerze

Flash (pamięć programu):

- Nieulotna (dane przetrwają wyłączenie zasilania)
- Przechowuje kod programu
- Ograniczona liczba cykli zapisu (~10,000-100,000)

RAM (pamięć danych):

- Ulotna (dane tracone po wyłączeniu)
- Przechowuje zmienne, stos, sterta
- Szybki dostęp, ale ograniczona pojemność

W ESP32-CAM:

- Flash: 4 MB
- SRAM: 520 KB
- PSRAM: 4 MB (zewnętrzna, do dużych buforów jak klatki kamery)

2.2 GPIO - General Purpose Input/Output

2.2.1 Czym jest GPIO?

GPIO to piny mikrokontrolera, które możemy programowo konfigurować jako:

- **Wejście** - odczyt stanu (przycisk, czujnik)
- **Wyjście** - ustawienie stanu (LED, przekaźnik)

2.2.2 Stany logiczne

Stan	Napięcie (3.3V logic)	Wartość w kodzie
LOW	0V (GND)	0
HIGH	3.3V (VCC)	1

2.2.3 Konfiguracja GPIO w ESP-IDF

Aby użyć pinu GPIO, musimy go najpierw skonfigurować. W ESP-IDF używamy struktury `gpio_config_t`[5]:

```
// Plik: components/robot_hal/src/hal_gpio.c

esp_err_t hal_gpio_init_output(gpio_num_t pin) {
    // Linia 13-17: Konfiguracja pinu jako wyjście
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << pin),    // Maska bitowa - który pin
        .mode = GPIO_MODE_OUTPUT,          // Tryb: wyjście
        .pull_up_en = GPIO_PULLUP_DISABLE, // Bez rezystora pull-up
        .pull_down_en = GPIO_PULLDOWN_DISABLE, // Bez rezystora pull-down
        .intr_type = GPIO_INTR_DISABLE     // Bez przerwań
    };
```

```
    // Linia 19: Zastosowanie konfiguracji
    esp_err_t ret = gpio_config(&io_conf);

    if (ret == ESP_OK) {
        // Linia 21: Ustawienie początkowego stanu na LOW
        gpio_set_level(pin, 0);
        ESP_LOGD(TAG, "GPIO %d initialized as output", pin);
    } else {
        ESP_LOGE(TAG, "Failed to init GPIO %d: %s", pin,
esp_err_to_name(ret));
    }

    return ret;
}
```

Wyjaśnienie linia po linii:

Linia	Kod	Wyjaśnienie
13	`gpio_config_t io_conf = {`	Struktura konfiguracyjna GPIO[5:1]
14	`.pin_bit_mask = (1ULL << pin)`	Maska bitowa wybierająca pin. `1ULL << 12` = bit 12 ustawiony
15	`.mode = GPIO_MODE_OUTPUT`	Pin będzie wyjściem[6]
16	`.pull_up_en = GPIO_PULLUP_DISABLE`	Wyłączenie wewnętrznego rezystora pull-up
17	`.pull_down_en = GPIO_PULLDOWN_DISABLE`	Wyłączenie rezystora pull-down
18	`.intr_type = GPIO_INTR_DISABLE`	Bez obsługi przerwań
19	`gpio_config(&io_conf)`	Zastosowanie konfiguracji[7]
21	`gpio_set_level(pin, 0)`	Ustawienie początkowego stanu LOW[8]

2.2.4 Ustawianie stanu GPIO

```
// Plik: components/robot_hal/src/hal_gpio.c

esp_err_t hal_gpio_set_level(gpio_num_t pin, uint8_t level) {
    // Linia 31: Wywołanie funkcji ESP-IDF
    // level ? 1 : 0 - normalizacja do 0 lub 1
    esp_err_t ret = gpio_set_level(pin, level ? 1 : 0);

    if (ret != ESP_OK) {
```

```
        ESP_LOGE(TAG, "Failed to set GPIO %d to %d: %s",
                 pin, level, esp_err_to_name(ret));
    }
    return ret;
}
```

2.2.5 Ćwiczenie 2.1: Mruganie LED

Zadanie: Napisz funkcję, która miga wbudowaną diodą LED (GPIO 4 na ESP32-CAM) 5 razy.

Rozwiązanie:

```
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#define LED_GPIO 4

void blink_led(int times) {
    // Konfiguracja pinu jako wyjście
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << LED_GPIO),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&io_conf);

    // Mruganie
    for (int i = 0; i < times; i++) {
        gpio_set_level(LED_GPIO, 1);  // LED ON
        vTaskDelay(pdMS_TO_TICKS(500)); // Czekaj 500ms
        gpio_set_level(LED_GPIO, 0);  // LED OFF
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

2.3 PWM - Pulse Width Modulation

2.3.1 Czym jest PWM?

PWM (Modulacja Szerokości Impulsu) to technika generowania sygnału analogowego za pomocą cyfrowego wyjścia. Zmieniając stosunek czasu włączenia do wyłączenia (duty cycle), możemy kontrolować:

- Jasność LED
- Prędkość silnika DC
- Pozycję serwa

```
   Duty cycle 25%:          Duty cycle 50%:          Duty cycle 75%:

   ┌─┐   ┌─┐   ┌─┐         ┌──┐  ┌──┐  ┌──┐         ┌───┐ ┌───┐ ┌───┐
   │ │   │ │   │ │         │  │  │  │  │  │         │   │ │   │ │   │
───┘ └───┘ └───┘ └──       ──┘  └──┘  └──┘  └──      ──┘   └─┘   └─┘   └──

   25%   75%                50%    50%               75%    25%
```

2.3.2 Parametry PWM

Parametr	Opis	Typowe wartości
Częstotliwość	Ile razy na sekundę powtarza się cykl	Silniki: 1-20 kHz, Serwa: 50 Hz
Duty Cycle	Procent czasu w stanie HIGH	0-100%
Rozdzielczość	Ile poziomów duty cycle	8-bit (256), 13-bit (8192)

2.3.3 LEDC - PWM w ESP32

ESP32 posiada peryferium LEDC (LED Control), które pomimo nazwy służy do generowania dowolnych sygnałów PWM[9].

```c
// Plik: components/robot_hal/src/hal_pwm.c

// Linia 15-16: Stałe konfiguracyjne
#define LEDC_TIMER_RESOLUTION LEDC_TIMER_13_BIT  // Rozdzielczość 13 bitów
#define LEDC_MAX_DUTY         ((1 << LEDC_TIMER_RESOLUTION) - 1)  // = 8191

esp_err_t hal_pwm_init(gpio_num_t pin, uint32_t frequency_hz,
                       hal_pwm_channel_t *channel) {
    // Sprawdzenie argumentów
    if (channel == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    // Znalezienie wolnego kanału
    hal_pwm_channel_t ch = find_free_channel();
    if (ch == HAL_PWM_CHANNEL_INVALID) {
        ESP_LOGE(TAG, "No free PWM channels available");
        return ESP_ERR_NO_MEM;
    }
```

```
    // Linia 46-51: Konfiguracja timera LEDC
    ledc_timer_config_t timer_conf = {
        .speed_mode = LEDC_LOW_SPEED_MODE,     // Tryb niskiej prędkości
        .timer_num = (ledc_timer_t)(ch / 2),   // 2 kanały na timer
        .duty_resolution = LEDC_TIMER_RESOLUTION, // Rozdzielczość 13-bit
        .freq_hz = frequency_hz,               // Częstotliwość
        .clk_cfg = LEDC_AUTO_CLK               // Automatyczny wybór zegara
    };

    esp_err_t ret = ledc_timer_config(&timer_conf);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "LEDC timer config failed: %s", esp_err_to_name(ret));
        return ret;
    }

    // Linia 60-66: Konfiguracja kanału LEDC
    ledc_channel_config_t channel_conf = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .channel = (ledc_channel_t)ch,          // Numer kanału
        .timer_sel = (ledc_timer_t)(ch / 2),    // Powiązany timer
        .intr_type = LEDC_INTR_DISABLE,         // Bez przerwań
        .gpio_num = pin,                        // Numer pinu GPIO
        .duty = 0,                              // Początkowy duty cycle: 0
        .hpoint = 0                             // Punkt startu impulsu
    };

    ret = ledc_channel_config(&channel_conf);
    // ...

    *channel = ch;  // Zwrócenie przydzielonego kanału
    return ESP_OK;
}
```

Wyjaśnienie kluczowych funkcji ESP-IDF:

Funkcja	Opis
`ledc_timer_config()`[10]	Konfiguruje timer generujący bazową częstotliwość
`ledc_channel_config()`[11]	Łączy timer z pinem GPIO
`ledc_set_duty()`[12]	Ustawia wartość duty cycle
`ledc_update_duty()`[13]	Aktywuje zmianę duty cycle

2.3.4 Ustawianie Duty Cycle

```
// Plik: components/robot_hal/src/hal_pwm.c

esp_err_t hal_pwm_set_duty(hal_pwm_channel_t channel, uint8_t duty_percent)
{
    // Sprawdzenie poprawności kanału
    if (!hal_pwm_is_valid(channel)) {
        return ESP_ERR_INVALID_ARG;
    }

    // Ograniczenie do 100%
    if (duty_percent > HAL_PWM_DUTY_MAX) {
        duty_percent = HAL_PWM_DUTY_MAX;
    }

    // Linia 92: Konwersja procentów na wartość rejestru
    // HAL_PWM_DUTY_MAX = 100, LEDC_MAX_DUTY = 8191
    // duty_percent=50 → duty = 8191 * 50 / 100 = 4095
    uint32_t duty = (LEDC_MAX_DUTY * duty_percent) / HAL_PWM_DUTY_MAX;

    // Linia 94-96: Ustawienie i aktywacja
    esp_err_t ret = ledc_set_duty(LEDC_LOW_SPEED_MODE,
                                  (ledc_channel_t)channel, duty);
    if (ret == ESP_OK) {
        ret = ledc_update_duty(LEDC_LOW_SPEED_MODE,
(ledc_channel_t)channel);
    }

    return ret;
}
```

2.3.5 PWM dla serw - szerokość impulsu

Serwomechanizmy używają PWM o stałej częstotliwości 50 Hz (okres 20 ms), gdzie pozycja jest określana przez szerokość impulsu:

Pozycja	Szerokość impulsu	Angle
Minimum	500-1000 μs	0°
Środek	1500 μs	90°
Maksimum	2000-2500 μs	180°

```c
// Plik: components/robot_hal/src/hal_pwm.c

esp_err_t hal_pwm_set_servo_pulse(hal_pwm_channel_t channel, uint16_t
pulse_us) {
    if (!hal_pwm_is_valid(channel)) {
        return ESP_ERR_INVALID_ARG;
    }

    // Linia 116-117: Obliczenie duty cycle dla danej szerokości impulsu
    // Przy 50Hz: okres = 20000µs
    // duty = (pulse_us / period_us) * max_duty
    uint32_t period_us = 1000000 / channel_freq[channel]; // np. 20000 dla
50Hz
    uint32_t duty = (LEDC_MAX_DUTY * pulse_us) / period_us;

    if (duty > LEDC_MAX_DUTY) {
        duty = LEDC_MAX_DUTY;
    }

    esp_err_t ret = ledc_set_duty(LEDC_LOW_SPEED_MODE,
                                  (ledc_channel_t)channel, duty);
    if (ret == ESP_OK) {
        ret = ledc_update_duty(LEDC_LOW_SPEED_MODE,
(ledc_channel_t)channel);
    }

    return ret;
}
```

2.4 FreeRTOS - System Operacyjny Czasu Rzeczywistego

2.4.1 Czym jest RTOS?

RTOS (Real-Time Operating System) to lekki system operacyjny zaprojektowany dla systemów embedded. FreeRTOS[14] jest najpopularniejszym otwartym RTOS-em i jest domyślnie używany przez ESP-IDF.

Główne koncepty:

- **Task (zadanie)** - niezależny wątek wykonania
- **Scheduler** - przydziela czas procesora zadaniom
- **Priority** - zadania o wyższym priorytecie mają pierwszeństwo

2.4.2 Tworzenie zadania

```c
// Plik: components/motor_control/src/pwm_ramper.c

// Linia 110-111: Tworzenie zadania FreeRTOS
BaseType_t ret = xTaskCreate(
    ramp_task,           // Wskaźnik do funkcji zadania
    "pwm_ramp",          // Nazwa (do debugowania)
    2048,                // Rozmiar stosu w bajtach
    NULL,                // Argument przekazywany do zadania
    5,                   // Priorytet (0 = najniższy)
    &s_ramper.task_handle // Handle do kontroli zadania
);
```

Parametry `xTaskCreate()`[15]:

Parametr	Typ	Opis
`pvTaskCode`	Wskaźnik do funkcji	Funkcja do wykonania w zadaniu
`pcName`	String	Nazwa zadania (max 16 znaków)
`usStackDepth`	uint16_t	Rozmiar stosu w słowach (×4 na ESP32)
`pvParameters`	void*	Argument przekazywany do funkcji
`uxPriority`	UBaseType_t	Priorytet (0-24 w ESP-IDF)
`pxCreatedTask`	TaskHandle_t*	Handle do późniejszej kontroli

2.4.3 Funkcja zadania

```c
// Plik: components/motor_control/src/pwm_ramper.c

// Linia 31-85: Funkcja zadania rampy PWM
static void ramp_task(void *arg) {
    (void)arg;  // Nieużywany argument

    while (1) {  // Nieskończona pętla - zadanie działa cały czas
        // Linia 36: Czekanie na powiadomienie
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

        // Sprawdzenie czy nie żądano zatrzymania
        if (s_ramper.stop_requested) {
            continue;
        }

        // ... logika rampowania PWM ...
```

```
        // Linia 70: Opóźnienie między krokami
        vTaskDelay(pdMS_TO_TICKS(step_delay_ms));
    }
}
```

2.4.4 Kluczowe funkcje FreeRTOS

Funkcja	Opis
`vTaskDelay()`[16]	Wstrzymuje zadanie na określony czas
`ulTaskNotifyTake()`[17]	Czeka na powiadomienie z innego zadania
`xTaskNotifyGive()`[18]	Wysyła powiadomienie do zadania
`xSemaphoreCreateMutex()`[19]	Tworzy mutex do synchronizacji
`xSemaphoreTake()`/`xSemaphoreGive()`	Blokuje/zwalnia mutex

2.4.5 Makro pdMS_TO_TICKS

FreeRTOS operuje na "tickach" - jednostkach czasu schedulera. Makro `pdMS_TO_TICKS()` konwertuje milisekundy na ticki:

```
// Przy CONFIG_FREERTOS_HZ = 1000 (1000 ticków/sekundę):
// pdMS_TO_TICKS(100) = 100 ticków = 100 ms

vTaskDelay(pdMS_TO_TICKS(500));  // Czekaj 500 ms
```

2.5 Timery

2.5.1 ESP Timer API

ESP-IDF dostarcza wygodne API do timerów programowych[20]. Istnieją dwa rodzaje:

- **Periodic** - wywoływany cyklicznie co określony czas
- **One-shot** - wywoływany jednokrotnie po upływie czasu

2.5.2 Tworzenie timera

```
// Plik: components/safety_handler/src/safety_handler.c

// Linia 57-67: Tworzenie timera auto-stop (one-shot)
esp_timer_create_args_t auto_stop_args = {
    .callback = auto_stop_callback,        // Funkcja wywoływana
    .arg = NULL,                           // Argument dla callbacka
    .dispatch_method = ESP_TIMER_TASK,     // Wykonanie w kontekście zadania
```

```
    .name = "auto_stop"                    // Nazwa do debugowania
};

esp_err_t ret = esp_timer_create(&auto_stop_args,
&s_safety.auto_stop_timer);

// Linia 84: Uruchomienie timera periodycznego (watchdog)
ret = esp_timer_start_periodic(
    s_safety.watchdog_timer,
    config->watchdog_timeout_ms * 1000  // Czas w mikrosekundach!
);
```

2.5.3 Callback timera

```
// Plik: components/safety_handler/src/safety_handler.c

// Linia 30-34: Callback auto-stop
static void auto_stop_callback(void *arg) {
    (void)arg;
    ESP_LOGW(TAG, "Auto-stop triggered");
    motor_stop_all();  // Zatrzymaj silniki
}

// Linia 39-48: Callback watchdoga
static void watchdog_callback(void *arg) {
    (void)arg;

    // Jeśli watchdog nie był "nakarmiony" - awaryjne zatrzymanie
    if (!s_safety.watchdog_fed) {
        ESP_LOGE(TAG, "Watchdog timeout - emergency shutdown!");
        safety_emergency_shutdown();
    }

    // Reset flagi - musi być ponownie "nakarmiony"
    s_safety.watchdog_fed = false;
}
```

2.5.4 Funkcje ESP Timer API

Funkcja	Opis
`esp_timer_create()`[21]	Tworzy timer
`esp_timer_start_once()`[22]	Uruchamia timer jednorazowy

Funkcja	Opis
`esp_timer_start_periodic()`[23]	Uruchamia timer cykliczny
`esp_timer_stop()`[24]	Zatrzymuje timer
`esp_timer_delete()`[25]	Usuwa timer i zwalnia zasoby

2.6 Ćwiczenie 2.2: Zadanie mrugające LED

Zadanie: Utwórz zadanie FreeRTOS, które mruga LED co 250 ms, jednocześnie pozwalając głównemu programowi wykonywać inne operacje.

Rozwiązanie:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define LED_GPIO 4
static const char *TAG = "blink_task";

// Funkcja zadania
static void blink_task(void *arg) {
    int interval_ms = (int)arg;  // Odbiór argumentu

    // Konfiguracja GPIO
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << LED_GPIO),
        .mode = GPIO_MODE_OUTPUT,
    };
    gpio_config(&io_conf);

    bool state = false;

    while (1) {
        state = !state;
        gpio_set_level(LED_GPIO, state);
        ESP_LOGI(TAG, "LED %s", state ? "ON" : "OFF");
        vTaskDelay(pdMS_TO_TICKS(interval_ms));
    }
}
```

```
void app_main(void) {
    // Utworzenie zadania z interwałem 250 ms
    xTaskCreate(
        blink_task,
        "blink",
        2048,
        (void*)250,  // Przekazanie interwału jako argument
        5,
        NULL
    );

    // Główna pętla może robić inne rzeczy
    while (1) {
        ESP_LOGI(TAG, "Main loop running...");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

2.7 Podsumowanie rozdziału

W tym rozdziale poznałeś:

1. **GPIO** - konfiguracja pinów jako wejścia/wyjścia
2. **PWM** - generowanie sygnałów o zmiennym wypełnieniu (LEDC)
3. **FreeRTOS** - tworzenie zadań, opóźnienia, powiadomienia
4. **Timery** - periodic i one-shot timery

Kluczowe punkty:

- ✅ Zawsze konfiguruj GPIO przed użyciem (`gpio_config()`)
- ✅ LEDC używa 13-bitowej rozdzielczości (0-8191)
- ✅ Zadania FreeRTOS działają w nieskończonej pętli
- ✅ `vTaskDelay()` zwalnia procesor dla innych zadań
- ✅ Timery ESP używają mikrosekund, nie milisekund!

Rozdział 3: ESP32 - Architektura i Ekosystem

3.1 Dlaczego ESP32?

3.1.1 Porównanie z alternatywami

Cecha	ESP32	Arduino Uno	STM32F4	Raspberry Pi Pico
Rdzeń	Dual Xtensa 240MHz	AVR 16MHz	ARM Cortex-M4 168MHz	Dual ARM Cortex-M0+ 133MHz
RAM	520 KB	2 KB	192 KB	264 KB
Flash	4-16 MB	32 KB	512 KB-1 MB	2 MB
WiFi	Wbudowane	Brak	Brak	Pico W: tak
Bluetooth	Wbudowane	Brak	Brak	Brak
Cena	~$3-5	~$3-5	~$5-15	~$4-6
IDE	ESP-IDF/Arduino	Arduino IDE	STM32CubeIDE	SDK/Arduino
Dla kogo	IoT, robotyka	Początkujący	Zaawansowani	Hobby/edukacja

3.1.2 Zalety ESP32 dla projektu robota

1. **Wbudowane WiFi** - sterowanie przez HTTP bez dodatkowych modułów
2. **Wystarczająca moc** - dwa rdzenie pozwalają na streaming wideo i sterowanie
3. **PSRAM** - ESP32-CAM ma 4 MB zewnętrznej RAM dla buforów kamery
4. **Bogate peryferia** - PWM, ADC, I2C, SPI w jednym układzie
5. **Niska cena** - ESP32-CAM kosztuje ~$5-10
6. **Duża społeczność** - łatwo znaleźć pomoc i przykłady

3.2 Architektura ESP32

3.2.1 Diagram blokowy

```
┌─────────────────────────────────────────────────────────────────────────┐
│                                   ESP32                                 │
│                                                                         │
│   ┌─────────────────────────────────────────────────────────────────┐   │
│   │                 Procesor Xtensa LX6 Dual-Core                   │   │
│   │   ┌──────────────────┐     ┌──────────────────┐                │   │
│   │   │     PRO_CPU      │     │     APP_CPU      │                │   │
│   │   │    (Protocol)    │     │   (Application)  │                │   │
│   │   │     240 MHz      │     │     240 MHz      │                │   │
│   │   └──────────────────┘     └──────────────────┘                │   │
│   └─────────────────────────────────────────────────────────────────┘   │
│                                  │                                      │
```

```
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                        Bus Matrix                        │   │
│   └──────────────────────────────────────────────────────────┘   │
│                │                │                │                 │
│   ┌────────────┴──────┐  ┌──────┴──────────┐  ┌──┴──────────────┐  │
│   │      Memory       │  │   Peripherals   │  │    Connectivity │  │
│   │  ┌──────────────┐ │  │  ┌──────────────┐ │  │ ┌──────────────┐ │  │
│   │  │ SRAM 520KB│  │  │  │GPIO (34x) │  │  │ │ WiFi 802.11  │  │  │
│   │  │ RTC 8KB    │  │  │  │SPI (4x)    │  │  │ │ b/g/n        │  │  │
│   │  │ ROM 448KB │  │  │  │I2C (2x)    │  │  │ ├──────────────┤  │  │
│   │  └──────────────┘ │  │  │UART (3x)   │  │  │ │ Bluetooth 4.2 │  │  │
│   │                   │  │  │ADC (18x)   │  │  │ │ BLE          │  │  │
│   │ External:         │  │  │DAC (2x)    │  │  │ └──────────────┘  │  │
│   │ PSRAM 4MB         │  │  │PWM (16x)   │  │  │                   │  │
│   │ Flash 4-16MB      │  │  │Timer (4x) │  │  │                   │  │
│   └──────────────────┘  │  │I2S (2x)    │  │  │                   │  │
│                         │  └──────────────┘  │  └──────────────────┘  │
│                         └────────────────────┘                        │
└──────────────────────────────────────────────────────────────────────┘
```

3.2.2 Kluczowe parametry ESP32

Parametr	Wartość
Procesor	Xtensa LX6 Dual-Core, 80-240 MHz
SRAM	520 KB
RTC SRAM	8 KB (zachowywany w deep sleep)
ROM	448 KB
Flash (zewnętrzny)	4-16 MB (SPI)
PSRAM (opcjonalny)	Do 8 MB
GPIO	34 piny (nie wszystkie dostępne)
ADC	18 kanałów, 12-bit
DAC	2 kanały, 8-bit
PWM	16 kanałów (LEDC)
Touch	10 pinów pojemnościowych
UART	3 porty
SPI	4 interfejsy

Parametr	Wartość
I2C	2 interfejsy
WiFi	802.11 b/g/n (2.4 GHz)
Bluetooth	Classic + BLE 4.2
Zasilanie	3.3V, ~240 mA przy WiFi TX

3.3 ESP32-CAM AI-Thinker

3.3.1 Specyfikacja modułu

ESP32-CAM to moduł łączący ESP32 z kamerą OV2640:

```
┌──────────────────────────────────────────┐
│              ESP32-CAM                   │
│  ┌──────────────────────────────────┐    │
│  │       Kamera OV2640              │    │
│  │       (2MP, JPEG)                │    │
│  └──────────────────────────────────┘    │
│                                          │
│  ┌──────────────────────────────────┐    │
│  │        ESP32-S                   │    │
│  │   • Dual-Core 240MHz             │    │
│  │   • 520KB SRAM + 4MB PSRAM       │    │
│  │   • 4MB Flash                    │    │
│  │   • WiFi + Bluetooth             │    │
│  └──────────────────────────────────┘    │
│                                          │
│  [Flash LED GPIO4] [MicroSD slot]        │
│                                          │
│  GPIO:  3V3 GND GPIO16 GPIO0 ...         │
└──────────────────────────────────────────┘
```

3.3.2 Ograniczenia GPIO

Na ESP32-CAM wiele pinów GPIO jest zajętych przez kamerę i inne funkcje:

GPIO	Funkcja	Dostępność dla projektu
0	XCLK kamery, boot mode	❌ Zajęty
1	UART TX	❌ Programowanie

GPIO	Funkcja	Dostępność dla projektu
2	Dostępny	✅ Motors Enable
3	UART RX	❌ Programowanie
4	Flash LED	⚠️ LED włączony przy HIGH
5, 18, 19, 21, 34, 35, 36, 39	Data kamery	❌ Zajęte
12, 13, 14, 15	SD card	✅ Użyjemy dla silników
16	PSRAM	⚠️ Działa jako wyjście
22, 23, 25, 26, 27, 32	Kamera	❌ Zajęte
33	Wewnętrzny LED	⚠️ Odwrócona logika

3.3.3 Pinout projektu

W naszym projekcie używamy pinów:

```
// Z pliku Kconfig.projbuild – domyślne wartości GPIO
#define APP_MOTOR_LEFT_IN1   12  // SD D2 (wolny gdy nie używamy SD)
#define APP_MOTOR_LEFT_IN2   13  // SD D3
#define APP_MOTOR_RIGHT_IN1  14  // SD CLK
#define APP_MOTOR_RIGHT_IN2  15  // SD CMD
#define APP_MOTORS_ENABLE    2   // SD D0
#define APP_SERVO_GPIO       16  // U2RXD (PSRAM CS – działa jako wyjście)
```

> **Kompromis:** Rezygnujemy z karty MicroSD, aby uwolnić piny dla sterowania silnikami.

3.4 ESP-IDF Framework

3.4.1 Czym jest ESP-IDF?

ESP-IDF (Espressif IoT Development Framework) to oficjalny framework do programowania ESP32[26]. Oferuje:

- Kompletny zestaw sterowników dla wszystkich peryferiów
- FreeRTOS do wielozadaniowości
- Stos sieciowy (WiFi, Bluetooth, TCP/IP)
- System budowania oparty na CMake
- Konfigurację przez menuconfig (jak jądro Linux)

3.4.2 Struktura projektu ESP-IDF

```
projekt/
├── CMakeLists.txt          # Główny plik CMake projektu
├── sdkconfig               # Wygenerowana konfiguracja (nie edytuj!)
├── sdkconfig.defaults      # Domyślne wartości konfiguracji
├── partitions.csv          # Tablica partycji flash
│
├── main/                   # Główny komponent (entry point)
│   ├── CMakeLists.txt      # CMake dla main
│   ├── main.c              # Punkt wejścia app_main()
│   ├── Kconfig.projbuild   # Konfiguracja w menuconfig
│   └── include/            # Nagłówki dla main
│
├── components/             # Własne komponenty
│   ├── motor_control/
│   │   ├── CMakeLists.txt
│   │   ├── src/
│   │   │   └── motor_control.c
│   │   └── include/
│   │       └── motor_control.h
│   └── ...
│
├── spiffs_data/            # Pliki dla systemu plików SPIFFS
│   ├── index.html
│   └── style.css
│
└── test/                   # Testy jednostkowe
    └── main/
        ├── test_main.c
        └── test_*.c
```

3.4.3 CMakeLists.txt - system budowania

Główny CMakeLists.txt projektu:

```
# Wymagana wersja CMake
cmake_minimum_required(VERSION 3.16)

# Dołączenie frameworka ESP-IDF
include($ENV{IDF_PATH}/tools/cmake/project.cmake)

# Nazwa projektu
project(esp32_robot)
```

CMakeLists.txt komponentu (np. motor_control):

```
# Rejestracja komponentu
idf_component_register(
    SRCS
        "src/motor_control.c"    # Pliki źródłowe
        "src/pwm_ramper.c"
    INCLUDE_DIRS
        "include"                 # Katalog z nagłówkami
    REQUIRES
        robot_hal                 # Zależności od innych komponentów
        freertos
)
```

3.4.4 Kconfig - system konfiguracji

Kconfig pozwala na konfigurację projektu przez menu tekstowe (`idf.py menuconfig`):

```
# Plik: main/Kconfig.projbuild

menu "Robot Controller Configuration"

    menu "WiFi Settings"
        config ROBOT_WIFI_SSID
            string "Access Point SSID"
            default "ESP32-Robot"
            help
                WiFi Access Point network name.

        config ROBOT_WIFI_PASSWORD
            string "Access Point Password"
            default ""
            help
                Minimum 8 characters for WPA2.
    endmenu

    menu "Motor Control"
        config ROBOT_MOTOR_LEFT_IN1
            int "Left Motor IN1 GPIO"
            default 12
            range 0 39
            help
                GPIO pin connected to left motor DRV8833 IN1.
    endmenu

endmenu
```

Dostęp do wartości w kodzie C:

```
// Wartości z menuconfig są dostępne jako makra CONFIG_*
const char *ssid = CONFIG_ROBOT_WIFI_SSID;       // "ESP32-Robot"
int motor_pin = CONFIG_ROBOT_MOTOR_LEFT_IN1;      // 12
```

3.5 Instalacja środowiska

3.5.1 Linux/macOS

```
# 1. Instalacja zależności (Ubuntu/Debian)
sudo apt-get install git wget flex bison gperf python3 python3-pip \
    python3-venv cmake ninja-build ccache libffi-dev libssl-dev \
    dfu-util libusb-1.0-0

# 2. Klonowanie ESP-IDF
mkdir -p ~/esp
cd ~/esp
git clone -b v5.2 --recursive https://github.com/espressif/esp-idf.git

# 3. Instalacja narzędzi
cd ~/esp/esp-idf
./install.sh esp32

# 4. Aktywacja środowiska (przy każdym nowym terminalu)
. ~/esp/esp-idf/export.sh
```

3.5.2 Windows

1. Pobierz [ESP-IDF Tools Installer]
2. Uruchom instalator i wybierz wersję ESP-IDF v5.2
3. Używaj "ESP-IDF Command Prompt" do budowania projektów

3.5.3 Weryfikacja instalacji

```
# Sprawdzenie wersji
idf.py --version
# Powinno wyświetlić: ESP-IDF v5.2.x

# Tworzenie projektu testowego
cd ~/esp
cp -r esp-idf/examples/get-started/hello_world .
cd hello_world
idf.py set-target esp32
idf.py build
```

3.6 Podsumowanie rozdziału

W tym rozdziale poznałeś:

1. **Dlaczego ESP32** - porównanie z alternatywami
2. **Architektura ESP32** - dual-core, peryferia, pamięć
3. **ESP32-CAM** - specyfikacja modułu, ograniczenia GPIO
4. **ESP-IDF** - struktura projektu, CMake, Kconfig
5. **Instalacja** - konfiguracja środowiska deweloperskiego

Kluczowe punkty:

- ✅ ESP32 to doskonały wybór dla IoT i robotyki
- ✅ ESP32-CAM ma ograniczone GPIO przez kamerę i SD
- ✅ ESP-IDF używa systemu komponentów
- ✅ Konfiguracja przez `idf.py menuconfig`
- ✅ Zawsze aktywuj środowisko przed budowaniem: `. ~/esp/esp-idf/export.sh`

(Kontynuacja w kolejnych rozdziałach...)

Dodatek A: Pełna Lista Referencji do Dokumentacji

ESP-IDF API Reference

Moduł	URL
GPIO Driver	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/gpio.html
LEDC (PWM)	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/ledc.html
WiFi	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_wifi.html
HTTP Server	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/esp_http_server.html
FreeRTOS	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html
ESP Timer	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_timer.html

Moduł	URL
ESP Event	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_event.html
NVS Flash	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html
SPIFFS	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/spiffs.html
Error Handling	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_err.html
Logging	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/log.html

Zewnętrzne biblioteki

Biblioteka	URL
ESP32-Camera	https://github.com/espressif/esp32-camera
cJSON	https://github.com/DaveGamble/cJSON
Unity Test	http://www.throwtheswitch.org/unity
FreeRTOS	https://www.freertos.org/Documentation.html

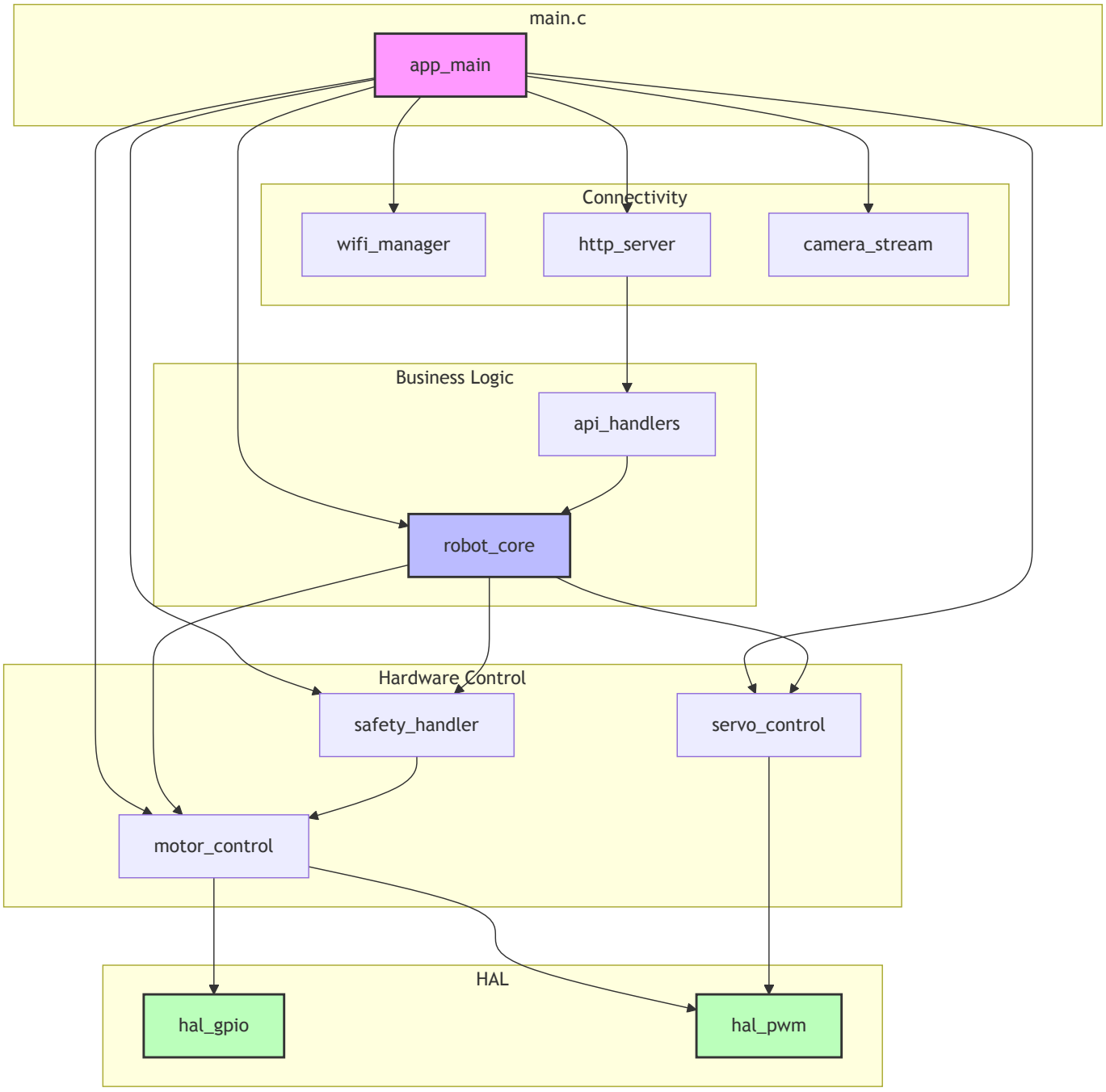
Datasheets

Komponent	URL
ESP32 Technical Reference	https://www.espressif.com/sites/default/files/documentation/esp32_technical_refe
DRV8833 Datasheet	https://www.ti.com/lit/ds/symlink/drv8833.pdf
SG90 Servo	http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sg90_datasheet.pdf
OV2640 Camera	https://www.uctronics.com/download/cam_module/OV2640DS.pdf

Dodatek B: Słownik Polsko-Angielski

Polski	English	Opis
Mikrokontroler	Microcontroller	Układ scalony z procesorem, pamięcią i peryferiami
Mostek H	H-Bridge	Układ do sterowania kierunkiem silnika DC
Wypełnienie	Duty Cycle	Procent czasu w stanie wysokim sygnału PWM
Szerokość impulsu	Pulse Width	Czas trwania impulsu w sygnale PWM
Strumień	Stream	Ciągły przepływ danych (np. MJPEG)
Strażnik	Watchdog	Timer resetujący system przy zawieszeniu
Punkt dostępowy	Access Point (AP)	Urządzenie tworzące sieć WiFi
Sterownik	Driver	Oprogramowanie kontrolujące sprzęt
Peryferium	Peripheral	Wbudowany moduł sprzętowy (GPIO, SPI...)
Przerwanie	Interrupt	Sygnał przerywający normalne wykonanie programu
Zadanie	Task	Niezależny wątek w FreeRTOS
Semafor	Semaphore	Mechanizm synchronizacji zadań
Muteks	Mutex	Semafor do ochrony zasobów
Callback	Callback	Funkcja wywoływana w odpowiedzi na zdarzenie
Inicjalizacja	Initialization	Konfiguracja początkowa modułu
Deserializacja	Deserialization	Konwersja danych (np. JSON) na strukturę

Dodatek C: Diagram Zależności Komponentów



Rozdział 4: Komponenty Sprzętowe Projektu

4.1 Przegląd systemu

Nasz robot składa się z następujących komponentów sprzętowych:

```
┌──────────────────────────────────────────────────────────────┐
│                          ROBOT TANK                          │
│                                                              │
│   ┌──────────────┐       ┌──────────────┐       ┌──────────┐
│   │  ESP32-CAM   │───────│   DRV8833    │───────│ 2x Silnik │
│   │  (mózg)      │       │ (sterownik)  │       │   DC     │
│   └──────────────┘       └──────────────┘       └──────────┘
│          │
```

```
│          │                                                    │
│          │                ┌──────────────┐       ┌──────────────┐      │
│          └────────────────│  Serwo SG90  │───────│   Wieża      │      │
│                           │  (obrót)     │       │  (kamera)    │      │
│                           └──────────────┘       └──────────────┘      │
│                                                                         │
│  Zasilanie: 5V dla ESP32, 3-10V dla silników                            │
└─────────────────────────────────────────────────────────────────────────┘
```

4.2 Sterownik silników DRV8833

4.2.1 Czym jest mostek H?

Mostek H to układ elektroniczny pozwalający na sterowanie kierunkiem prądu płynącego przez obciążenie (np. silnik DC). Nazwa pochodzi od kształtu schematu przypominającego literę H:

```
        VCC                     VCC
         │                       │
     ┌───┴───┐               ┌───┴───┐
     │  Q1   │               │  Q3   │
     │  SW   │               │  SW   │
     └───┬───┘               └───┬───┘
         │                       │
         ├──────[ MOTOR ]────────┤
         │                       │
     ┌───┴───┐               ┌───┴───┐
     │  Q2   │               │  Q4   │
     │  SW   │               │  SW   │
     └───┬───┘               └───┬───┘
         │                       │
        GND                     GND
```

Działanie:

- Q1+Q4 ON, Q2+Q3 OFF → Prąd płynie od lewej do prawej → Obrót w prawo
- Q2+Q3 ON, Q1+Q4 OFF → Prąd płynie od prawej do lewej → Obrót w lewo
- Wszystkie OFF → Silnik wolny (coast)
- Q1+Q3 lub Q2+Q4 ON → Hamowanie (brake)

4.2.2 Tablica prawdy DRV8833

DRV8833 to podwójny mostek H firmy Texas Instruments, sterowany sygnałami IN1/IN2 dla każdego kanału:

IN1	IN2	Tryb	Opis
LOW	LOW	Coast	Silnik wolny, bez hamowania
HIGH	LOW	Forward	Obrót w jednym kierunku
LOW	HIGH	Backward	Obrót w drugim kierunku
HIGH	HIGH	Brake	Aktywne hamowanie

4.2.3 Implementacja w kodzie

```
// Plik: components/robot_core/include/robot_types.h

// Linia 41-46: Enum odpowiadający tablicy prawdy
typedef enum {
    MOTOR_MODE_COAST = 0, /**< IN1=LOW,  IN2=LOW  - freewheeling */
    MOTOR_MODE_FORWARD,   /**< IN1=HIGH, IN2=LOW  */
    MOTOR_MODE_BACKWARD,  /**< IN1=LOW,  IN2=HIGH */
    MOTOR_MODE_BRAKE      /**< IN1=HIGH, IN2=HIGH */
} motor_mode_t;
```

```
// Plik: components/motor_control/src/motor_control.c

// Linia 28-48: Funkcja ustawiająca kierunek silnika
static void set_motor_direction(const motor_pins_t *motor, motor_mode_t
mode) {
    switch (mode) {
        case MOTOR_MODE_FORWARD:
            hal_gpio_set_level(motor->in1, 1);  // IN1 = HIGH
            hal_gpio_set_level(motor->in2, 0);  // IN2 = LOW
            break;
        case MOTOR_MODE_BACKWARD:
            hal_gpio_set_level(motor->in1, 0);  // IN1 = LOW
            hal_gpio_set_level(motor->in2, 1);  // IN2 = HIGH
            break;
        case MOTOR_MODE_BRAKE:
            hal_gpio_set_level(motor->in1, 1);  // IN1 = HIGH
            hal_gpio_set_level(motor->in2, 1);  // IN2 = HIGH
            break;
        case MOTOR_MODE_COAST:
        default:
            hal_gpio_set_level(motor->in1, 0);  // IN1 = LOW
            hal_gpio_set_level(motor->in2, 0);  // IN2 = LOW
            break;
```

```
    }
}
```

4.2.4 Schemat podłączenia DRV8833

```
ESP32-CAM                    DRV8833                    Silniki
─────────                    ───────                    ───────

GPIO 12 ──────────────────── AIN1 ──────┐
                                        ├──── OUT1 ────── Motor L (+)
GPIO 13 ──────────────────── AIN2 ──────┤
                                        └──── OUT2 ────── Motor L (-)

GPIO 14 ──────────────────── BIN1 ──────┐
                                        ├──── OUT1 ────── Motor R (+)
GPIO 15 ──────────────────── BIN2 ──────┤
                                        └──── OUT2 ────── Motor R (-)

GPIO 2  ──────────────────── nSLEEP/EEP (PWM dla prędkości)

3.3V    ──────────────────── VIN (logika)
VM (3-10V) ───────────────── VM (zasilanie silników)
GND     ──────────────────── GND
```

4.2.5 Soft-start (rampa PWM)

Problem: Nagłe włączenie silnika powoduje duży prąd rozruchowy, który może:

- Zakłócić zasilanie ESP32
- Uszkodzić sterownik
- Powodować szarpanie mechaniczne

Rozwiązanie: Stopniowe zwiększanie PWM (soft-start):

```
Duty Cycle %
    │
100 │                    ┌──────────────────
    │                  /
 75 │               /
    │             /
 50 │          /
    │        /
 25 │      /
    │    /
  0 │────┘
    └──────────────────────────────────────► Czas
```

```
        |        500ms          |
        Start                Pełna prędkość
```

4.3 Serwomechanizm SG90

4.3.1 Zasada działania serwa

Serwomechanizm to silnik z wbudowanym sterownikiem pozycji. Pozycja jest kontrolowana przez szerokość impulsu PWM:

```
                |◄──── 20ms (50Hz) ────►|
                |                       |
   500µs        |  ┌─┐                  |      → 0° (lewo)
                |  │ │                  |
                |  └─┘──────────────────|
                |                       |
  1500µs        |  ┌───┐               |      → 90° (środek)
                |  │   │               |
                |  └───┘───────────────|
                |                       |
  2400µs        |  ┌───────┐           |      → 180° (prawo)
                |  │       │           |
                |  └───────┘───────────|
```

4.3.2 Parametry SG90

Parametr	Wartość
Napięcie zasilania	4.8V - 6V
Moment obrotowy	1.8 kg·cm (4.8V)
Prędkość	60° w 0.1s
Zakres ruchu	0° - 180°
PWM	50 Hz (okres 20 ms)
Impulsy	500 µs - 2400 µs
Waga	9 g

4.3.3 Konwersja kąt → impulsu

```c
// Plik: components/servo_control/src/servo_control.c

// Linia 29-44: Konwersja kąta na szerokość impulsu
static uint16_t angle_to_pulse(uint8_t angle) {
```

```
    // Ograniczenie do zakresu
    if (angle < s_servo.config.min_angle) {
        angle = s_servo.config.min_angle;
    }
    if (angle > s_servo.config.max_angle) {
        angle = s_servo.config.max_angle;
    }

    // Obliczenie zakresów
    uint16_t pulse_range = s_servo.config.max_pulse_us -
                           s_servo.config.min_pulse_us;  // np. 2400-
500=1900
    uint8_t angle_range = s_servo.config.max_angle -
                          s_servo.config.min_angle;      // np. 180-0=180

    // Mapowanie liniowe
    // pulse = min_pulse + (angle/angle_range) * pulse_range
    uint16_t pulse = s_servo.config.min_pulse_us +
                     ((angle - s_servo.config.min_angle) * pulse_range)
                     / angle_range;

    return pulse;
}
```

Przykład obliczeń:

- Dla kąta 90°: `pulse = 500 + (90/180) × 1900 = 500 + 950 = 1450 µs`
- Dla kąta 45°: `pulse = 500 + (45/180) × 1900 = 500 + 475 = 975 µs`

4.3.4 Schemat podłączenia SG90

```
ESP32-CAM                SG90 Servo
─────────                ──────────

GPIO 16 ──────────────── Sygnał (pomarańczowy)
5V      ──────────────── VCC (czerwony)
GND     ──────────────── GND (brązowy)
```

4.4 Kamera OV2640

4.4.1 Specyfikacja

Parametr	Wartość
Rozdzielczość	Do 2MP (1600×1200)

Parametr	Wartość
Format wyjścia	JPEG, YUV, RGB
Interfejs	DVP (8-bit parallel)
SCCB (I2C)	Konfiguracja sensora
Częstotliwość	20 MHz XCLK

4.4.2 Pinout kamery (AI-Thinker ESP32-CAM)

```
// Plik: components/camera_stream/src/camera_stream.c

// Linia 96-122: Konfiguracja pinów kamery
camera_config_t cam_config = {
    .pin_pwdn = 32,       // Power down control
    .pin_reset = -1,      // Reset (nieużywany)
    .pin_xclk = 0,        // Master clock output
    .pin_sccb_sda = 26,   // I2C data
    .pin_sccb_scl = 27,   // I2C clock
    .pin_d7 = 35,         // Data bit 7
    .pin_d6 = 34,         // Data bit 6
    .pin_d5 = 39,         // Data bit 5
    .pin_d4 = 36,         // Data bit 4
    .pin_d3 = 21,         // Data bit 3
    .pin_d2 = 19,         // Data bit 2
    .pin_d1 = 18,         // Data bit 1
    .pin_d0 = 5,          // Data bit 0
    .pin_vsync = 25,      // Vertical sync
    .pin_href = 23,       // Horizontal reference
    .pin_pclk = 22,       // Pixel clock

    .xclk_freq_hz = 20000000,  // 20 MHz
    // ...
};
```

4.5 Schemat połączeń całego projektu

```
                    ┌──────────────────────────────────────┐
                    │              ESP32-CAM               │
                    │                                      │
                    │  [Kamera OV2640]                     │
                    │  (piny 0,5,18,19,21,22,23,25,26,27,  │
                    │   32,34,35,36,39 - zajęte)           │
```

```
                    │                                        │
 Motor L (+) ◄──────┤ GPIO 12 (AIN1)                         │
 Motor L (−) ◄──────┤ GPIO 13 (AIN2)      ┌──────────────┐   │
                    │                     │   DRV8833    │   │
 Motor R (+) ◄──────┤ GPIO 14 (BIN1) ─────┤              │   │
 Motor R (−) ◄──────┤ GPIO 15 (BIN2)      │   VIN=3.3V   │   │
                    │                     │   VM=5-10V   │   │
 PWM Enable  ◄──────┤ GPIO 2 (EEP) ───────┤   GND        │   │
                    │                     └──────────────┘   │
                    │                                        │
 Servo Signal◄──────┤ GPIO 16 ────────────[ SG90 Servo ]     │
                    │                                        │
                    │  3.3V ── Logika                        │
                    │  5V ─── Serwo                          │
                    │  GND ── Wspólna masa                   │
                    │                                        │
                    │  [UART] GPIO 1 (TX), GPIO 3 (RX)       │
                    │  (programowanie)                       │
                    │                                        │
                    │  [Flash LED] GPIO 4                    │
                    │                                        │
                    └────────────────────────────────────────┘
```

4.6 Ćwiczenie 4.1

Zadanie: Oblicz szerokość impulsu PWM potrzebną do ustawienia serwa na kąt 135°, jeśli:

- Minimum: 500 µs = 0°
- Maksimum: 2400 µs = 180°

Rozwiązanie:

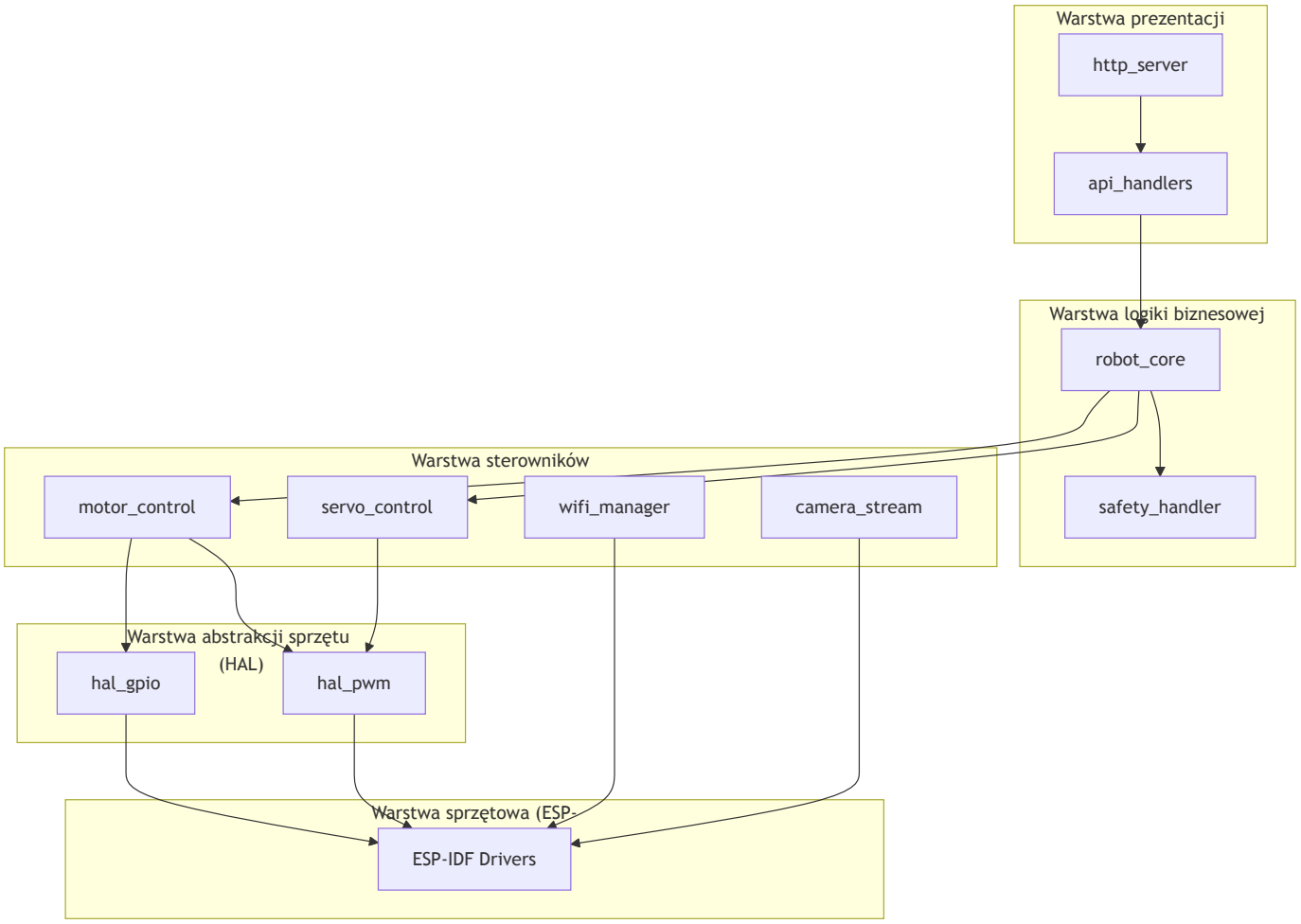
```
pulse_range = 2400 − 500 = 1900 µs
angle_range = 180 − 0 = 180°

pulse = 500 + (135/180) × 1900
pulse = 500 + 0.75 × 1900
pulse = 500 + 1425
pulse = 1925 µs
```

CZĘŚĆ II: BUDOWA PROJEKTU

Rozdział 5: Struktura Projektu i Konfiguracja

5.1 Architektura warstwowa

Nasz projekt używa architektury warstwowej, gdzie każda warstwa zależy tylko od warstw niższych:



Zalety tej architektury:

- **Separacja odpowiedzialności** - każdy moduł robi jedną rzecz
- **Testowalność** - możemy mockować warstwy niższe
- **Przenośność** - zmiana HAL pozwala na port na inną platformę
- **Czytelność** - łatwo zrozumieć zależności

5.2 Punkt wejścia - main.c

```
// Plik: main/main.c (kompletny)

/**
 * @file main.c
 * @brief ESP32-CAM Robot Controller Entry Point
 */
```

```c
#include <stdio.h>
#include <string.h>

#include "app_config.h"        // Konfiguracja z Kconfig
#include "camera_stream.h"
#include "esp_err.h"
#include "esp_log.h"
#include "esp_spiffs.h"
#include "esp_system.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "http_server.h"
#include "motor_control.h"
#include "nvs_flash.h"
#include "robot.h"
#include "safety_handler.h"
#include "servo_control.h"
#include "wifi_manager.h"

static const char *TAG = "main";
```

Wyjaśnienie linia po linii:

Linia	Opis
1-7	Komentarz dokumentujący plik
9-10	Standardowe nagłówki C
12-24	Nagłówki komponentów projektu
26	Tag do logowania - identyfikuje źródło komunikatów

5.2.1 Funkcje inicjalizacyjne

```c
// Linia 37-44: Inicjalizacja NVS (Non-Volatile Storage)
static esp_err_t init_nvs(void) {
    esp_err_t ret = nvs_flash_init();
    // Jeśli NVS jest uszkodzony lub ma starą wersję - wyczyść i
zainicjalizuj ponownie
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
        ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ret = nvs_flash_init();
    }
```

```
    return ret;
}
```

NVS (Non-Volatile Storage)[27] to system przechowywania danych klucz-wartość w pamięci flash. Jest używany przez WiFi do zapisywania kalibracji i przez inne komponenty.

```
// Linia 49-76: Inicjalizacja SPIFFS (system plików)
static esp_err_t init_spiffs(void) {
    ESP_LOGI(TAG, "Initializing SPIFFS");

    // Konfiguracja SPIFFS
    esp_vfs_spiffs_conf_t conf = {
        .base_path = APP_SPIFFS_BASE_PATH,  // np. "/spiffs"
        .partition_label = NULL,             // Domyślna partycja
        .max_files = APP_SPIFFS_MAX_FILES,   // Maks. otwartych plików
        .format_if_mount_failed = false      // Nie formatuj automatycznie
    };

    esp_err_t ret = esp_vfs_spiffs_register(&conf);

    if (ret != ESP_OK) {
        if (ret == ESP_FAIL) {
            ESP_LOGE(TAG, "Failed to mount SPIFFS");
        } else if (ret == ESP_ERR_NOT_FOUND) {
            ESP_LOGE(TAG, "SPIFFS partition not found");
        } else {
            ESP_LOGE(TAG, "SPIFFS init failed: %s", esp_err_to_name(ret));
        }
        return ret;
    }

    // Wyświetl informacje o zajętości
    size_t total = 0, used = 0;
    ret = esp_spiffs_info(NULL, &total, &used);
    if (ret == ESP_OK) {
        ESP_LOGI(TAG, "SPIFFS: total=%zu, used=%zu", total, used);
    }

    return ESP_OK;
}
```

SPIFFS (SPI Flash File System)[28] to prosty system plików dla pamięci flash. Używamy go do przechowywania plików statycznych (HTML, CSS, JS) dla interfejsu webowego.

5.2.2 Funkcja app_main()

```
// Linia 170-215: Główna funkcja aplikacji
void app_main(void) {
    ESP_LOGI(TAG, "ESP32-CAM Robot Controller starting...");
    ESP_LOGI(TAG, "Mock mode: %s", APP_MOCK_MODE ? "enabled" : "disabled");

    /* Inicjalizacja NVS */
    ESP_ERROR_CHECK(init_nvs());

    /* Inicjalizacja SPIFFS dla web UI */
    ESP_ERROR_CHECK(init_spiffs());

    /* Inicjalizacja WiFi AP (pomiń w mock mode) */
    if (!APP_MOCK_MODE) {
        ESP_ERROR_CHECK(wifi_manager_init());
        ESP_ERROR_CHECK(wifi_manager_start_ap(APP_WIFI_SSID,
APP_WIFI_PASSWORD));
        ESP_LOGI(TAG, "WiFi AP started");
    } else {
        ESP_LOGI(TAG, "WiFi initialization skipped (mock mode)");
    }

    /* Inicjalizacja podsystemów sprzętowych */
    if (!APP_MOCK_MODE) {
        ESP_ERROR_CHECK(init_motors());
        ESP_ERROR_CHECK(init_servo());
        ESP_ERROR_CHECK(init_camera());
    } else {
        ESP_LOGI(TAG, "Hardware initialization skipped (mock mode)");
    }

    /* Inicjalizacja robot core */
    ESP_ERROR_CHECK(init_robot());

    /* Inicjalizacja safety handler */
    ESP_ERROR_CHECK(init_safety());

    /* Uruchomienie serwera HTTP */
    ESP_ERROR_CHECK(start_server());

    ESP_LOGI(TAG, "Robot controller initialized successfully");
    ESP_LOGI(TAG, "Web UI available at http://10.42.0.1:%d/",
APP_HTTP_PORT);
```

```
    /* Główna pętla - karmienie watchdoga */
    while (1) {
        safety_feed_watchdog();
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

Kolejność inicjalizacji jest ważna:

1. **NVS** - wymagany przez WiFi
2. **SPIFFS** - potrzebny przed startem HTTP (pliki statyczne)
3. **WiFi** - potrzebny przed HTTP (sieć)
4. **Sprzęt** - silniki, serwo, kamera
5. **Robot core** - logika biznesowa
6. **Safety** - watchdog musi być ostatni
7. **HTTP server** - uruchamiamy gdy wszystko gotowe

5.3 Konfiguracja - sdkconfig.defaults

```
# Plik: sdkconfig.defaults

# Target ESP32 (AI-Thinker ESP32-CAM)
CONFIG_IDF_TARGET="esp32"

# Flash configuration - 4MB, tryb QIO (szybszy)
CONFIG_ESPTOOLPY_FLASHSIZE_4MB=y
CONFIG_ESPTOOLPY_FLASHMODE_QIO=y

# Własna tablica partycji
CONFIG_PARTITION_TABLE_CUSTOM=y
CONFIG_PARTITION_TABLE_CUSTOM_FILENAME="partitions.csv"

# PSRAM - ESP32-CAM ma 4MB zewnętrznej RAM
CONFIG_ESP32_SPIRAM_SUPPORT=y
CONFIG_SPIRAM_TYPE_AUTO=y
CONFIG_SPIRAM_SPEED_80M=y

# Konfiguracja kamery
CONFIG_CAMERA_TASK_PINNED_TO_CORE=0
CONFIG_CAMERA_CORE0_TASK_PRIORITY=2
CONFIG_CAMERA_GRAB_WHEN_EMPTY=y
```

```
# HTTP Server - większe bufory
CONFIG_HTTPD_MAX_REQ_HDR_LEN=1024
CONFIG_HTTPD_MAX_URI_LEN=512

# WiFi Access Point
CONFIG_ROBOT_WIFI_SSID="ESP32"
CONFIG_ROBOT_WIFI_PASSWORD="eEspetrzyjsci2a"
CONFIG_ROBOT_WIFI_MAX_CONNECTIONS=4

# Port HTTP
CONFIG_ROBOT_HTTP_PORT=4567

# FreeRTOS - 1000 tików/sekundę dla precyzji
CONFIG_FREERTOS_HZ=1000

# Logging
CONFIG_LOG_DEFAULT_LEVEL_INFO=y
CONFIG_LOG_MAXIMUM_LEVEL_DEBUG=y

# Watchdog timeout
CONFIG_ESP_TASK_WDT_TIMEOUT_S=10
```

5.4 Tablica partycji

```
# Plik: partitions.csv

# Name,   Type, SubType, Offset,  Size,    Flags
nvs,      data, nvs,     0x9000,  0x6000,
phy_init, data, phy,     0xf000,  0x1000,
factory,  app,  factory, 0x10000, 0x300000,
spiffs,   data, spiffs,  0x310000,0xF0000,
```

Wyjaśnienie:

Partycja	Typ	Offset	Rozmiar	Opis
nvs	data/nvs	0x9000	24 KB	Non-Volatile Storage
phy_init	data/phy	0xf000	4 KB	Dane kalibracji WiFi
factory	app	0x10000	3 MB	Nasza aplikacja
spiffs	data/spiffs	0x310000	960 KB	System plików (web UI)

Rozdział 6: Komponent robot_hal - Warstwa Abstrakcji Sprzętu

6.1 Po co warstwa HAL?

HAL (Hardware Abstraction Layer) izoluje kod od konkretnego sprzętu. Dzięki temu:

1. **Testowalność** - możemy podmienić HAL na mock w testach
2. **Przenośność** - zmiana platformy = zmiana HAL, reszta bez zmian
3. **Czytelność** - ukrywamy szczegóły ESP-IDF

6.2 hal_types.h - Definicje typów

```
// Plik: components/robot_hal/include/hal_types.h (kompletny)

/**
 * @file hal_types.h
 * @brief Hardware Abstraction Layer type definitions
 */

#ifndef HAL_TYPES_H
#define HAL_TYPES_H

#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

/**
 * @brief PWM channel handle
 */
typedef uint8_t hal_pwm_channel_t;

/**
 * @brief Invalid PWM channel marker
 */
#define HAL_PWM_CHANNEL_INVALID 0xFF

/**
 * @brief Maximum number of PWM channels
 */
#define HAL_PWM_MAX_CHANNELS 8
```

```
/**
 * @brief PWM duty cycle maximum value (100%)
 */
#define HAL_PWM_DUTY_MAX 100

/**
 * @brief Servo PWM frequency (Hz)
 */
#define HAL_SERVO_PWM_FREQ 50

#ifdef __cplusplus
}
#endif

#endif /* HAL_TYPES_H */
```

Wyjaśnienie linia po linii:

Linia	Kod	Wyjaśnienie
6-7	`#ifndef HAL_TYPES_H` / `#define HAL_TYPES_H`	Include guard
9-10	`#include <stdbool.h>` / `<stdint.h>`	Typy bool i uint8_t
12-14	`#ifdef __cplusplus` / `extern "C" {`	Kompatybilność z C++
19	`typedef uint8_t hal_pwm_channel_t`	Typ reprezentujący kanał PWM
24	`#define HAL_PWM_CHANNEL_INVALID 0xFF`	Wartość specjalna = brak kanału
29	`#define HAL_PWM_MAX_CHANNELS 8`	ESP32 ma 8 kanałów LEDC
34	`#define HAL_PWM_DUTY_MAX 100`	Duty cycle wyrażony w procentach
39	`#define HAL_SERVO_PWM_FREQ 50`	Standardowa częstotliwość serwa

6.3 hal_gpio.h - Interfejs GPIO

```
// Plik: components/robot_hal/include/hal_gpio.h (kompletny)

/**
 * @file hal_gpio.h
 * @brief GPIO Hardware Abstraction Layer
 */
```

```c
#ifndef HAL_GPIO_H
#define HAL_GPIO_H

#include <esp_err.h>
#include <driver/gpio.h>
#include "hal_types.h"

#ifdef __cplusplus
extern "C" {
#endif

/**
 * @brief Initialize GPIO pin as output
 * @param pin GPIO pin number
 * @return ESP_OK on success
 */
esp_err_t hal_gpio_init_output(gpio_num_t pin);

/**
 * @brief Set GPIO pin output level
 * @param pin GPIO pin number
 * @param level Output level (0 = LOW, 1 = HIGH)
 * @return ESP_OK on success
 */
esp_err_t hal_gpio_set_level(gpio_num_t pin, uint8_t level);

/**
 * @brief Get GPIO pin input level
 * @param pin GPIO pin number
 * @return Pin level (0 or 1)
 */
uint8_t hal_gpio_get_level(gpio_num_t pin);

/**
 * @brief Reset GPIO pin to default state (LOW)
 * @param pin GPIO pin number
 */
void hal_gpio_reset(gpio_num_t pin);

/**
 * @brief Reset multiple GPIO pins
 * @param pins Array of GPIO pin numbers
```

```
 * @param count Number of pins
 */
void hal_gpio_reset_multiple(const gpio_num_t *pins, size_t count);

#ifdef __cplusplus
}
#endif

#endif /* HAL_GPIO_H */
```

6.4 hal_gpio.c - Implementacja GPIO

```
// Plik: components/robot_hal/src/hal_gpio.c (kompletny)

/**
 * @file hal_gpio.c
 * @brief GPIO HAL implementation using ESP-IDF driver
 */

#include "hal_gpio.h"
#include <esp_log.h>

static const char *TAG = "hal_gpio";

esp_err_t hal_gpio_init_output(gpio_num_t pin) {
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << pin),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };

    esp_err_t ret = gpio_config(&io_conf);

    if (ret == ESP_OK) {
        gpio_set_level(pin, 0);
        ESP_LOGD(TAG, "GPIO %d initialized as output", pin);
    } else {
        ESP_LOGE(TAG, "Failed to init GPIO %d: %s", pin,
esp_err_to_name(ret));
    }

    return ret;
```

```c
}

esp_err_t hal_gpio_set_level(gpio_num_t pin, uint8_t level) {
    esp_err_t ret = gpio_set_level(pin, level ? 1 : 0);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to set GPIO %d to %d: %s",
                 pin, level, esp_err_to_name(ret));
    }
    return ret;
}

uint8_t hal_gpio_get_level(gpio_num_t pin) {
    return gpio_get_level(pin) ? 1 : 0;
}

void hal_gpio_reset(gpio_num_t pin) {
    gpio_set_level(pin, 0);
    ESP_LOGD(TAG, "GPIO %d reset to LOW", pin);
}

void hal_gpio_reset_multiple(const gpio_num_t *pins, size_t count) {
    for (size_t i = 0; i < count; i++) {
        hal_gpio_reset(pins[i]);
    }
    ESP_LOGD(TAG, "Reset %zu GPIO pins", count);
}
```

6.5 hal_pwm.c - Implementacja PWM

```c
// Plik: components/robot_hal/src/hal_pwm.c (kompletny z adnotacjami)

#include "hal_pwm.h"
#include <esp_log.h>
#include <driver/ledc.h>

static const char *TAG = "hal_pwm";

// Konfiguracja LEDC
#define LEDC_TIMER_RESOLUTION LEDC_TIMER_13_BIT  // 13 bitów = 8192 poziomów
#define LEDC_MAX_DUTY         ((1 << LEDC_TIMER_RESOLUTION) - 1)  // 8191

// Śledzenie przydzielonych kanałów
static bool channel_used[HAL_PWM_MAX_CHANNELS] = {false};
static uint32_t channel_freq[HAL_PWM_MAX_CHANNELS] = {0};
```

```
// Znajdź wolny kanał
static hal_pwm_channel_t find_free_channel(void) {
    for (uint8_t i = 0; i < HAL_PWM_MAX_CHANNELS; i++) {
        if (!channel_used[i]) {
            return i;
        }
    }
    return HAL_PWM_CHANNEL_INVALID;
}

esp_err_t hal_pwm_init(gpio_num_t pin, uint32_t frequency_hz,
                       hal_pwm_channel_t *channel) {
    // Walidacja argumentów
    if (channel == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    // Przydziel kanał
    hal_pwm_channel_t ch = find_free_channel();
    if (ch == HAL_PWM_CHANNEL_INVALID) {
        ESP_LOGE(TAG, "No free PWM channels available");
        return ESP_ERR_NO_MEM;
    }

    // Konfiguracja timera LEDC
    // ESP32 ma 4 timery, każdy obsługuje 2 kanały
    ledc_timer_config_t timer_conf = {
        .speed_mode = LEDC_LOW_SPEED_MODE,     // Tryb low-speed
        .timer_num = (ledc_timer_t)(ch / 2),   // Timer 0-3
        .duty_resolution = LEDC_TIMER_RESOLUTION,
        .freq_hz = frequency_hz,
        .clk_cfg = LEDC_AUTO_CLK               // Automatyczny wybór źródła
zegara
    };

    esp_err_t ret = ledc_timer_config(&timer_conf);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "LEDC timer config failed: %s", esp_err_to_name(ret));
        return ret;
    }

    // Konfiguracja kanału LEDC
```

```
    ledc_channel_config_t channel_conf = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .channel = (ledc_channel_t)ch,
        .timer_sel = (ledc_timer_t)(ch / 2),
        .intr_type = LEDC_INTR_DISABLE,
        .gpio_num = pin,
        .duty = 0,                              // Start z 0%
        .hpoint = 0
    };

    ret = ledc_channel_config(&channel_conf);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "LEDC channel config failed: %s",
esp_err_to_name(ret));
        return ret;
    }

    // Zapisz stan
    channel_used[ch] = true;
    channel_freq[ch] = frequency_hz;
    *channel = ch;

    ESP_LOGI(TAG, "PWM channel %d initialized on GPIO %d at %lu Hz",
             ch, pin, frequency_hz);

    return ESP_OK;
}

esp_err_t hal_pwm_set_duty(hal_pwm_channel_t channel, uint8_t duty_percent)
{
    if (!hal_pwm_is_valid(channel)) {
        return ESP_ERR_INVALID_ARG;
    }

    // Clamp do 100%
    if (duty_percent > HAL_PWM_DUTY_MAX) {
        duty_percent = HAL_PWM_DUTY_MAX;
    }

    // Konwersja procent → wartość rejestru (0-8191)
    uint32_t duty = (LEDC_MAX_DUTY * duty_percent) / HAL_PWM_DUTY_MAX;

    esp_err_t ret = ledc_set_duty(LEDC_LOW_SPEED_MODE,
```

```
(ledc_channel_t)channel, duty);
    if (ret == ESP_OK) {
        ret = ledc_update_duty(LEDC_LOW_SPEED_MODE,
(ledc_channel_t)channel);
    }

    return ret;
}

esp_err_t hal_pwm_set_servo_pulse(hal_pwm_channel_t channel, uint16_t
pulse_us) {
    if (!hal_pwm_is_valid(channel)) {
        return ESP_ERR_INVALID_ARG;
    }

    // Dla serwa przy 50Hz: okres = 20000µs
    // duty = (pulse_us / period_us) * max_duty
    uint32_t period_us = 1000000 / channel_freq[channel];
    uint32_t duty = (LEDC_MAX_DUTY * pulse_us) / period_us;

    if (duty > LEDC_MAX_DUTY) {
        duty = LEDC_MAX_DUTY;
    }

    esp_err_t ret = ledc_set_duty(LEDC_LOW_SPEED_MODE,
(ledc_channel_t)channel, duty);
    if (ret == ESP_OK) {
        ret = ledc_update_duty(LEDC_LOW_SPEED_MODE,
(ledc_channel_t)channel);
    }

    return ret;
}

void hal_pwm_stop(hal_pwm_channel_t channel) {
    if (!hal_pwm_is_valid(channel)) {
        return;
    }
    ledc_set_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)channel, 0);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, (ledc_channel_t)channel);
}

void hal_pwm_cleanup(hal_pwm_channel_t channel) {
```

```
    if (!hal_pwm_is_valid(channel)) {
        return;
    }
    hal_pwm_stop(channel);
    ledc_stop(LEDC_LOW_SPEED_MODE, (ledc_channel_t)channel, 0);
    channel_used[channel] = false;
    channel_freq[channel] = 0;
    ESP_LOGI(TAG, "PWM channel %d cleaned up", channel);
}

bool hal_pwm_is_valid(hal_pwm_channel_t channel) {
    return channel < HAL_PWM_MAX_CHANNELS && channel_used[channel];
}
```

Rozdział 7: Komponent motor_control - Sterowanie Silnikami

7.1 Wprowadzenie

Komponent `motor_control` odpowiada za sterowanie dwoma silnikami DC poprzez sterownik DRV8833. Jest to jeden z najważniejszych komponentów projektu - od niego zależy mobilność robota.

Główne funkcje komponentu:

- Sterowanie kierunkiem silników (przód, tył, obrót)
- Soft-start - płynne narastanie prędkości
- Zatrzymanie awaryjne

Zależności:

- `robot_hal` - abstrakcja GPIO i PWM
- `robot_types.h` - definicje enum kierunków

7.2 Plik nagłówkowy motor_control.h

```
// Plik: components/motor_control/include/motor_control.h
// Linia 1-8: Nagłówek dokumentacji i include guards
/**
 * @file motor_control.h
 * @brief DRV8833 motor driver control interface
 *
 * Controls two DC motors via DRV8833 H-bridge drivers with
 * shared PWM enable for soft-start ramping.
 */
```

```
#ifndef MOTOR_CONTROL_H  // Include guard - zapobiega wielokrotnemu
włączeniu
#define MOTOR_CONTROL_H  // Definiujemy makro MOTOR_CONTROL_H

// Linia 10-18: Includy systemowe
#include <esp_err.h>         // Typy błędów ESP-IDF (ESP_OK, ESP_ERR_*)
#include <driver/gpio.h>     // Typ gpio_num_t
#include <stdint.h>          // Typy uint32_t, uint8_t
#include "robot_types.h"     // Enum motor_mode_t

#ifdef __cplusplus           // Kompatybilność z C++
extern "C" {
#endif

// Linia 24-30: Struktura pinów jednego silnika
/**
 * @brief Motor pin configuration
 */
typedef struct {
    gpio_num_t in1; /**< Forward direction pin */   // Pin IN1 sterownika
    gpio_num_t in2; /**< Backward direction pin */  // Pin IN2 sterownika
} motor_pins_t;

// Linia 35-42: Struktura konfiguracji całego modułu
/**
 * @brief Motor control configuration
 */
typedef struct {
    motor_pins_t left_motor;    /**< Left motor pins */     // Piny lewego
silnika
    motor_pins_t right_motor;   /**< Right motor pins */    // Piny prawego
silnika
    gpio_num_t enable_pin;      /**< Shared PWM enable pin */ // Wspólny pin
PWM
    uint32_t pwm_frequency_hz;  /**< PWM frequency */       // Częstotliwość
PWM (np. 1000 Hz)
    uint32_t ramp_duration_ms;  /**< Soft-start ramp time */ // Czas rampy
(np. 500 ms)
    uint8_t ramp_steps;         /**< Number of ramp steps */ // Liczba
kroków rampy (np. 25)
} motor_control_config_t;
```

```
// Linia 52: Inicjalizacja modułu
esp_err_t motor_control_init(const motor_control_config_t *config);

// Linia 60-84: Funkcje ruchu
esp_err_t motor_move_forward(uint32_t duration_ms);   // Jazda do przodu
esp_err_t motor_move_backward(uint32_t duration_ms);  // Jazda do tyłu
esp_err_t motor_turn_left(uint32_t duration_ms);      // Skręt w lewo (tank
turn)
esp_err_t motor_turn_right(uint32_t duration_ms);     // Skręt w prawo (tank
turn)

// Linia 93: Stop awaryjny
esp_err_t motor_stop_all(void);

// Linia 98: Cleanup
void motor_control_cleanup(void);

#ifdef __cplusplus
}
#endif

#endif /* MOTOR_CONTROL_H */
```

7.2.1 Wyjaśnienie struktury motor_pins_t

Struktura `motor_pins_t` reprezentuje dwa piny GPIO sterujące jednym silnikiem przez mostek H:

Pin	Funkcja	Opis
`in1`	Kierunek 1	HIGH dla ruchu w przód
`in2`	Kierunek 2	HIGH dla ruchu w tył

7.2.2 Tablica prawdy DRV8833

Sterownik DRV8833 obsługuje 4 tryby pracy dla każdego silnika:

IN1	IN2	Tryb	Opis
LOW	LOW	Coast	Silnik się kręci swobodnie
HIGH	LOW	Forward	Silnik kręci się do przodu
LOW	HIGH	Backward	Silnik kręci się do tyłu
HIGH	HIGH	Brake	Silnik jest zahamowany

7.3 Implementacja motor_control.c

```c
// Plik: components/motor_control/src/motor_control.c
// Linia 1-5: Nagłówek dokumentacji
/**
 * @file motor_control.c
 * @brief DRV8833 motor driver implementation
 */

// Linia 6-14: Includy
#include "motor_control.h"    // Nasz nagłówek (ZAWSZE pierwszy!)
#include <esp_log.h>          // ESP_LOGI, ESP_LOGE, ESP_LOGW, ESP_LOGD
#include <string.h>           // memcpy()
#include "hal_gpio.h"         // hal_gpio_init_output, hal_gpio_set_level
#include "hal_pwm.h"          // hal_pwm_init, hal_pwm_set_duty
#include "pwm_ramper.h"       // pwm_ramper_init, pwm_ramper_start

// Linia 16: TAG dla logów
static const char *TAG = "motor_control";
// TAG to identyfikator modułu widoczny w logach szeregowych:
// I (1234) motor_control: Motor control initialized
// ^         ^             ^
// poziom    TAG           wiadomość

// Linia 19-23: Statyczna struktura stanu modułu
static struct {
    bool initialized;              // Czy moduł został zainicjalizowany?
    motor_control_config_t config; // Kopia konfiguracji
    hal_pwm_channel_t pwm_channel; // Numer kanału PWM
} s_motor = {0};                   // Zerowanie wszystkich pól (C99)

// s_motor to "singleton" - tylko jedna instancja w całym programie
// Prefiks s_ oznacza "static" (konwencja ESP-IDF)
```

7.3.1 Funkcja pomocnicza - ustawianie kierunku

```c
// Linia 28-48: Funkcja ustawiająca kierunek silnika
/**
 * @brief Set motor direction using DRV8833 truth table
 */
static void set_motor_direction(const motor_pins_t *motor, motor_mode_t
mode) {
    // 'static' oznacza że funkcja jest prywatna dla tego pliku
    // Nie jest widoczna na zewnątrz (nie ma jej w .h)
```

```
    switch (mode) {
        case MOTOR_MODE_FORWARD:
            // IN1=HIGH, IN2=LOW → silnik kręci się do przodu
            hal_gpio_set_level(motor->in1, 1);
            hal_gpio_set_level(motor->in2, 0);
            break;

        case MOTOR_MODE_BACKWARD:
            // IN1=LOW, IN2=HIGH → silnik kręci się do tyłu
            hal_gpio_set_level(motor->in1, 0);
            hal_gpio_set_level(motor->in2, 1);
            break;

        case MOTOR_MODE_BRAKE:
            // IN1=HIGH, IN2=HIGH → hamowanie aktywne
            hal_gpio_set_level(motor->in1, 1);
            hal_gpio_set_level(motor->in2, 1);
            break;

        case MOTOR_MODE_COAST:
        default:
            // IN1=LOW, IN2=LOW → silnik się kręci swobodnie
            hal_gpio_set_level(motor->in1, 0);
            hal_gpio_set_level(motor->in2, 0);
            break;
    }
}
```

7.3.2 Inicjalizacja modułu

```
// Linia 50-102: Funkcja inicjalizacji
esp_err_t motor_control_init(const motor_control_config_t *config) {
    // Walidacja parametru - zawsze sprawdzaj wskaźniki!
    if (config == NULL) {
        return ESP_ERR_INVALID_ARG;  // Błąd: nieprawidłowy argument
    }

    esp_err_t ret;  // Zmienna na kod błędu

    // === Inicjalizacja GPIO lewego silnika ===
    ret = hal_gpio_init_output(config->left_motor.in1);
    if (ret != ESP_OK) return ret;  // Wczesny return przy błędzie
```

```c
    ret = hal_gpio_init_output(config->left_motor.in2);
    if (ret != ESP_OK) return ret;

    // === Inicjalizacja GPIO prawego silnika ===
    ret = hal_gpio_init_output(config->right_motor.in1);
    if (ret != ESP_OK) return ret;

    ret = hal_gpio_init_output(config->right_motor.in2);
    if (ret != ESP_OK) return ret;

    // === Inicjalizacja PWM dla pinu enable ===
    ret = hal_pwm_init(
        config->enable_pin,        // GPIO dla PWM
        config->pwm_frequency_hz,  // Częstotliwość (np. 1000 Hz)
        &s_motor.pwm_channel       // [OUT] Przydzielony kanał
    );
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "PWM init failed");
        return ret;
    }

    // === Inicjalizacja rampera PWM ===
    // Ramper zapewnia płynne narastanie prędkości (soft-start)
    pwm_ramper_config_t ramper_config = {
        .ramp_duration_ms = config->ramp_duration_ms,  // np. 500 ms
        .num_steps = config->ramp_steps,               // np. 25 kroków
        .max_duty_percent = 100                        // Maksymalnie 100%
    };

    ret = pwm_ramper_init(s_motor.pwm_channel, &ramper_config);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "PWM ramper init failed");
        return ret;
    }

    // === Zapisanie stanu ===
    memcpy(&s_motor.config, config, sizeof(motor_control_config_t));
    // memcpy kopiuje blok pamięci:
    // - dst: &s_motor.config (gdzie kopiować)
    // - src: config (skąd kopiować)
    // - size: sizeof(...) (ile bajtów)

    s_motor.initialized = true;
```

```
    // === Ustawienie początkowe: coast mode ===
    motor_stop_all();

    // Log informacyjny
    ESP_LOGI(TAG, "Motor control initialized (freq=%lu Hz, ramp=%lu ms)",
             config->pwm_frequency_hz, config->ramp_duration_ms);

    return ESP_OK;  // Sukces!
}
```

> **Dokumentacja ESP-IDF:**
>
> - esp_err.h - Error Codes

7.3.3 Funkcje ruchu

```
// Linia 104-116: Jazda do przodu
esp_err_t motor_move_forward(uint32_t duration_ms) {
    // Sprawdzenie czy moduł jest zainicjalizowany
    if (!s_motor.initialized) {
        return ESP_ERR_INVALID_STATE;  // Błąd: nieprawidłowy stan
    }

    // Log debug (widoczny tylko przy poziomie DEBUG)
    ESP_LOGD(TAG, "Moving forward (duration=%lu ms)", duration_ms);

    // Ustaw kierunek obu silników na FORWARD
    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_FORWARD);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_FORWARD);

    // Uruchom rampę PWM (soft-start)
    pwm_ramper_start();

    return ESP_OK;
}

// Linia 118-130: Jazda do tyłu
esp_err_t motor_move_backward(uint32_t duration_ms) {
    if (!s_motor.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    ESP_LOGD(TAG, "Moving backward (duration=%lu ms)", duration_ms);
```

```
    // Oba silniki w trybie BACKWARD
    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_BACKWARD);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_BACKWARD);
    pwm_ramper_start();

    return ESP_OK;
}

// Linia 132-145: Skręt w lewo (tank turn)
esp_err_t motor_turn_left(uint32_t duration_ms) {
    if (!s_motor.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    ESP_LOGD(TAG, "Turning left (duration=%lu ms)", duration_ms);

    // Tank turn: lewy do tyłu, prawy do przodu
    // Robot obraca się w miejscu!
    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_BACKWARD);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_FORWARD);
    pwm_ramper_start();

    return ESP_OK;
}

// Linia 147-160: Skręt w prawo (tank turn)
esp_err_t motor_turn_right(uint32_t duration_ms) {
    if (!s_motor.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    ESP_LOGD(TAG, "Turning right (duration=%lu ms)", duration_ms);

    // Tank turn: lewy do przodu, prawy do tyłu
    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_FORWARD);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_BACKWARD);
    pwm_ramper_start();

    return ESP_OK;
}
```

7.3.4 Stop i cleanup

```c
// Linia 162-177: Zatrzymanie silników
esp_err_t motor_stop_all(void) {
    if (!s_motor.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    ESP_LOGD(TAG, "Stopping all motors");

    // Zatrzymaj rampę PWM (ustawia PWM na 0%)
    pwm_ramper_stop();

    // Ustaw oba silniki w tryb coast (swobodny obrót)
    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_COAST);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_COAST);

    return ESP_OK;
}

// Linia 179-196: Sprzątanie zasobów
void motor_control_cleanup(void) {
    if (!s_motor.initialized) {
        return;
    }

    ESP_LOGI(TAG, "Motor control cleanup");

    // Zatrzymaj silniki
    motor_stop_all();

    // Zwolnij zasoby rampera
    pwm_ramper_cleanup();

    // Zwolnij kanał PWM
    hal_pwm_cleanup(s_motor.pwm_channel);

    // Zresetuj piny GPIO do stanu domyślnego
    gpio_num_t pins[] = {
        s_motor.config.left_motor.in1,
        s_motor.config.left_motor.in2,
        s_motor.config.right_motor.in1,
        s_motor.config.right_motor.in2
    };
    hal_gpio_reset_multiple(pins, 4);
```

```
    s_motor.initialized = false;
}
```

7.4 Moduł PWM Ramper - Soft Start

7.4.1 Po co soft-start?

Gdy silnik DC startuje od razu z pełną mocą, występują następujące problemy:

1. **Szarpnięcie mechaniczne** - robot "skacze"
2. **Spike prądowy** - może uszkodzić zasilanie
3. **Hałas** - głośny start silników

Soft-start rozwiązuje te problemy przez stopniowe zwiększanie PWM od 0% do 100%.

7.4.2 Plik nagłówkowy pwm_ramper.h

```
// Plik: components/motor_control/include/pwm_ramper.h
/**
 * @file pwm_ramper.h
 * @brief PWM soft-start ramper for motor control
 *
 * Provides gradual PWM duty cycle ramping to prevent inrush
 * current spikes when starting motors.
 */

#ifndef PWM_RAMPER_H
#define PWM_RAMPER_H

#include <esp_err.h>
#include <stdbool.h>
#include <stdint.h>
#include "hal_pwm.h"  // Typ hal_pwm_channel_t

#ifdef __cplusplus
extern "C" {
#endif

// Konfiguracja rampera
typedef struct {
    uint32_t ramp_duration_ms;  // Całkowity czas rampy (np. 500 ms)
    uint8_t num_steps;          // Liczba kroków (np. 25)
    uint8_t max_duty_percent;   // Maksymalny duty cycle (0-100)
} pwm_ramper_config_t;
```

```c
// Inicjalizacja - tworzy task FreeRTOS
esp_err_t pwm_ramper_init(hal_pwm_channel_t channel, const
pwm_ramper_config_t *config);

// Start rampy (nieblokujący)
void pwm_ramper_start(void);

// Stop rampy i ustawienie PWM na 0
void pwm_ramper_stop(void);

// Ustawienie natychmiastowego duty (bez rampy)
void pwm_ramper_set_duty(uint8_t duty_percent);

// Sprawdzenie czy rampa jest aktywna
bool pwm_ramper_is_active(void);

// Zwolnienie zasobów
void pwm_ramper_cleanup(void);

#ifdef __cplusplus
}
#endif

#endif /* PWM_RAMPER_H */
```

7.4.3 Implementacja pwm_ramper.c

```c
// Plik: components/motor_control/src/pwm_ramper.c
/**
 * @file pwm_ramper.c
 * @brief PWM soft-start ramper implementation
 */

#include "pwm_ramper.h"
#include <esp_log.h>
#include <freertos/FreeRTOS.h>
#include <freertos/semphr.h>    // xSemaphoreCreateMutex, xSemaphoreTake
#include <freertos/task.h>      // xTaskCreate, vTaskDelay, xTaskNotifyGive

static const char *TAG = "pwm_ramper";

// Stan rampera - singleton
static struct {
```

```
    bool initialized;
    hal_pwm_channel_t channel;      // Kontrolowany kanał PWM
    pwm_ramper_config_t config;     // Konfiguracja
    TaskHandle_t task_handle;       // Uchwyt do taska FreeRTOS
    SemaphoreHandle_t mutex;        // Mutex do synchronizacji
    volatile bool ramp_active;      // Czy rampa jest aktywna?
    volatile bool stop_requested;   // Czy żądano stopu?
    uint8_t current_duty;           // Aktualny duty cycle
} s_ramper = {0};

// 'volatile' informuje kompilator że zmienna może być
// zmieniana asynchronicznie (przez inny task/przerwanie)
// Kompilator nie będzie optymalizował dostępu do niej
```

7.4.4 Task rampy

```
// Linia 31-85: Task działający w tle
/**
 * @brief Ramp task - runs in background
 */
static void ramp_task(void *arg) {
    (void)arg;  // Nieużywany parametr

    while (1) {  // Pętla nieskończona - task działa zawsze
        // === Czekanie na notyfikację ===
        // Task "śpi" tutaj dopóki nie otrzyma powiadomienia
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        // pdTRUE - czyści licznik powiadomień po otrzymaniu
        // portMAX_DELAY - czeka nieskończenie długo

        // Sprawdzenie czy nie żądano stopu
        if (s_ramper.stop_requested) {
            continue;  // Wróć na początek pętli (czekaj dalej)
        }

        // === Początek rampy ===
        xSemaphoreTake(s_ramper.mutex, portMAX_DELAY);
        s_ramper.ramp_active = true;
        xSemaphoreGive(s_ramper.mutex);
        // Mutex chroni dostęp do zmiennej współdzielonej

        // Obliczenie parametrów rampy
        uint32_t step_delay_ms =
            s_ramper.config.ramp_duration_ms / s_ramper.config.num_steps;
```

```
    // np. 500ms / 25 kroków = 20ms na krok

    uint8_t duty_step =
        s_ramper.config.max_duty_percent / s_ramper.config.num_steps;
    // np. 100% / 25 kroków = 4% na krok

    if (duty_step == 0) {
        duty_step = 1;  // Minimum 1% na krok
    }

    ESP_LOGD(TAG, "Starting ramp: %lu ms, %d steps, %d%% max",
             s_ramper.config.ramp_duration_ms,
             s_ramper.config.num_steps,
             s_ramper.config.max_duty_percent);

    s_ramper.current_duty = 0;

    // === Pętla rampy ===
    for (uint8_t step = 0; step < s_ramper.config.num_steps; step++) {
        // Sprawdzenie czy nie żądano stopu
        if (s_ramper.stop_requested) {
            break;  // Przerwij pętlę
        }

        // Zwiększenie duty cycle
        s_ramper.current_duty += duty_step;
        if (s_ramper.current_duty > s_ramper.config.max_duty_percent) {
            s_ramper.current_duty = s_ramper.config.max_duty_percent;
        }

        // Ustawienie PWM
        hal_pwm_set_duty(s_ramper.channel, s_ramper.current_duty);

        // Opóźnienie między krokami
        vTaskDelay(pdMS_TO_TICKS(step_delay_ms));
        // pdMS_TO_TICKS konwertuje milisekundy na ticki FreeRTOS
    }

    // === Koniec rampy ===
    // Upewnienie się że osiągnęliśmy max duty
    if (!s_ramper.stop_requested) {
        s_ramper.current_duty = s_ramper.config.max_duty_percent;
        hal_pwm_set_duty(s_ramper.channel, s_ramper.current_duty);
```

```
        }

        xSemaphoreTake(s_ramper.mutex, portMAX_DELAY);
        s_ramper.ramp_active = false;
        xSemaphoreGive(s_ramper.mutex);

        ESP_LOGD(TAG, "Ramp complete, duty=%d%%", s_ramper.current_duty);
    }
}
```

> **Dokumentacja FreeRTOS:**
>
> - Task Notifications
> - Semaphores

7.4.5 Inicjalizacja rampera

```
// Linia 87-125: Inicjalizacja
esp_err_t pwm_ramper_init(hal_pwm_channel_t channel, const
pwm_ramper_config_t *config) {
    if (config == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    // Walidacja kanału PWM
    if (!hal_pwm_is_valid(channel)) {
        ESP_LOGE(TAG, "Invalid PWM channel");
        return ESP_ERR_INVALID_ARG;
    }

    // Zapisanie stanu
    s_ramper.channel = channel;
    s_ramper.config = *config;
    s_ramper.ramp_active = false;
    s_ramper.stop_requested = false;
    s_ramper.current_duty = 0;

    // === Utworzenie mutexa ===
    s_ramper.mutex = xSemaphoreCreateMutex();
    if (s_ramper.mutex == NULL) {
        ESP_LOGE(TAG, "Failed to create mutex");
        return ESP_ERR_NO_MEM;  // Brak pamięci
    }
```

```
    // === Utworzenie taska rampy ===
    BaseType_t ret = xTaskCreate(
        ramp_task,              // Funkcja taska
        "pwm_ramp",             // Nazwa (dla debugowania)
        2048,                   // Rozmiar stosu (bajty)
        NULL,                   // Parametr (nieużywany)
        5,                      // Priorytet (0-24, wyższy = ważniejszy)
        &s_ramper.task_handle   // [OUT] Uchwyt do taska
    );

    if (ret != pdPASS) {
        ESP_LOGE(TAG, "Failed to create ramp task");
        vSemaphoreDelete(s_ramper.mutex);
        return ESP_ERR_NO_MEM;
    }

    s_ramper.initialized = true;

    ESP_LOGI(TAG, "PWM ramper initialized (duration=%lu ms, steps=%d)",
             config->ramp_duration_ms, config->num_steps);

    return ESP_OK;
}
```

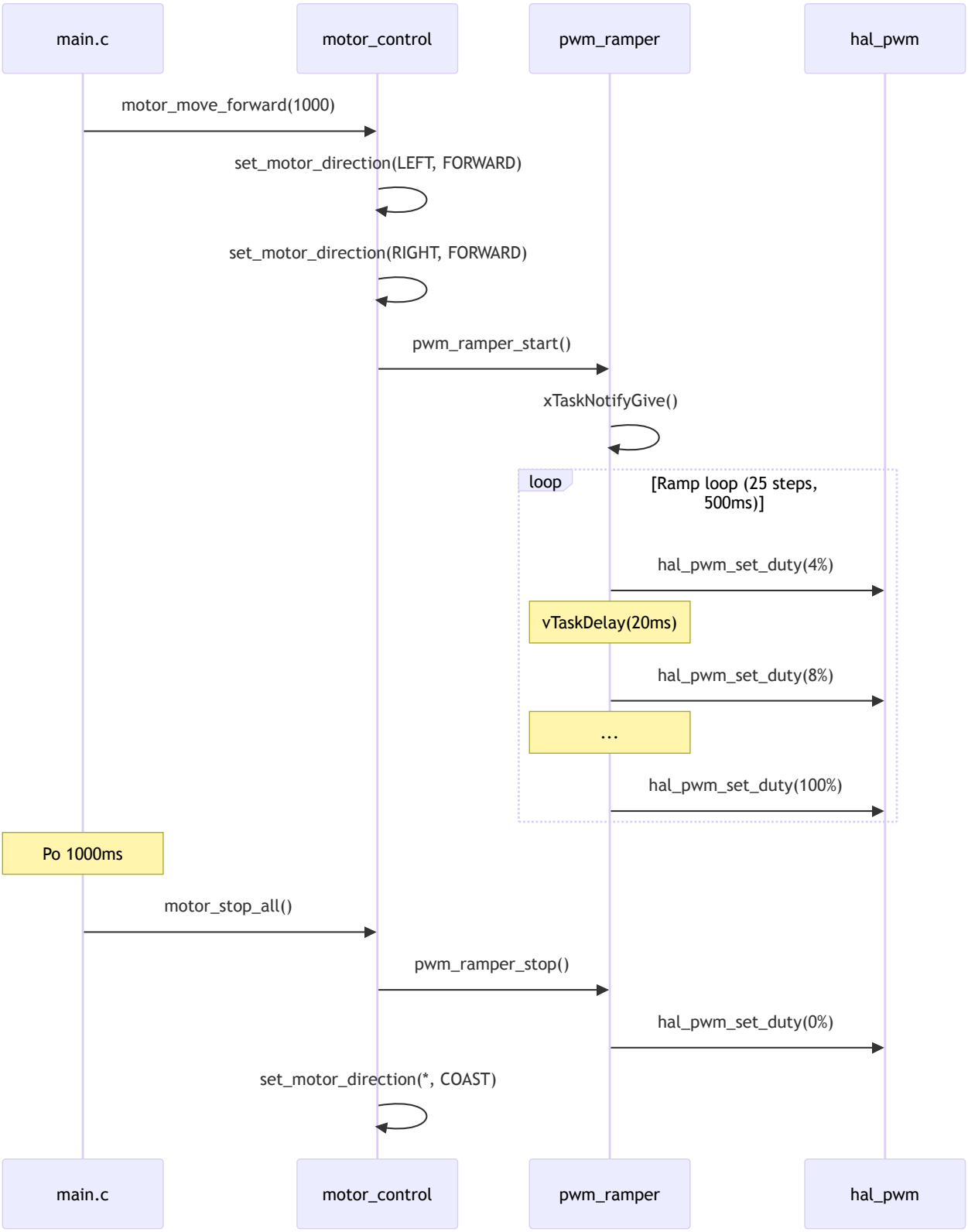
7.4.6 Funkcje sterujące

```
// Linia 127-144: Start rampy
void pwm_ramper_start(void) {
    if (!s_ramper.initialized) {
        return;
    }

    // Anulowanie aktywnej rampy (jeśli jest)
    s_ramper.stop_requested = true;

    // Czekanie aż rampa się zatrzyma
    while (s_ramper.ramp_active) {
        vTaskDelay(pdMS_TO_TICKS(1));  // Aktywne oczekiwanie
    }

    s_ramper.stop_requested = false;

    // === Wysłanie powiadomienia do taska ===
    xTaskNotifyGive(s_ramper.task_handle);
```

```
    // Task obudzi się i rozpocznie rampę
}

// Linia 146-163: Stop rampy
void pwm_ramper_stop(void) {
    if (!s_ramper.initialized) {
        return;
    }

    s_ramper.stop_requested = true;

    // Czekanie na zakończenie rampy
    while (s_ramper.ramp_active) {
        vTaskDelay(pdMS_TO_TICKS(1));
    }

    // Ustawienie duty na 0
    s_ramper.current_duty = 0;
    hal_pwm_set_duty(s_ramper.channel, 0);

    s_ramper.stop_requested = false;
}

// Linia 165-180: Ustawienie natychmiastowego duty
void pwm_ramper_set_duty(uint8_t duty_percent) {
    if (!s_ramper.initialized) {
        return;
    }

    // Zatrzymaj aktywną rampę
    pwm_ramper_stop();

    // Clamp do max
    if (duty_percent > s_ramper.config.max_duty_percent) {
        duty_percent = s_ramper.config.max_duty_percent;
    }

    s_ramper.current_duty = duty_percent;
    hal_pwm_set_duty(s_ramper.channel, duty_percent);
}

// Linia 182-184: Sprawdzenie aktywności
bool pwm_ramper_is_active(void) {
```

```
    return s_ramper.ramp_active;
}

// Linia 186-206: Cleanup
void pwm_ramper_cleanup(void) {
    if (!s_ramper.initialized) {
        return;
    }

    pwm_ramper_stop();

    // Usunięcie taska
    if (s_ramper.task_handle != NULL) {
        vTaskDelete(s_ramper.task_handle);
        s_ramper.task_handle = NULL;
    }

    // Usunięcie mutexa
    if (s_ramper.mutex != NULL) {
        vSemaphoreDelete(s_ramper.mutex);
        s_ramper.mutex = NULL;
    }

    s_ramper.initialized = false;

    ESP_LOGI(TAG, "PWM ramper cleanup complete");
}
```

7.5 Diagram przepływu



7.6 Ćwiczenia

Ćwiczenie 7.1: Różne prędkości

Zadanie: Zmodyfikuj `motor_control` aby obsługiwał parametr prędkości (0-100%).

Wskazówka: Dodaj parametr `speed_percent` do funkcji ruchu i przekaż go do `pwm_ramper_config_t.max_duty_percent`.

Rozwiązanie:

```
// W motor_control.h dodaj:
esp_err_t motor_move_forward_speed(uint32_t duration_ms, uint8_t
speed_percent);

// W motor_control.c:
esp_err_t motor_move_forward_speed(uint32_t duration_ms, uint8_t
speed_percent) {
    if (!s_motor.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    // Clamp prędkości
    if (speed_percent > 100) {
        speed_percent = 100;
    }

    ESP_LOGD(TAG, "Moving forward at %d%% (duration=%lu ms)",
             speed_percent, duration_ms);

    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_FORWARD);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_FORWARD);

    // Ustaw docelowy duty zamiast rampy
    pwm_ramper_set_duty(speed_percent);

    return ESP_OK;
}
```

Ćwiczenie 7.2: Hamowanie aktywne

Zadanie: Dodaj funkcję `motor_brake_all()` która używa trybu BRAKE zamiast COAST.

Rozwiązanie:

```
// W motor_control.h:
esp_err_t motor_brake_all(void);

// W motor_control.c:
esp_err_t motor_brake_all(void) {
    if (!s_motor.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    ESP_LOGD(TAG, "Braking all motors");
```

```
    pwm_ramper_stop();

    // BRAKE zamiast COAST - aktywne hamowanie
    set_motor_direction(&s_motor.config.left_motor, MOTOR_MODE_BRAKE);
    set_motor_direction(&s_motor.config.right_motor, MOTOR_MODE_BRAKE);

    return ESP_OK;
}
```

Rozdział 8: Komponent servo_control - Sterowanie Wieżą

8.1 Wprowadzenie

Komponent `servo_control` steruje serwomechanizmem SG90, który obraca wieżyczkę z kamerą. Serwomechanizmy są sterowane sygnałem PWM o stałej częstotliwości 50Hz, gdzie szerokość impulsu określa pozycję kątową.

Główne funkcje:

- Konwersja kąta na szerokość impulsu PWM
- Płynny ruch z interpolacją
- Funkcje step left/right dla kroków po 10°

8.2 Sterowanie serwomechanizmem - teoria

8.2.1 Sygnał PWM dla serwa

Serwomechanizmy RC używają sygnału PWM o częstotliwości **50Hz** (okres 20ms):

Szerokość impulsu	Kąt
500μs	0°
1000μs	45°
1500μs	90° (środek)
2000μs	135°
2400μs	180°

8.2.2 Wzór konwersji

```
pulse_us = min_pulse + (angle - min_angle) * (max_pulse - min_pulse) /
(max_angle - min_angle)
```

Przykład dla SG90:

- `min_pulse = 500µs`, `max_pulse = 2400µs`
- `min_angle = 0°`, `max_angle = 180°`

Dla kąta 90°:

```
pulse = 500 + (90 - 0) * (2400 - 500) / (180 - 0)
pulse = 500 + 90 * 1900 / 180
pulse = 500 + 950 = 1450µs
```

8.3 Plik nagłówkowy servo_control.h

```
// Plik: components/servo_control/include/servo_control.h
/**
 * @file servo_control.h
 * @brief SG90 servo controller interface
 */

#ifndef SERVO_CONTROL_H
#define SERVO_CONTROL_H

#include <esp_err.h>
#include <driver/gpio.h>
#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

// Struktura konfiguracji serwa
typedef struct {
    gpio_num_t signal_pin;        // Pin sygnału PWM
    uint16_t min_pulse_us;        // Min szerokość impulsu (500µs)
    uint16_t max_pulse_us;        // Max szerokość impulsu (2400µs)
    uint8_t min_angle;            // Minimalny kąt (0°)
    uint8_t max_angle;            // Maksymalny kąt (180°)
    uint8_t default_angle;        // Pozycja domyślna (90°)
    uint8_t step_angle;           // Krok dla step_left/right (10°)
    uint8_t smooth_step_degrees;  // Krok interpolacji (2°)
    uint8_t smooth_delay_ms;      // Opóźnienie między krokami (15ms)
} servo_config_t;
```

```c
// Funkcje API
esp_err_t servo_init(const servo_config_t *config);
uint8_t servo_get_angle(void);
esp_err_t servo_move_to(uint8_t angle, bool smooth);
esp_err_t servo_step_left(void);
esp_err_t servo_step_right(void);
esp_err_t servo_center(void);
void servo_stop(void);
void servo_release(void);
void servo_cleanup(void);

#ifdef __cplusplus
}
#endif

#endif /* SERVO_CONTROL_H */
```

8.4 Implementacja servo_control.c

```c
// Plik: components/servo_control/src/servo_control.c
/**
 * @file servo_control.c
 * @brief SG90 servo controller implementation
 */

#include "servo_control.h"
#include <esp_log.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include "hal_pwm.h"
#include "hal_types.h"

static const char *TAG = "servo_control";

// Stan serwa - singleton
static struct {
    bool initialized;
    servo_config_t config;
    hal_pwm_channel_t pwm_channel;  // Kanał PWM
    uint8_t current_angle;           // Aktualny kąt
} s_servo = {0};
```

8.4.1 Konwersja kąta na impuls

```c
// Linia 29-44: Konwersja kąta na szerokość impulsu
/**
 * @brief Convert angle to pulse width
 */
static uint16_t angle_to_pulse(uint8_t angle) {
    // Clamp kąta do zakresu
    if (angle < s_servo.config.min_angle) {
        angle = s_servo.config.min_angle;
    }
    if (angle > s_servo.config.max_angle) {
        angle = s_servo.config.max_angle;
    }

    // Obliczenie zakresów
    uint16_t pulse_range =
        s_servo.config.max_pulse_us - s_servo.config.min_pulse_us;
    // np. 2400 - 500 = 1900µs

    uint8_t angle_range =
        s_servo.config.max_angle - s_servo.config.min_angle;
    // np. 180 - 0 = 180°

    // Wzór liniowej interpolacji
    uint16_t pulse = s_servo.config.min_pulse_us +
        ((angle - s_servo.config.min_angle) * pulse_range) / angle_range;
    // np. dla 90°: 500 + (90 * 1900) / 180 = 500 + 950 = 1450µs

    return pulse;
}

// Linia 49-58: Ustawienie kąta (wewnętrzna)
/**
 * @brief Set servo to specific angle (internal)
 */
static esp_err_t set_angle_internal(uint8_t angle) {
    uint16_t pulse = angle_to_pulse(angle);

    // Użyj specjalnej funkcji HAL dla serwa
    esp_err_t ret = hal_pwm_set_servo_pulse(s_servo.pwm_channel, pulse);

    if (ret == ESP_OK) {
        s_servo.current_angle = angle;  // Zapisz nowy kąt
    }
```

```
    return ret;
}
```

8.4.2 Inicjalizacja serwa

```
// Linia 60-88: Inicjalizacja
esp_err_t servo_init(const servo_config_t *config) {
    if (config == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    // Inicjalizacja PWM dla serwa (50Hz)
    esp_err_t ret = hal_pwm_init(
        config->signal_pin,
        HAL_SERVO_PWM_FREQ,        // 50Hz (zdefiniowane w hal_types.h)
        &s_servo.pwm_channel
    );

    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "PWM init failed");
        return ret;
    }

    // Zapisanie konfiguracji
    s_servo.config = *config;
    s_servo.current_angle = config->default_angle;
    s_servo.initialized = true;

    // Przejście do pozycji domyślnej
    ret = set_angle_internal(config->default_angle);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to set default angle");
        return ret;
    }

    ESP_LOGI(TAG, "Servo initialized (pin=%d, default=%d°)",
             config->signal_pin, config->default_angle);

    return ESP_OK;
}
```

8.4.3 Płynny ruch z interpolacją

```
// Linia 94-137: Ruch do zadanego kąta
esp_err_t servo_move_to(uint8_t angle, bool smooth) {
    if (!s_servo.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    // Clamp kąta do zakresu
    if (angle < s_servo.config.min_angle) {
        angle = s_servo.config.min_angle;
    }
    if (angle > s_servo.config.max_angle) {
        angle = s_servo.config.max_angle;
    }

    // Jeśli smooth=false, ustaw od razu
    if (!smooth) {
        return set_angle_internal(angle);
    }

    // === Płynny ruch z interpolacją ===
    uint8_t current = s_servo.current_angle;

    // Określenie kierunku
    int8_t direction = (angle > current) ? 1 : -1;
    // +1 gdy kąt rośnie, -1 gdy maleje

    uint8_t step = s_servo.config.smooth_step_degrees;  // np. 2°

    ESP_LOGD(TAG, "Smooth move from %d° to %d°", current, angle);

    // Pętla interpolacji
    while (current != angle) {
        // Oblicz następny krok
        int next = current + (direction * step);

        // Clamp do celu (nie przeskocz!)
        if ((direction > 0 && next > angle) ||
            (direction < 0 && next < angle)) {
            next = angle;
        }

        // Ustaw nowy kąt
        esp_err_t ret = set_angle_internal((uint8_t)next);
```

```
        if (ret != ESP_OK) {
            return ret;
        }

        current = (uint8_t)next;

        // Opóźnienie między krokami (np. 15ms)
        vTaskDelay(pdMS_TO_TICKS(s_servo.config.smooth_delay_ms));
    }

    return ESP_OK;
}
```

8.4.4 Funkcje step

```
// Linia 139-153: Step w lewo (zmniejszenie kąta)
esp_err_t servo_step_left(void) {
    if (!s_servo.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    // Oblicz nowy kąt (zmniejszenie)
    int new_angle = s_servo.current_angle - s_servo.config.step_angle;
    // np. 90 - 10 = 80°

    // Clamp do minimum
    if (new_angle < s_servo.config.min_angle) {
        new_angle = s_servo.config.min_angle;
    }

    ESP_LOGD(TAG, "Step left: %d° -> %d°", s_servo.current_angle,
new_angle);

    // Użyj płynnego ruchu
    return servo_move_to((uint8_t)new_angle, true);
}

// Linia 155-169: Step w prawo (zwiększenie kąta)
esp_err_t servo_step_right(void) {
    if (!s_servo.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    // Oblicz nowy kąt (zwiększenie)
```

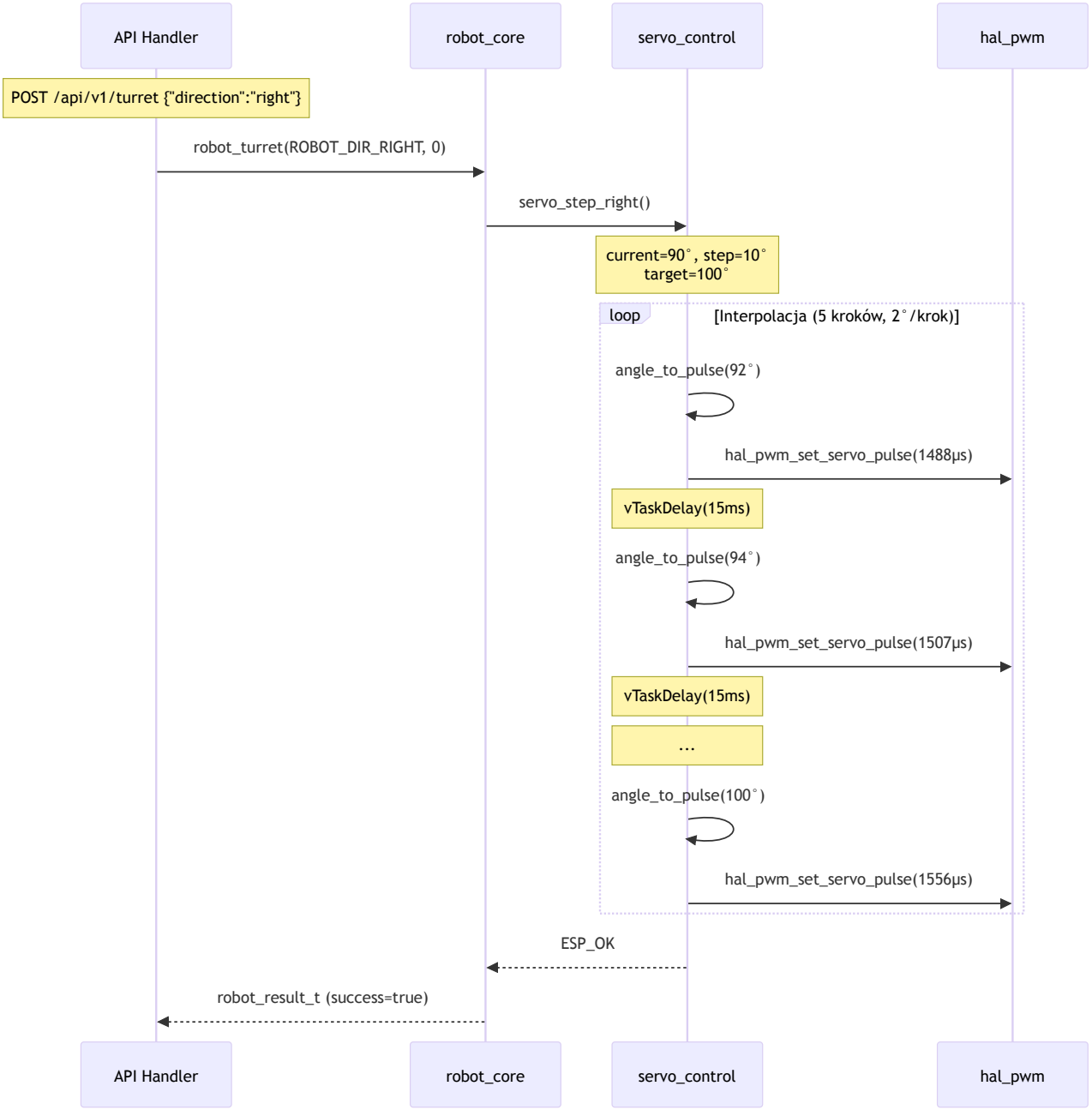
```
    int new_angle = s_servo.current_angle + s_servo.config.step_angle;
    // np. 90 + 10 = 100°

    // Clamp do maksimum
    if (new_angle > s_servo.config.max_angle) {
        new_angle = s_servo.config.max_angle;
    }

    ESP_LOGD(TAG, "Step right: %d° -> %d°", s_servo.current_angle,
new_angle);

    return servo_move_to((uint8_t)new_angle, true);
}

// Linia 171-179: Centrowanie
esp_err_t servo_center(void) {
    if (!s_servo.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    ESP_LOGD(TAG, "Centering to %d°", s_servo.config.default_angle);

    // Płynny powrót do pozycji domyślnej
    return servo_move_to(s_servo.config.default_angle, true);
}
```

8.4.5 Release i cleanup

```
// Linia 181-192: Zwolnienie serwa (zatrzymanie PWM)
void servo_release(void) {
    if (!s_servo.initialized) {
        return;
    }

    // Zatrzymanie sygnału PWM
    // Serwo staje się "luźne" - można je ręcznie obracać
    hal_pwm_stop(s_servo.pwm_channel);
    ESP_LOGD(TAG, "Servo released");
}

// Linia 194-204: Pełne sprzątanie
void servo_cleanup(void) {
    if (!s_servo.initialized) {
        return;
```

```
    }

    servo_release();
    hal_pwm_cleanup(s_servo.pwm_channel);
    s_servo.initialized = false;

    ESP_LOGI(TAG, "Servo cleanup complete");
}
```

8.5 Diagram czasowy



8.6 Ćwiczenia

Ćwiczenie 8.1: Sweep

Zadanie: Napisz funkcję `servo_sweep()` która płynnie obraca serwo od min do max i z powrotem.

Rozwiązanie:

```
// W servo_control.h:
esp_err_t servo_sweep(uint8_t num_sweeps);

// W servo_control.c:
esp_err_t servo_sweep(uint8_t num_sweeps) {
    if (!s_servo.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    esp_err_t ret;

    for (uint8_t i = 0; i < num_sweeps; i++) {
        // Do minimum
        ret = servo_move_to(s_servo.config.min_angle, true);
        if (ret != ESP_OK) return ret;

        // Do maksimum
        ret = servo_move_to(s_servo.config.max_angle, true);
        if (ret != ESP_OK) return ret;
    }

    // Powrót do środka
    return servo_center();
}
```

Rozdział 9: Komponent wifi_manager - Zarządzanie WiFi

9.1 Wprowadzenie

Komponent `wifi_manager` konfiguruje ESP32 jako Access Point (punkt dostępowy). Robot tworzy własną sieć WiFi, do której można połączyć telefon lub laptop, aby sterować robotem.

Kluczowe cechy:

- Tryb AP (Access Point) - robot jest "routerem"
- Statyczne IP: 10.42.0.1
- DHCP dla klientów
- Obsługa zdarzeń WiFi

9.2 Tryb AP vs STA

ESP32 może pracować w trzech trybach WiFi:

Tryb	Opis	Zastosowanie
STA (Station)	Łączy się do istniejącej sieci	Robot w sieci domowej
AP (Access Point)	Tworzy własną sieć	Robot autonomiczny
AP+STA	Oba tryby	Most WiFi

W naszym projekcie używamy **trybu AP** - robot jest niezależny od infrastruktury.

9.3 Plik nagłówkowy wifi_manager.h

```
// Plik: components/wifi_manager/include/wifi_manager.h
/**
 * @file wifi_manager.h
 * @brief WiFi Access Point management
 */

#ifndef WIFI_MANAGER_H
#define WIFI_MANAGER_H

#include <esp_err.h>
#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

// Inicjalizacja stosu WiFi
esp_err_t wifi_manager_init(void);

// Uruchomienie AP z podanym SSID i hasłem
esp_err_t wifi_manager_start_ap(const char *ssid, const char *password);

// Zatrzymanie AP
esp_err_t wifi_manager_stop_ap(void);

// Sprawdzenie czy AP jest aktywny
bool wifi_manager_is_active(void);

// Liczba połączonych stacji
```

```c
uint8_t wifi_manager_get_station_count(void);

// Pobranie IP adresu AP
esp_err_t wifi_manager_get_ip(char *buffer, size_t buffer_size);

#ifdef __cplusplus
}
#endif

#endif /* WIFI_MANAGER_H */
```

9.4 Implementacja wifi_manager.c

```c
// Plik: components/wifi_manager/src/wifi_manager.c
/**
 * @file wifi_manager.c
 * @brief WiFi Access Point management implementation
 */

#include "wifi_manager.h"
#include <esp_event.h>         // System zdarzeń ESP-IDF
#include <esp_log.h>
#include <esp_netif.h>         // Interfejs sieciowy
#include <esp_wifi.h>          // Sterownik WiFi
#include <freertos/FreeRTOS.h>
#include <freertos/event_groups.h>
#include <string.h>

static const char *TAG = "wifi_manager";

// Konfiguracja sieci AP
#define AP_IP_ADDR "10.42.0.1"     // Adres IP routera
#define AP_GATEWAY "10.42.0.1"     // Brama (ten sam adres)
#define AP_NETMASK "255.255.255.0" // Maska podsieci

// Stan modułu WiFi - singleton
static struct {
    bool initialized;
    bool active;
    esp_netif_t *netif;       // Interfejs sieciowy
    uint8_t station_count;    // Liczba połączonych klientów
} s_wifi = {0};
```

9.4.1 Handler zdarzeń WiFi

```c
// Linia 36-72: Obsługa zdarzeń WiFi
/**
 * @brief WiFi event handler
 */
static void wifi_event_handler(
    void *arg,                    // Parametr użytkownika (nieużywany)
    esp_event_base_t event_base,  // Typ zdarzenia (WIFI_EVENT lub IP_EVENT)
    int32_t event_id,             // ID konkretnego zdarzenia
    void *event_data              // Dane zdarzenia
) {
    (void)arg;  // Ignoruj nieużywany parametr

    // === Zdarzenia WiFi ===
    if (event_base == WIFI_EVENT) {
        switch (event_id) {
            case WIFI_EVENT_AP_START:
                // AP zostało uruchomione
                s_wifi.active = true;
                ESP_LOGI(TAG, "AP started");
                break;

            case WIFI_EVENT_AP_STOP:
                // AP zostało zatrzymane
                s_wifi.active = false;
                s_wifi.station_count = 0;
                ESP_LOGI(TAG, "AP stopped");
                break;

            case WIFI_EVENT_AP_STACONNECTED:
                // Nowy klient się połączył
                s_wifi.station_count++;
                ESP_LOGI(TAG, "Station connected (total: %d)",
                         s_wifi.station_count);
                break;

            case WIFI_EVENT_AP_STADISCONNECTED:
                // Klient się rozłączył
                if (s_wifi.station_count > 0) {
                    s_wifi.station_count--;
                }
                ESP_LOGI(TAG, "Station disconnected (total: %d)",
                         s_wifi.station_count);
                break;
```

```
            default:
                break;
        }
    }
    // === Zdarzenia IP ===
    else if (event_base == IP_EVENT && event_id ==
IP_EVENT_AP_STAIPASSIGNED) {
        // Klient otrzymał adres IP z DHCP
        ip_event_ap_staipassigned_t *event =
            (ip_event_ap_staipassigned_t *)event_data;
        ESP_LOGI(TAG, "Station assigned IP: " IPSTR, IP2STR(&event->ip));
        // IPSTR i IP2STR to makra formatujące adres IP:
        // np. "192.168.4.2"
    }
}
```

> **Dokumentacja ESP-IDF:**
>
> - ESP Event Loop
> - WiFi Events

9.4.2 Inicjalizacja stosu sieciowego

```
// Linia 74-141: Inicjalizacja
esp_err_t wifi_manager_init(void) {
    if (s_wifi.initialized) {
        return ESP_OK;  // Już zainicjalizowane
    }

    // === Krok 1: Inicjalizacja interfejsu sieciowego ===
    esp_err_t ret = esp_netif_init();
    // Inicjalizuje stos TCP/IP (lwIP)
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "netif init failed");
        return ret;
    }

    // === Krok 2: Utworzenie pętli zdarzeń ===
    ret = esp_event_loop_create_default();
    // Tworzy domyślną pętlę dla systemu zdarzeń
    if (ret != ESP_OK && ret != ESP_ERR_INVALID_STATE) {
        // ESP_ERR_INVALID_STATE = już istnieje (OK)
        ESP_LOGE(TAG, "event loop create failed");
```

```
    return ret;
}

// === Krok 3: Utworzenie interfejsu AP ===
s_wifi.netif = esp_netif_create_default_wifi_ap();
// Tworzy interfejs sieciowy dla trybu AP
if (s_wifi.netif == NULL) {
    ESP_LOGE(TAG, "netif create failed");
    return ESP_FAIL;
}

// === Krok 4: Konfiguracja statycznego IP ===
esp_netif_dhcps_stop(s_wifi.netif);  // Zatrzymaj DHCP serwer

esp_netif_ip_info_t ip_info = {0};
ip_info.ip.addr = esp_ip4addr_aton(AP_IP_ADDR);       // 10.42.0.1
ip_info.gw.addr = esp_ip4addr_aton(AP_GATEWAY);       // 10.42.0.1
ip_info.netmask.addr = esp_ip4addr_aton(AP_NETMASK);  // 255.255.255.0
// esp_ip4addr_aton konwertuje string "10.42.0.1" na uint32_t

ret = esp_netif_set_ip_info(s_wifi.netif, &ip_info);
if (ret != ESP_OK) {
    ESP_LOGE(TAG, "Failed to set IP info");
    return ret;
}

esp_netif_dhcps_start(s_wifi.netif);  // Uruchom DHCP serwer

// === Krok 5: Inicjalizacja sterownika WiFi ===
wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
// Makro tworzące domyślną konfigurację

ret = esp_wifi_init(&cfg);
if (ret != ESP_OK) {
    ESP_LOGE(TAG, "WiFi init failed");
    return ret;
}

// === Krok 6: Rejestracja handlerów zdarzeń ===
ret = esp_event_handler_instance_register(
    WIFI_EVENT,              // Typ zdarzeń
    ESP_EVENT_ANY_ID,        // Wszystkie zdarzenia WiFi
    &wifi_event_handler,     // Nasza funkcja
```

```
        NULL,                       // Parametr użytkownika
        NULL                        // [OUT] Handle do wyrejestrowania
    );
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to register WiFi event handler");
        return ret;
    }

    ret = esp_event_handler_instance_register(
        IP_EVENT,
        IP_EVENT_AP_STAIPASSIGNED,  // Tylko to zdarzenie
        &wifi_event_handler,
        NULL,
        NULL
    );
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to register IP event handler");
        return ret;
    }

    s_wifi.initialized = true;
    ESP_LOGI(TAG, "WiFi manager initialized");

    return ESP_OK;
}
```

9.4.3 Uruchomienie Access Point

```
// Linia 143-192: Start AP
esp_err_t wifi_manager_start_ap(const char *ssid, const char *password) {
    if (!s_wifi.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    if (ssid == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    // === Konfiguracja AP ===
    wifi_config_t wifi_config = {
        .ap = {
            .max_connection = 4,            // Max 4 klientów
            .authmode = WIFI_AUTH_OPEN,     // Domyślnie: otwarta sieć
        },
```

```
    };

    // Kopiowanie SSID
    strncpy((char *)wifi_config.ap.ssid, ssid,
            sizeof(wifi_config.ap.ssid) - 1);
    wifi_config.ap.ssid_len = strlen(ssid);

    // Ustawienie hasła (jeśli podane i wystarczająco długie)
    if (password != NULL && strlen(password) >= 8) {
        strncpy((char *)wifi_config.ap.password, password,
                sizeof(wifi_config.ap.password) - 1);
        wifi_config.ap.authmode = WIFI_AUTH_WPA2_PSK;
        // WPA2-PSK wymaga hasła min. 8 znaków
    } else if (password != NULL && strlen(password) > 0) {
        ESP_LOGW(TAG, "Password too short (min 8 chars), creating open
network");
    }

    // === Ustawienie trybu AP ===
    esp_err_t ret = esp_wifi_set_mode(WIFI_MODE_AP);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to set WiFi mode");
        return ret;
    }

    // === Aplikacja konfiguracji ===
    ret = esp_wifi_set_config(WIFI_IF_AP, &wifi_config);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to set WiFi config");
        return ret;
    }

    // === Start WiFi ===
    ret = esp_wifi_start();
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to start WiFi");
        return ret;
    }

    ESP_LOGI(TAG, "AP started - SSID: %s, Password: %s, IP: %s",
             ssid,
             (wifi_config.ap.authmode == WIFI_AUTH_OPEN) ? "(open)" : "***",
             AP_IP_ADDR);
```

```
    return ESP_OK;
}
```

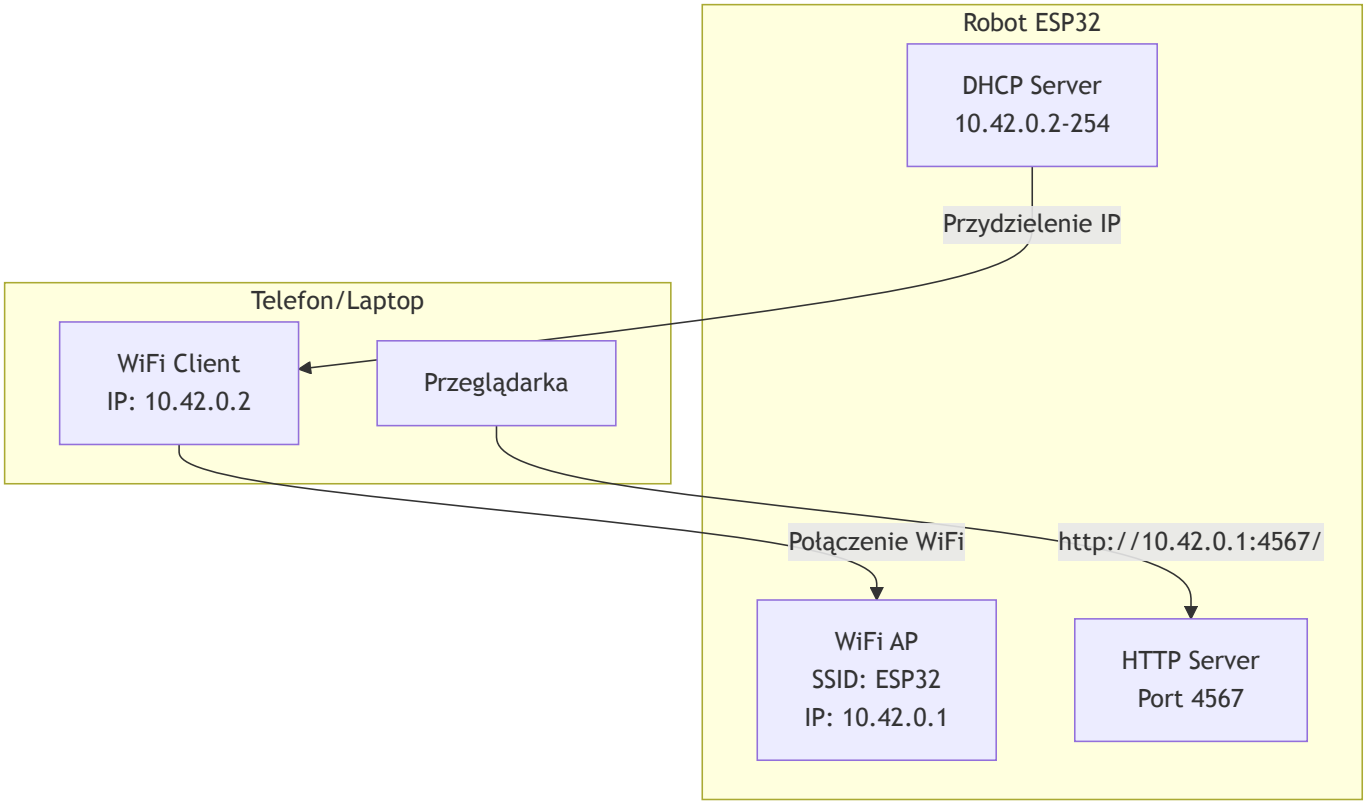
> **Dokumentacja ESP-IDF:**
>
> - ESP WiFi
> - WiFi Configuration

9.4.4 Pozostałe funkcje

```
// Linia 194-206: Stop AP
esp_err_t wifi_manager_stop_ap(void) {
    if (!s_wifi.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    esp_err_t ret = esp_wifi_stop();
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to stop WiFi");
        return ret;
    }

    return ESP_OK;
}

// Linia 208-210: Sprawdzenie statusu
bool wifi_manager_is_active(void) {
    return s_wifi.active;
}

// Linia 212-214: Liczba klientów
uint8_t wifi_manager_get_station_count(void) {
    return s_wifi.station_count;
}

// Linia 216-229: Pobranie IP
esp_err_t wifi_manager_get_ip(char *buffer, size_t buffer_size) {
    if (buffer == NULL || buffer_size == 0) {
        return ESP_ERR_INVALID_ARG;
    }

    if (!s_wifi.active) {
        return ESP_ERR_INVALID_STATE;
```

```
    }

    strncpy(buffer, AP_IP_ADDR, buffer_size - 1);
    buffer[buffer_size - 1] = '\0';  // Zawsze null-terminate

    return ESP_OK;
}
```

9.5 Diagram połączeń



9.6 Ćwiczenie

Ćwiczenie 9.1: Ukryte SSID

Zadanie: Zmodyfikuj `wifi_manager_start_ap()` aby obsługiwał ukryte sieci (SSID nie jest rozgłaszany).

Wskazówka: Użyj pola `wifi_config.ap.ssid_hidden`.

Rozwiązanie:

```
esp_err_t wifi_manager_start_ap_hidden(const char *ssid, const char
*password, bool hidden) {
    // ... (walidacja jak wcześniej) ...

    wifi_config_t wifi_config = {
        .ap = {
```

```
            .max_connection = 4,
            .authmode = WIFI_AUTH_OPEN,
            .ssid_hidden = hidden ? 1 : 0,  // 1 = ukryte SSID
        },
    };

    // ... (reszta jak wcześniej) ...
}
```

Rozdział 10: Komponent camera_stream - Streaming MJPEG

10.1 Wprowadzenie

Komponent `camera_stream` obsługuje kamerę OV2640 wbudowaną w ESP32-CAM i udostępnia strumień MJPEG przez HTTP. MJPEG (Motion JPEG) to format wideo, w którym każda klatka jest osobnym obrazem JPEG.

Główne funkcje:

- Inicjalizacja kamery OV2640
- Streaming MJPEG przez endpoint `/stream`
- Konfiguracja jakości i rozdzielczości

10.2 Format MJPEG

MJPEG to seria obrazów JPEG wysyłanych jako "multipart" HTTP:

```
HTTP/1.1 200 OK
Content-Type: multipart/x-mixed-replace;boundary=myboundary

--myboundary
Content-Type: image/jpeg
Content-Length: 12345

<dane JPEG klatka 1>
--myboundary
Content-Type: image/jpeg
Content-Length: 12346

<dane JPEG klatka 2>
...
```

Przeglądarka automatycznie rozpoznaje ten format i wyświetla wideo.

10.3 Plik nagłówkowy camera_stream.h

```
// Plik: components/camera_stream/include/camera_stream.h
#ifndef CAMERA_STREAM_H
#define CAMERA_STREAM_H

#include <esp_err.h>
#include <esp_http_server.h>  // httpd_handle_t

#ifdef __cplusplus
extern "C" {
#endif

// Konfiguracja kamery
typedef struct {
    int frame_size;    // Rozdzielczość (FRAMESIZE_VGA, etc.)
    int jpeg_quality;  // Jakość JPEG 10-63 (niższa = lepsza)
    int fb_count;      // Liczba buforów ramki
} camera_stream_config_t;

// Inicjalizacja kamery
esp_err_t camera_stream_init(const camera_stream_config_t *config);

// Rejestracja handlera HTTP
esp_err_t camera_stream_register_handler(httpd_handle_t server);

// Pobranie ścieżki streamu
const char *camera_stream_get_path(void);

// Sprawdzenie gotowości
bool camera_stream_is_ready(void);

// Cleanup
void camera_stream_cleanup(void);

#ifdef __cplusplus
}
#endif

#endif /* CAMERA_STREAM_H */
```

10.4 Implementacja camera_stream.c

```c
// Plik: components/camera_stream/src/camera_stream.c
#include "camera_stream.h"
#include <esp_camera.h>  // Biblioteka esp32-camera
#include <esp_log.h>
#include <string.h>

static const char *TAG = "camera_stream";

// Definicje MJPEG
#define PART_BOUNDARY "123456789000000000000987654321"
static const char *STREAM_CONTENT_TYPE =
    "multipart/x-mixed-replace;boundary=" PART_BOUNDARY;
static const char *STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";
static const char *STREAM_PART =
    "Content-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n";

// Stan kamery
static struct {
    bool initialized;
    camera_stream_config_t config;
} s_camera = {0};

static const char *STREAM_PATH = "/stream";
```

10.4.1 Handler streamu MJPEG

```c
// Linia 33-88: Handler HTTP dla streamu
/**
 * @brief MJPEG stream handler
 */
static esp_err_t stream_handler(httpd_req_t *req) {
    camera_fb_t *fb = NULL;  // Frame buffer
    esp_err_t res = ESP_OK;
    char part_buf[64];       // Bufor na nagłówek części

    // === Ustawienie typu odpowiedzi ===
    res = httpd_resp_set_type(req, STREAM_CONTENT_TYPE);
    // Content-Type: multipart/x-mixed-replace;boundary=...
    if (res != ESP_OK) {
        return res;
    }

    // Dodanie nagłówków CORS i framerate
    httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
```

```
    httpd_resp_set_hdr(req, "X-Framerate", "25");

    ESP_LOGI(TAG, "MJPEG stream started");

    // === Pętla streamingu ===
    while (true) {
        // Pobranie klatki z kamery
        fb = esp_camera_fb_get();
        if (!fb) {
            ESP_LOGE(TAG, "Camera capture failed");
            res = ESP_FAIL;
            break;
        }

        // Sprawdzenie formatu (musi być JPEG)
        if (fb->format != PIXFORMAT_JPEG) {
            ESP_LOGE(TAG, "Camera not in JPEG mode");
            esp_camera_fb_return(fb);  // Zwróć bufor
            res = ESP_FAIL;
            break;
        }

        // Przygotowanie nagłówka części
        // "Content-Type: image/jpeg\r\nContent-Length: 12345\r\n\r\n"
        size_t hlen = snprintf(part_buf, sizeof(part_buf),
                               STREAM_PART, fb->len);

        // Wysłanie boundary
        // "\r\n--123456789000000000000987654321\r\n"
        res = httpd_resp_send_chunk(req, STREAM_BOUNDARY,
                                    strlen(STREAM_BOUNDARY));
        if (res != ESP_OK) {
            esp_camera_fb_return(fb);
            break;
        }

        // Wysłanie nagłówka części
        res = httpd_resp_send_chunk(req, part_buf, hlen);
        if (res != ESP_OK) {
            esp_camera_fb_return(fb);
            break;
        }
```

```
        // Wysłanie danych JPEG
        res = httpd_resp_send_chunk(req, (const char *)fb->buf, fb->len);

        // Zwrócenie bufora do puli
        esp_camera_fb_return(fb);

        if (res != ESP_OK) {
            break;  // Klient rozłączony
        }
    }

    ESP_LOGI(TAG, "MJPEG stream ended");

    return res;
}
```

> **Dokumentacja esp32-camera:**
>
> - [esp32-camera GitHub]

10.4.2 Inicjalizacja kamery

```
// Linia 90-159: Inicjalizacja
esp_err_t camera_stream_init(const camera_stream_config_t *config) {
    if (config == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    // === Konfiguracja pinów AI-Thinker ESP32-CAM ===
    // Te piny są stałe dla modułu AI-Thinker!
    camera_config_t cam_config = {
        // Piny sterujące
        .pin_pwdn = 32,      // Power down
        .pin_reset = -1,     // Reset (nieużywany)
        .pin_xclk = 0,       // External clock

        // Piny I2C (SCCB)
        .pin_sccb_sda = 26,  // Dane I2C
        .pin_sccb_scl = 27,  // Zegar I2C

        // Piny danych (8-bit parallel DVP)
        .pin_d7 = 35,
        .pin_d6 = 34,
        .pin_d5 = 39,
        .pin_d4 = 36,
```

```
    .pin_d3 = 21,
    .pin_d2 = 19,
    .pin_d1 = 18,
    .pin_d0 = 5,

    // Piny synchronizacji
    .pin_vsync = 25,     // Vertical sync
    .pin_href = 23,      // Horizontal reference
    .pin_pclk = 22,      // Pixel clock

    // Konfiguracja zegara
    .xclk_freq_hz = 20000000,  // 20 MHz
    .ledc_timer = LEDC_TIMER_0,
    .ledc_channel = LEDC_CHANNEL_0,

    // Format obrazu
    .pixel_format = PIXFORMAT_JPEG,      // JPEG dla MJPEG
    .frame_size = config->frame_size,    // np. FRAMESIZE_VGA
    .jpeg_quality = config->jpeg_quality, // 10-63
    .fb_count = config->fb_count,        // Liczba buforów

    // Lokalizacja buforów
    .fb_location = CAMERA_FB_IN_PSRAM,   // Użyj PSRAM!
    .grab_mode = CAMERA_GRAB_WHEN_EMPTY  // Pobieraj gdy bufor pusty
};

// Inicjalizacja kamery
esp_err_t ret = esp_camera_init(&cam_config);
if (ret != ESP_OK) {
    ESP_LOGE(TAG, "Camera init failed: %s", esp_err_to_name(ret));
    return ret;
}

// === Konfiguracja sensora ===
sensor_t *sensor = esp_camera_sensor_get();
if (sensor) {
    // Ustawienia obrazu
    sensor->set_brightness(sensor, 0);    // Jasność
    sensor->set_contrast(sensor, 0);      // Kontrast
    sensor->set_saturation(sensor, 0);    // Nasycenie

    // Auto white balance
    sensor->set_whitebal(sensor, 1);      // Włącz AWB
```

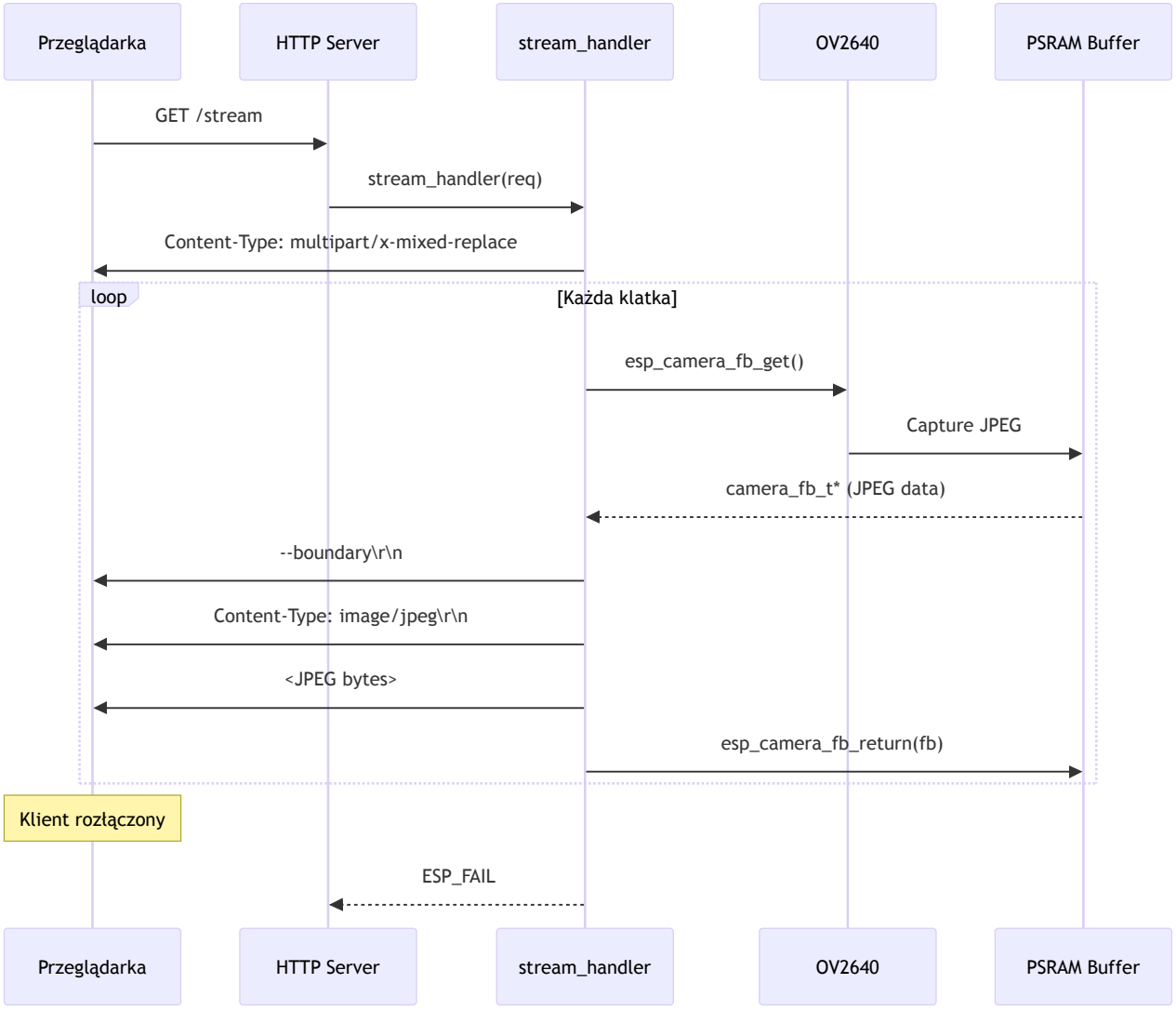
```
        sensor->set_awb_gain(sensor, 1);      // Włącz gain AWB
        sensor->set_wb_mode(sensor, 0);       // Auto WB mode

        // Auto exposure
        sensor->set_exposure_ctrl(sensor, 1); // Włącz AEC
        sensor->set_aec2(sensor, 1);          // AEC DSP

        // Auto gain
        sensor->set_gain_ctrl(sensor, 1);     // Włącz AGC
        sensor->set_agc_gain(sensor, 0);      // AGC gain
        sensor->set_gainceiling(sensor, (gainceiling_t)2);  // Max gain

        // Korekcje
        sensor->set_bpc(sensor, 1);           // Black pixel correction
        sensor->set_wpc(sensor, 1);           // White pixel correction
        sensor->set_raw_gma(sensor, 1);       // Raw gamma
        sensor->set_lenc(sensor, 1);          // Lens correction

        // Orientacja
        sensor->set_hmirror(sensor, 0);       // Bez lustrzanego odbicia
        sensor->set_vflip(sensor, 0);         // Bez odwrócenia
    }

    s_camera.config = *config;
    s_camera.initialized = true;

    ESP_LOGI(TAG, "Camera initialized (frame_size=%d, quality=%d,
fb_count=%d)",
             config->frame_size, config->jpeg_quality, config->fb_count);

    return ESP_OK;
}
```

10.4.3 Rejestracja handlera HTTP

```
// Linia 161-182: Rejestracja w serwerze HTTP
esp_err_t camera_stream_register_handler(httpd_handle_t server) {
    if (!s_camera.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    if (server == NULL) {
        return ESP_ERR_INVALID_ARG;
    }
```

```
    // Definicja endpointu
    httpd_uri_t stream_uri = {
        .uri = STREAM_PATH,       // "/stream"
        .method = HTTP_GET,       // Metoda GET
        .handler = stream_handler, // Nasza funkcja
        .user_ctx = NULL          // Brak kontekstu
    };

    // Rejestracja w serwerze
    esp_err_t ret = httpd_register_uri_handler(server, &stream_uri);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to register stream handler");
        return ret;
    }

    ESP_LOGI(TAG, "Stream handler registered at %s", STREAM_PATH);

    return ESP_OK;
}

// Linia 184-191: Pomocnicze funkcje
const char *camera_stream_get_path(void) {
    return STREAM_PATH;
}

bool camera_stream_is_ready(void) {
    return s_camera.initialized;
}

void camera_stream_cleanup(void) {
    if (!s_camera.initialized) {
        return;
    }

    esp_camera_deinit();  // Deinicjalizacja kamery
    s_camera.initialized = false;

    ESP_LOGI(TAG, "Camera cleanup complete");
}
```

10.5 Diagram przepływu danych



Rozdział 11: Komponent safety_handler - Bezpieczeństwo

11.1 Wprowadzenie

Bezpieczeństwo jest krytyczne w robotyce. Komponent `safety_handler` implementuje mechanizmy zabezpieczające:

1. **Watchdog Timer** - resetuje system jeśli główna pętla się zawiesi
2. **Auto-stop Timer** - automatycznie zatrzymuje silniki po określonym czasie
3. **Walidacja duration** - ogranicza maksymalny czas ruchu

11.2 Plik nagłówkowy safety_handler.h

```
// Plik: components/safety_handler/include/safety_handler.h
#ifndef SAFETY_HANDLER_H
#define SAFETY_HANDLER_H

#include <esp_err.h>
#include <stdint.h>
```

```c
#ifdef __cplusplus
extern "C" {
#endif

// Konfiguracja bezpieczeństwa
typedef struct {
    uint32_t watchdog_timeout_ms;  // Timeout watchdoga (np. 10000 ms)
    uint32_t movement_timeout_ms;  // Max czas ruchu (np. 5000 ms)
    uint32_t turret_timeout_ms;    // Max czas wieżyczki (np. 2000 ms)
} safety_config_t;

// Inicjalizacja
esp_err_t safety_handler_init(const safety_config_t *config);

// Awaryjne zatrzymanie wszystkiego
void safety_emergency_shutdown(void);

// "Karmienie" watchdoga (resetuje timer)
void safety_feed_watchdog(void);

// Walidacja i clamp czasu trwania
uint32_t safety_validate_duration(uint32_t duration_ms, uint32_t
max_duration_ms);

// Zaplanowanie auto-stopu
esp_err_t safety_schedule_auto_stop(uint32_t duration_ms);

// Anulowanie auto-stopu
void safety_cancel_auto_stop(void);

// Cleanup
void safety_handler_cleanup(void);

#ifdef __cplusplus
}
#endif

#endif /* SAFETY_HANDLER_H */
```

11.3 Implementacja safety_handler.c

```c
// Plik: components/safety_handler/src/safety_handler.c
#include "safety_handler.h"
```

```c
#include <esp_log.h>
#include <esp_timer.h>           // ESP Timer API
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include "motor_control.h"       // motor_stop_all()

static const char *TAG = "safety_handler";

// Stan modułu
static struct {
    bool initialized;
    safety_config_t config;
    esp_timer_handle_t auto_stop_timer;   // Timer jednorazowy
    esp_timer_handle_t watchdog_timer;    // Timer periodyczny
    volatile bool watchdog_fed;           // Flaga "nakarmienia"
} s_safety = {0};
```

11.3.1 Callbacki timerów

```c
// Linia 30-35: Callback auto-stopu
/**
 * @brief Auto-stop timer callback
 */
static void auto_stop_callback(void *arg) {
    (void)arg;
    ESP_LOGW(TAG, "Auto-stop triggered");
    motor_stop_all();  // Zatrzymaj silniki!
}

// Linia 39-48: Callback watchdoga
/**
 * @brief Watchdog timer callback
 */
static void watchdog_callback(void *arg) {
    (void)arg;

    // Sprawdź czy watchdog był "nakarmiony"
    if (!s_safety.watchdog_fed) {
        // NIE! System się zawiesił!
        ESP_LOGE(TAG, "Watchdog timeout - emergency shutdown!");
        safety_emergency_shutdown();
    }

    // Resetuj flagę na następny cykl
```

```
    s_safety.watchdog_fed = false;
}
```

> **Dokumentacja ESP-IDF:**
>
> - ESP Timer

11.3.2 Inicjalizacja

```
// Linia 50-100: Inicjalizacja
esp_err_t safety_handler_init(const safety_config_t *config) {
    if (config == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    s_safety.config = *config;

    // === Utworzenie timera auto-stop (one-shot) ===
    esp_timer_create_args_t auto_stop_args = {
        .callback = auto_stop_callback,     // Funkcja callback
        .arg = NULL,                        // Argument
        .dispatch_method = ESP_TIMER_TASK,  // Uruchom w tasku timera
        .name = "auto_stop"                 // Nazwa (debug)
    };

    esp_err_t ret = esp_timer_create(&auto_stop_args,
&s_safety.auto_stop_timer);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to create auto-stop timer");
        return ret;
    }

    // === Utworzenie timera watchdog (periodic) ===
    if (config->watchdog_timeout_ms > 0) {
        esp_timer_create_args_t watchdog_args = {
            .callback = watchdog_callback,
            .arg = NULL,
            .dispatch_method = ESP_TIMER_TASK,
            .name = "watchdog"
        };

        ret = esp_timer_create(&watchdog_args, &s_safety.watchdog_timer);
        if (ret != ESP_OK) {
            ESP_LOGE(TAG, "Failed to create watchdog timer");
            esp_timer_delete(s_safety.auto_stop_timer);
```

```
            return ret;
        }

        // Uruchomienie watchdoga jako periodic timer
        ret = esp_timer_start_periodic(
            s_safety.watchdog_timer,
            config->watchdog_timeout_ms * 1000  // Konwersja ms → µs
        );
        if (ret != ESP_OK) {
            ESP_LOGE(TAG, "Failed to start watchdog timer");
            esp_timer_delete(s_safety.auto_stop_timer);
            esp_timer_delete(s_safety.watchdog_timer);
            return ret;
        }

        s_safety.watchdog_fed = true;  // Początkowo "nakarmiony"
    }

    s_safety.initialized = true;

    ESP_LOGI(TAG, "Safety handler initialized (watchdog=%lu ms)",
             config->watchdog_timeout_ms);

    return ESP_OK;
}
```

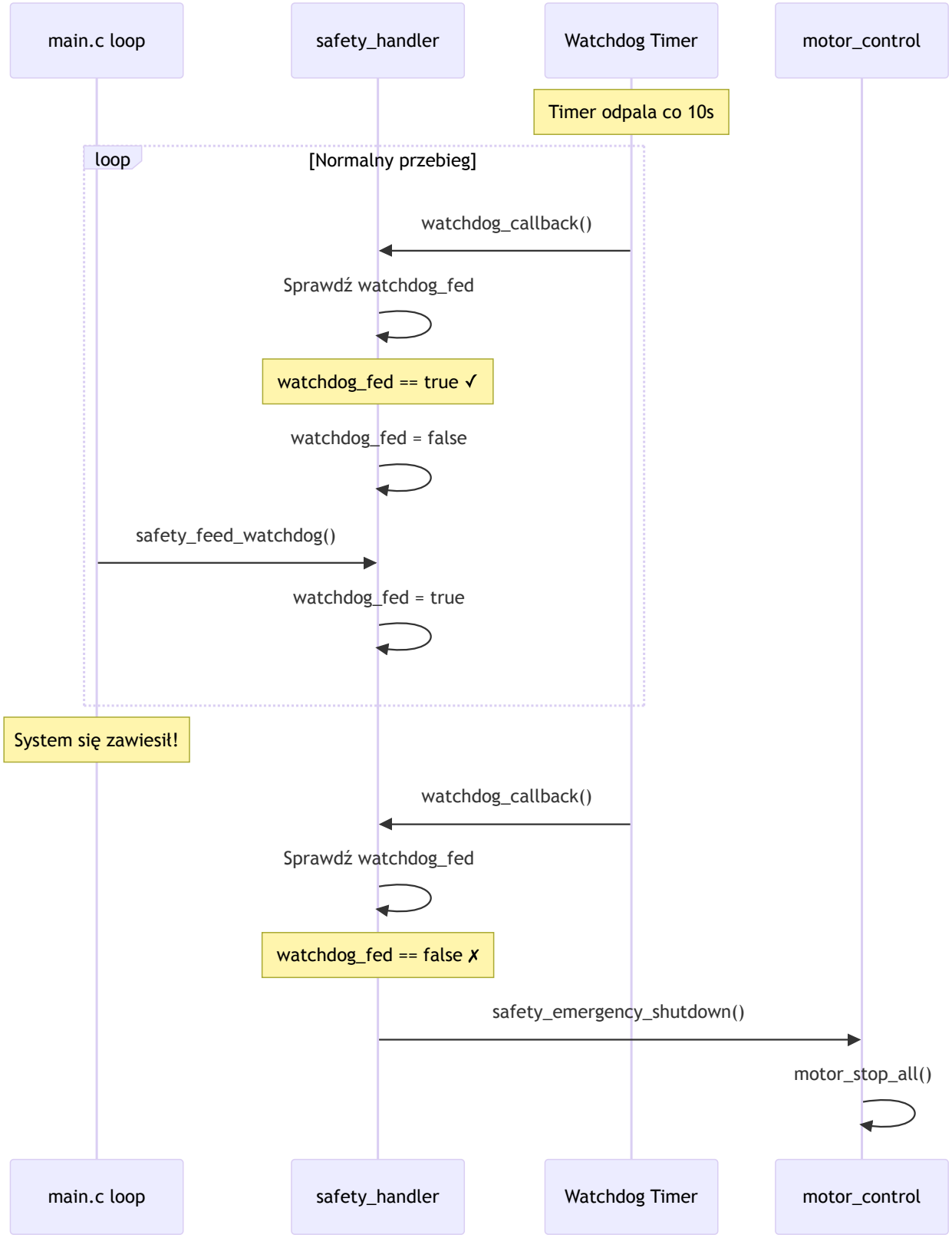
11.3.3 Funkcje bezpieczeństwa

```
// Linia 102-110: Awaryjne zatrzymanie
void safety_emergency_shutdown(void) {
    ESP_LOGE(TAG, "EMERGENCY SHUTDOWN");

    // Natychmiastowe zatrzymanie silników
    motor_stop_all();

    // Anulowanie zaplanowanego auto-stopu
    safety_cancel_auto_stop();
}

// Linia 112-114: Karmienie watchdoga
void safety_feed_watchdog(void) {
    s_safety.watchdog_fed = true;
    // Ustawienie flagi oznacza "system działa poprawnie"
    // Callback watchdoga sprawdzi tę flagę
```

```
}

// Linia 116-126: Walidacja czasu trwania
uint32_t safety_validate_duration(uint32_t duration_ms, uint32_t
max_duration_ms) {
    // 0 oznacza "ciągły ruch" - nie walidujemy
    if (duration_ms == 0) {
        return 0;
    }

    // Clamp do maksimum
    if (duration_ms > max_duration_ms) {
        return max_duration_ms;
    }

    return duration_ms;
}

// Linia 128-151: Planowanie auto-stopu
esp_err_t safety_schedule_auto_stop(uint32_t duration_ms) {
    if (!s_safety.initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    // 0 = ciągły ruch, nie planujemy stopu
    if (duration_ms == 0) {
        return ESP_OK;
    }

    // Anuluj poprzedni (jeśli był)
    safety_cancel_auto_stop();

    // Uruchom one-shot timer
    esp_err_t ret = esp_timer_start_once(
        s_safety.auto_stop_timer,
        duration_ms * 1000  // Konwersja ms → µs
    );

    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to schedule auto-stop");
        return ret;
    }
```

```
    ESP_LOGD(TAG, "Auto-stop scheduled in %lu ms", duration_ms);

    return ESP_OK;
}

// Linia 153-160: Anulowanie auto-stopu
void safety_cancel_auto_stop(void) {
    if (!s_safety.initialized) {
        return;
    }

    esp_timer_stop(s_safety.auto_stop_timer);
    // esp_timer_stop jest bezpieczne nawet gdy timer nie działa
    ESP_LOGD(TAG, "Auto-stop cancelled");
}
```

11.4 Diagram działania watchdoga



Rozdział 12: Komponent robot_core - Fasada

12.1 Wprowadzenie

Komponent `robot_core` implementuje wzorzec projektowy **Facade** (Fasada). Zamiast bezpośredniego wywoływania `motor_control`, `servo_control` i `safety_handler`, warstwy wyższe (HTTP API) używają prostego interfejsu `robot_move()`, `robot_turret()`, `robot_stop()`.

Korzyści:

- Uproszczenie API
- Centralna logika walidacji
- Łatwa wymiana implementacji (mock mode)
- Jeden punkt odpowiedzialności

12.2 Plik robot_types.h - Współdzielone typy

```
// Plik: components/robot_core/include/robot_types.h
/**
 * @file robot_types.h
 * @brief Shared type definitions for robot control
 */

#ifndef ROBOT_TYPES_H
#define ROBOT_TYPES_H

#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

// Kierunki ruchu/wieżyczki
typedef enum {
    ROBOT_DIR_FORWARD = 0,  // Do przodu
    ROBOT_DIR_BACKWARD,     // Do tyłu
    ROBOT_DIR_LEFT,         // W lewo
    ROBOT_DIR_RIGHT         // W prawo
} robot_direction_t;

// Typy akcji (dla odpowiedzi API)
typedef enum {
    ROBOT_ACTION_FORWARD = 0,
    ROBOT_ACTION_BACKWARD,
    ROBOT_ACTION_LEFT,
    ROBOT_ACTION_RIGHT,
    ROBOT_ACTION_TURRET_LEFT,
    ROBOT_ACTION_TURRET_RIGHT,
    ROBOT_ACTION_STOP_ALL
} robot_action_t;
```

```c
// Tryby silnika (tablica prawdy DRV8833)
typedef enum {
    MOTOR_MODE_COAST = 0,  // IN1=LOW,  IN2=LOW  - swobodny
    MOTOR_MODE_FORWARD,    // IN1=HIGH, IN2=LOW  - przód
    MOTOR_MODE_BACKWARD,   // IN1=LOW,  IN2=HIGH - tył
    MOTOR_MODE_BRAKE       // IN1=HIGH, IN2=HIGH - hamowanie
} motor_mode_t;

// Funkcje pomocnicze - inline dla szybkości
static inline const char *robot_direction_to_str(robot_direction_t dir) {
    switch (dir) {
        case ROBOT_DIR_FORWARD:  return "forward";
        case ROBOT_DIR_BACKWARD: return "backward";
        case ROBOT_DIR_LEFT:     return "left";
        case ROBOT_DIR_RIGHT:    return "right";
        default: return "unknown";
    }
}

static inline const char *robot_action_to_str(robot_action_t action) {
    switch (action) {
        case ROBOT_ACTION_FORWARD:      return "forward";
        case ROBOT_ACTION_BACKWARD:     return "backward";
        case ROBOT_ACTION_LEFT:         return "left";
        case ROBOT_ACTION_RIGHT:        return "right";
        case ROBOT_ACTION_TURRET_LEFT:  return "turret_left";
        case ROBOT_ACTION_TURRET_RIGHT: return "turret_right";
        case ROBOT_ACTION_STOP_ALL:     return "stop_all";
        default: return "unknown";
    }
}

#ifdef __cplusplus
}
#endif

#endif /* ROBOT_TYPES_H */
```

12.3 Plik nagłówkowy robot.h

```c
// Plik: components/robot_core/include/robot.h
/**
 * @file robot.h
 * @brief Robot controller facade interface
```

```
 */

#ifndef ROBOT_H
#define ROBOT_H

#include <esp_err.h>
#include <stdbool.h>
#include <stdint.h>
#include "robot_types.h"

#ifdef __cplusplus
extern "C" {
#endif

// Konfiguracja robota
typedef struct {
    uint32_t movement_timeout_ms;  // Max czas ruchu
    uint32_t turret_timeout_ms;    // Max czas wieżyczki
    bool gpio_enabled;             // true = sprzęt, false = mock
    const char *camera_url;        // URL streamu
} robot_config_t;

// Wynik operacji
typedef struct {
    robot_action_t action;   // Wykonana akcja
    uint32_t duration_ms;    // Faktyczny czas
    bool success;            // Czy sukces?
} robot_result_t;

// Status robota
typedef struct {
    bool connected;          // Czy aktywny?
    bool gpio_enabled;       // Czy tryb sprzętowy?
    const char *camera_url;  // URL kamery
} robot_status_t;

// API fasady
esp_err_t robot_init(const robot_config_t *config);
robot_result_t robot_move(robot_direction_t direction, uint32_t
duration_ms);
robot_result_t robot_turret(robot_direction_t direction, uint32_t
duration_ms);
robot_result_t robot_stop(void);
```

```
robot_status_t robot_get_status(void);
void robot_cleanup(void);

#ifdef __cplusplus
}
#endif

#endif /* ROBOT_H */
```

12.4 Implementacja robot.c

```
// Plik: components/robot_core/src/robot.c
#include "robot.h"
#include <esp_log.h>
#include <string.h>
#include "motor_control.h"
#include "safety_handler.h"
#include "servo_control.h"

static const char *TAG = "robot";

// Stan robota - singleton
static struct {
    bool initialized;
    robot_config_t config;
} s_robot = {0};
```

12.4.1 Inicjalizacja

```
// Linia 24-36: Inicjalizacja fasady
esp_err_t robot_init(const robot_config_t *config) {
    if (config == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    // Kopiowanie konfiguracji
    memcpy(&s_robot.config, config, sizeof(robot_config_t));
    s_robot.initialized = true;

    ESP_LOGI(TAG, "Robot initialized (gpio_enabled=%d, movement_timeout=%lu,
turret_timeout=%lu)",
             config->gpio_enabled,
             config->movement_timeout_ms,
             config->turret_timeout_ms);
```

```
    return ESP_OK;
}
```

12.4.2 Funkcja robot_move()

```
// Linia 38-105: Ruch robota
robot_result_t robot_move(robot_direction_t direction, uint32_t duration_ms)
{
    // Domyślny wynik - błąd
    robot_result_t result = {
        .success = false,
        .duration_ms = 0,
        .action = ROBOT_ACTION_STOP_ALL
    };

    if (!s_robot.initialized) {
        ESP_LOGE(TAG, "Robot not initialized");
        return result;
    }

    // === Walidacja czasu trwania ===
    uint32_t validated_duration = safety_validate_duration(
        duration_ms,
        s_robot.config.movement_timeout_ms  // Max np. 5000 ms
    );

    // Log jeśli było ograniczenie
    if (validated_duration != duration_ms && duration_ms > 0) {
        ESP_LOGW(TAG, "Duration clamped from %lu to %lu ms",
                 duration_ms, validated_duration);
    }

    esp_err_t err = ESP_OK;

    // === Dispatch do odpowiedniej funkcji silników ===
    switch (direction) {
        case ROBOT_DIR_FORWARD:
            result.action = ROBOT_ACTION_FORWARD;
            if (s_robot.config.gpio_enabled) {
                err = motor_move_forward(validated_duration);
            }
            break;

        case ROBOT_DIR_BACKWARD:
```

```
            result.action = ROBOT_ACTION_BACKWARD;
            if (s_robot.config.gpio_enabled) {
                err = motor_move_backward(validated_duration);
            }
            break;

        case ROBOT_DIR_LEFT:
            result.action = ROBOT_ACTION_LEFT;
            if (s_robot.config.gpio_enabled) {
                err = motor_turn_left(validated_duration);
            }
            break;

        case ROBOT_DIR_RIGHT:
            result.action = ROBOT_ACTION_RIGHT;
            if (s_robot.config.gpio_enabled) {
                err = motor_turn_right(validated_duration);
            }
            break;

        default:
            ESP_LOGE(TAG, "Invalid direction: %d", direction);
            return result;
    }

    // === Obsługa wyniku ===
    if (err == ESP_OK) {
        result.success = true;
        result.duration_ms = validated_duration;

        // Zaplanuj auto-stop jeśli podano czas
        if (validated_duration > 0) {
            safety_schedule_auto_stop(validated_duration);
        }

        ESP_LOGI(TAG, "Move %s for %lu ms",
                 robot_action_to_str(result.action), validated_duration);
    } else {
        ESP_LOGE(TAG, "Move failed: %s", esp_err_to_name(err));
    }

    return result;
}
```

12.4.3 Funkcja robot_turret()

```
// Linia 107-150: Sterowanie wieżyczką
robot_result_t robot_turret(robot_direction_t direction, uint32_t
duration_ms) {
    robot_result_t result = {
        .success = false,
        .duration_ms = 0,
        .action = ROBOT_ACTION_STOP_ALL
    };

    if (!s_robot.initialized) {
        ESP_LOGE(TAG, "Robot not initialized");
        return result;
    }

    // Walidacja (dla logów, serwo używa kroków)
    uint32_t validated_duration = safety_validate_duration(
        duration_ms,
        s_robot.config.turret_timeout_ms
    );

    esp_err_t err = ESP_OK;

    // Tylko LEFT i RIGHT są dozwolone dla wieżyczki
    switch (direction) {
        case ROBOT_DIR_LEFT:
            result.action = ROBOT_ACTION_TURRET_LEFT;
            if (s_robot.config.gpio_enabled) {
                err = servo_step_left();  // Krok w lewo
            }
            break;

        case ROBOT_DIR_RIGHT:
            result.action = ROBOT_ACTION_TURRET_RIGHT;
            if (s_robot.config.gpio_enabled) {
                err = servo_step_right();  // Krok w prawo
            }
            break;

        default:
            ESP_LOGE(TAG, "Invalid turret direction: %d", direction);
            return result;
    }
```

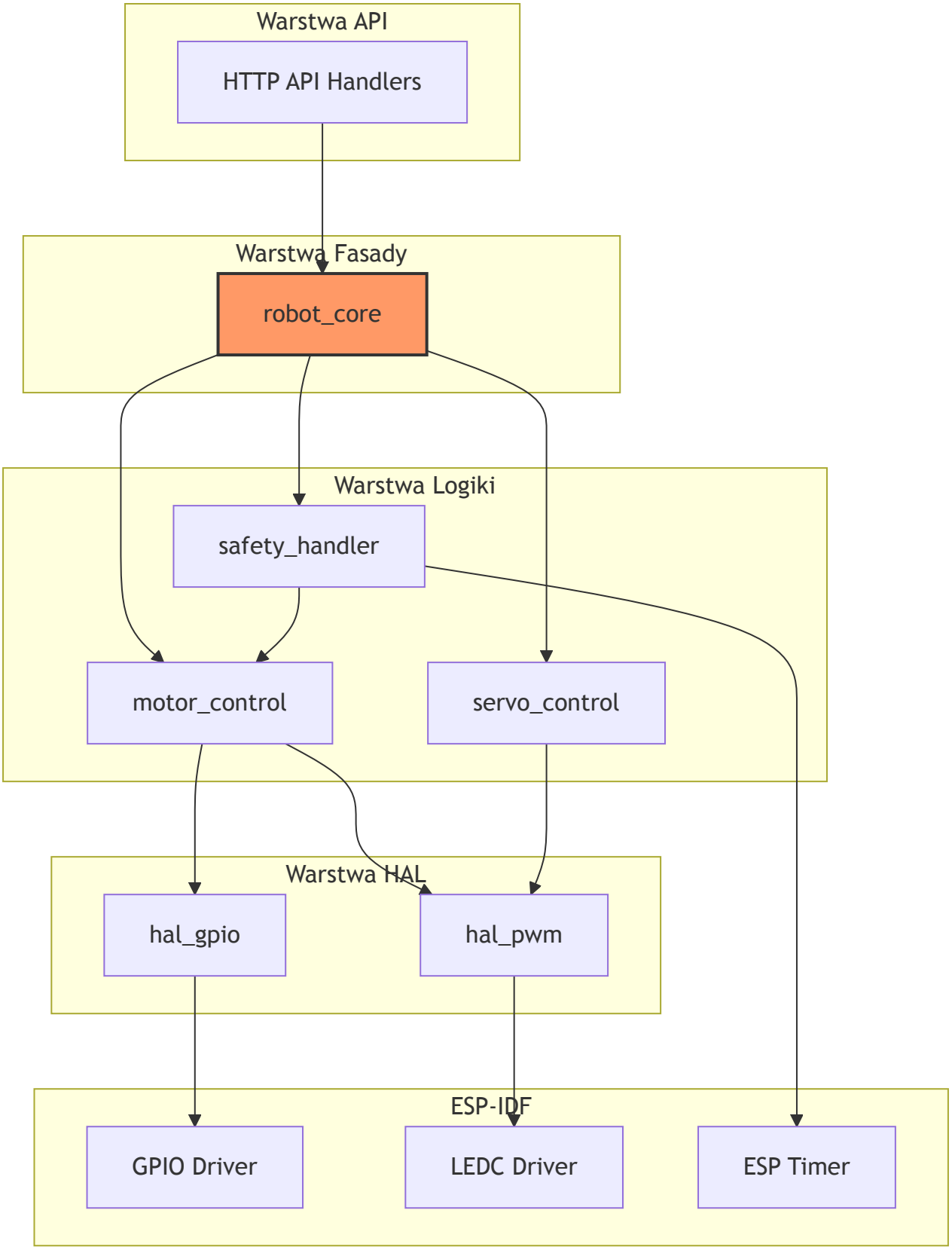
```
    if (err == ESP_OK) {
        result.success = true;
        result.duration_ms = validated_duration;
        ESP_LOGI(TAG, "Turret %s", robot_action_to_str(result.action));
    } else {
        ESP_LOGE(TAG, "Turret failed: %s", esp_err_to_name(err));
    }

    return result;
}
```

12.4.4 Stop i status

```
// Linia 152-175: Awaryjny stop
robot_result_t robot_stop(void) {
    robot_result_t result = {
        .action = ROBOT_ACTION_STOP_ALL,
        .duration_ms = 0,
        .success = true  // Domyślnie sukces
    };

    if (!s_robot.initialized) {
        ESP_LOGE(TAG, "Robot not initialized");
        result.success = false;
        return result;
    }

    // Anuluj zaplanowany auto-stop
    safety_cancel_auto_stop();

    // Zatrzymaj silniki
    if (s_robot.config.gpio_enabled) {
        esp_err_t err = motor_stop_all();
        if (err != ESP_OK) {
            ESP_LOGE(TAG, "Motor stop failed: %s", esp_err_to_name(err));
            result.success = false;
        }
    }

    ESP_LOGI(TAG, "Emergency stop");
    return result;
}
```

```c
// Linia 177-183: Pobranie statusu
robot_status_t robot_get_status(void) {
    robot_status_t status = {
        .connected = s_robot.initialized,
        .gpio_enabled = s_robot.config.gpio_enabled,
        .camera_url = s_robot.config.camera_url
    };

    return status;
}

// Linia 185-201: Cleanup
void robot_cleanup(void) {
    if (!s_robot.initialized) {
        return;
    }

    ESP_LOGI(TAG, "Robot cleanup");

    safety_cancel_auto_stop();

    if (s_robot.config.gpio_enabled) {
        motor_stop_all();
        motor_control_cleanup();
        servo_cleanup();
    }

    s_robot.initialized = false;
}
```

12.5 Diagram warstw



12.6 Ćwiczenie

Ćwiczenie 12.1: Dodanie trybu turbo

Zadanie: Dodaj funkcję `robot_move_turbo()` która pomija soft-start i od razu ustawia 100% PWM.

Rozwiązanie:

```
// W robot.h:
robot_result_t robot_move_turbo(robot_direction_t direction, uint32_t
duration_ms);

// W robot.c:
robot_result_t robot_move_turbo(robot_direction_t direction, uint32_t
duration_ms) {
    robot_result_t result = {.success = false, .duration_ms = 0, .action =
ROBOT_ACTION_STOP_ALL};

    if (!s_robot.initialized) {
        return result;
    }

    uint32_t validated_duration = safety_validate_duration(
        duration_ms, s_robot.config.movement_timeout_ms);

    // Bezpośrednie ustawienie PWM 100% (bez rampy)
    if (s_robot.config.gpio_enabled) {
        pwm_ramper_set_duty(100);  // Natychmiastowe 100%
    }

    // Reszta jak w robot_move()...
    // (ustawienie kierunku silników, etc.)

    return result;
}
```

CZĘŚĆ III: HTTP API

Rozdział 13: Komponent http_server - Serwer REST

13.1 Wprowadzenie

Komponent `http_server` uruchamia serwer HTTP na ESP32, który obsługuje REST API i serwuje pliki statyczne (Web UI). ESP-IDF dostarcza lekką bibliotekę `esp_http_server` idealną dla embedded.

Główne funkcje:

- Uruchomienie serwera HTTP na porcie 4567
- Rejestracja endpointów API
- Serwowanie plików z SPIFFS

13.2 Architektura serwera

13.3 Plik nagłówkowy http_server.h

```
// Plik: components/http_server/include/http_server.h
/**
 * @file http_server.h
 * @brief HTTP server initialization and management
 */

#ifndef HTTP_SERVER_H
#define HTTP_SERVER_H

#include <esp_err.h>
#include <esp_http_server.h>  // httpd_handle_t
#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

// Konfiguracja serwera
typedef struct {
    uint16_t port;          // Port serwera (np. 4567)
    const char *base_path;  // Ścieżka bazowa SPIFFS (np. "/spiffs")
    bool auth_enabled;      // Uwierzytelnianie (przyszła funkcja)
} http_server_config_t;

// Start serwera - zwraca handle lub NULL
httpd_handle_t http_server_start(const http_server_config_t *config);

// Stop serwera
esp_err_t http_server_stop(httpd_handle_t server);

#ifdef __cplusplus
}
#endif
```

```
#endif /* HTTP_SERVER_H */
```

13.4 Implementacja http_server.c

```
// Plik: components/http_server/src/http_server.c
/**
 * @file http_server.c
 * @brief HTTP server initialization and management implementation
 */

#include "http_server.h"
#include <esp_log.h>
#include "api_handlers.h"  // api_handlers_register()

static const char *TAG = "http_server";

// Linia 14-46: Uruchomienie serwera
httpd_handle_t http_server_start(const http_server_config_t *config) {
    if (config == NULL) {
        ESP_LOGE(TAG, "Invalid config");
        return NULL;
    }

    // === Konfiguracja serwera HTTP ===
    httpd_config_t httpd_config = HTTPD_DEFAULT_CONFIG();
    // Makro ustawiające domyślne wartości

    // Nadpisanie wartości domyślnych
    httpd_config.server_port = config->port;     // Port np. 4567
    httpd_config.max_uri_handlers = 20;          // Max 20 endpointów
    httpd_config.max_resp_headers = 8;           // Max 8 nagłówków
    httpd_config.stack_size = 8192;              // Stos taska (bajty)
    httpd_config.lru_purge_enable = true;        // Usuwaj stare połączenia

    httpd_handle_t server = NULL;

    // === Start serwera ===
    esp_err_t ret = httpd_start(&server, &httpd_config);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to start HTTP server: %s",
                 esp_err_to_name(ret));
        return NULL;
    }
```

```
    // === Rejestracja handlerów API ===
    ret = api_handlers_register(server, config->base_path);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Failed to register API handlers");
        httpd_stop(server);  // Cleanup przy błędzie
        return NULL;
    }

    ESP_LOGI(TAG, "HTTP server started on port %d", config->port);

    return server;
}

// Linia 48-59: Stop serwera
esp_err_t http_server_stop(httpd_handle_t server) {
    if (server == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    esp_err_t ret = httpd_stop(server);
    if (ret == ESP_OK) {
        ESP_LOGI(TAG, "HTTP server stopped");
    }

    return ret;
}
```

> **Dokumentacja ESP-IDF:**
>
> - HTTP Server

13.5 Helpery API - api_helpers.c

```
// Plik: components/http_server/src/api_helpers.c
/**
 * @file api_helpers.c
 * @brief API response and parsing helpers implementation
 */

#include "api_helpers.h"
#include <esp_log.h>
#include <stdio.h>
#include <string.h>
```

```c
#include "robot_types.h"

static const char *TAG = "api_helpers";
```

13.5.1 Wysyłanie odpowiedzi JSON

```c
// Linia 17-36: Sukces
esp_err_t api_send_success(httpd_req_t *req, const char *json_body) {
    // Ustawienie nagłówków CORS (Cross-Origin Resource Sharing)
    api_set_cors_headers(req);

    // Typ odpowiedzi: JSON
    httpd_resp_set_type(req, "application/json");

    char response[512];
    int len;

    // Formatowanie odpowiedzi
    if (json_body == NULL || strlen(json_body) == 0) {
        // Puste body → tylko {"success":true}
        len = snprintf(response, sizeof(response), "{\"success\":true}");
    } else {
        // Body podane → {"success":true, ...body...}
        len = snprintf(response, sizeof(response),
                       "{\"success\":true,%s}", json_body);
    }

    // Sprawdzenie overflow
    if (len >= (int)sizeof(response)) {
        ESP_LOGE(TAG, "Response buffer overflow");
        return ESP_FAIL;
    }

    // Wysłanie odpowiedzi
    return httpd_resp_send(req, response, len);
}

// Linia 38-51: Błąd
esp_err_t api_send_error(httpd_req_t *req, int status_code, const char
*message) {
    api_set_cors_headers(req);
    httpd_resp_set_type(req, "application/json");

    // Ustawienie kodu statusu HTTP (np. "400", "500")
```

```
    char status_str[4];
    snprintf(status_str, sizeof(status_str), "%d", status_code);
    httpd_resp_set_status(req, status_str);

    // Formatowanie błędu
    char response[256];
    int len = snprintf(response, sizeof(response),
                       "{\"success\":false,\"error\":\"%s\"}",
                       message ? message : "Unknown error");

    return httpd_resp_send(req, response, len);
}
```

13.5.2 Odczyt body żądania

```
// Linia 53-79: Odczyt body
esp_err_t api_read_body(httpd_req_t *req, char *buffer, size_t buffer_size)
{
    if (req == NULL || buffer == NULL || buffer_size == 0) {
        return ESP_ERR_INVALID_ARG;
    }

    // Długość body
    int content_len = req->content_len;

    // Puste body
    if (content_len <= 0) {
        buffer[0] = '\0';
        return ESP_OK;
    }

    // Sprawdzenie rozmiaru
    if ((size_t)content_len >= buffer_size) {
        ESP_LOGE(TAG, "Body too large: %d >= %zu", content_len,
buffer_size);
        return ESP_FAIL;
    }

    // Odczyt danych
    int received = httpd_req_recv(req, buffer, content_len);
    if (received != content_len) {
        ESP_LOGE(TAG, "Failed to receive body: %d != %d",
                 received, content_len);
        return ESP_FAIL;
```

```
    }

    // Null-terminate
    buffer[content_len] = '\0';

    return ESP_OK;
}
```

13.5.3 Parsowanie kierunku i CORS

```
// Linia 81-104: Parsowanie kierunku z JSON
bool api_parse_direction(const char *direction, int *result) {
    if (direction == NULL || result == NULL) {
        return false;
    }

    // strcmp zwraca 0 gdy stringi są równe
    if (strcmp(direction, "forward") == 0) {
        *result = ROBOT_DIR_FORWARD;
        return true;
    }
    if (strcmp(direction, "backward") == 0) {
        *result = ROBOT_DIR_BACKWARD;
        return true;
    }
    if (strcmp(direction, "left") == 0) {
        *result = ROBOT_DIR_LEFT;
        return true;
    }
    if (strcmp(direction, "right") == 0) {
        *result = ROBOT_DIR_RIGHT;
        return true;
    }

    return false;  // Nieznany kierunek
}

// Linia 106-110: Nagłówki CORS
void api_set_cors_headers(httpd_req_t *req) {
    // Pozwól na żądania z dowolnej domeny
    httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");

    // Dozwolone metody HTTP
    httpd_resp_set_hdr(req, "Access-Control-Allow-Methods",
```

```
                        "GET, POST, PUT, DELETE, OPTIONS");

    // Dozwolone nagłówki
    httpd_resp_set_hdr(req, "Access-Control-Allow-Headers",
                        "Content-Type, Authorization");
}

// CORS jest niezbędny gdy Web UI jest ładowany z innej domeny
// niż API (np. podczas developmentu)
```

13.5.4 Określanie typu MIME

```
// Linia 112-154: Typ MIME na podstawie rozszerzenia pliku
const char *api_get_mime_type(const char *filename) {
    if (filename == NULL) {
        return "application/octet-stream";
    }

    // Znajdź ostatnią kropkę
    const char *ext = strrchr(filename, '.');
    if (ext == NULL) {
        return "application/octet-stream";
    }

    // Mapowanie rozszerzenie → MIME type
    if (strcmp(ext, ".html") == 0 || strcmp(ext, ".htm") == 0) {
        return "text/html";
    }
    if (strcmp(ext, ".css") == 0) {
        return "text/css";
    }
    if (strcmp(ext, ".js") == 0) {
        return "application/javascript";
    }
    if (strcmp(ext, ".json") == 0) {
        return "application/json";
    }
    if (strcmp(ext, ".yaml") == 0 || strcmp(ext, ".yml") == 0) {
        return "text/yaml";
    }
    if (strcmp(ext, ".png") == 0) {
        return "image/png";
    }
    if (strcmp(ext, ".jpg") == 0 || strcmp(ext, ".jpeg") == 0) {
```

```
        return "image/jpeg";
    }
    if (strcmp(ext, ".svg") == 0) {
        return "image/svg+xml";
    }
    if (strcmp(ext, ".ico") == 0) {
        return "image/x-icon";
    }

    return "application/octet-stream";  // Domyślny
}
```

Rozdział 14: API Handlers - Endpointy REST

14.1 Wprowadzenie

Plik `api_handlers.c` zawiera implementacje wszystkich endpointów REST API. Każdy handler:

1. Odczytuje body żądania (dla POST)
2. Parsuje JSON używając biblioteki cJSON
3. Wywołuje odpowiednią funkcję fasady `robot_core`
4. Zwraca odpowiedź JSON

14.2 Biblioteka cJSON

ESP-IDF zawiera wbudowaną bibliotekę cJSON do parsowania i generowania JSON:

```
#include <cJSON.h>

// Parsowanie stringa JSON
cJSON *json = cJSON_Parse("{\"direction\":\"forward\",\"duration\":1000}");

// Pobieranie wartości
cJSON *dir = cJSON_GetObjectItem(json, "direction");
if (cJSON_IsString(dir)) {
    printf("Direction: %s\n", dir->valuestring);
}

cJSON *dur = cJSON_GetObjectItem(json, "duration");
if (cJSON_IsNumber(dur)) {
    printf("Duration: %d\n", (int)dur->valuedouble);
}
```

```
// WAŻNE: Zwolnij pamięć!
cJSON_Delete(json);
```

14.3 Implementacja api_handlers.c

```
// Plik: components/http_server/src/api_handlers.c
#include "api_handlers.h"
#include <esp_log.h>
#include <cJSON.h>          // Parsowanie JSON
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>       // stat() dla plików
#include "api_helpers.h"
#include "camera_stream.h"
#include "robot.h"

static const char *TAG = "api_handlers";

// Ścieżka bazowa SPIFFS
static const char *s_base_path = NULL;
```

14.3.1 Serwowanie plików statycznych

```
// Linia 27-65: Serwowanie pliku z SPIFFS
/**
 * @brief Serve static file from SPIFFS
 */
static esp_err_t serve_static_file(httpd_req_t *req, const char *filename) {
    if (s_base_path == NULL) {
        return api_send_error(req, 500, "SPIFFS not mounted");
    }

    // Budowanie pełnej ścieżki
    char filepath[256];
    snprintf(filepath, sizeof(filepath), "%s/%s", s_base_path, filename);
    // np. "/spiffs/index.html"

    // Sprawdzenie czy plik istnieje
    struct stat file_stat;
    if (stat(filepath, &file_stat) != 0) {
        ESP_LOGE(TAG, "File not found: %s", filepath);
        return api_send_error(req, 404, "File not found");
    }

    // Otwarcie pliku
```

```
    FILE *f = fopen(filepath, "r");
    if (f == NULL) {
        ESP_LOGE(TAG, "Failed to open file: %s", filepath);
        return api_send_error(req, 500, "Failed to open file");
    }

    // Ustawienie typu MIME
    httpd_resp_set_type(req, api_get_mime_type(filename));
    api_set_cors_headers(req);

    // Wysyłanie chunkami (dla dużych plików)
    char buf[512];
    size_t read_bytes;

    while ((read_bytes = fread(buf, 1, sizeof(buf), f)) > 0) {
        if (httpd_resp_send_chunk(req, buf, read_bytes) != ESP_OK) {
            fclose(f);
            ESP_LOGE(TAG, "Failed to send file chunk");
            return ESP_FAIL;
        }
    }

    fclose(f);

    // Zakończenie chunked transfer
    httpd_resp_send_chunk(req, NULL, 0);

    return ESP_OK;
}
```

14.3.2 Handler POST /api/v1/move

```
// Linia 76-123: Handler ruchu
esp_err_t api_handle_move(httpd_req_t *req) {
    char body[API_MAX_BODY_SIZE];  // 512 bajtów

    // === Krok 1: Odczyt body ===
    esp_err_t ret = api_read_body(req, body, sizeof(body));
    if (ret != ESP_OK) {
        return api_send_error(req, 400, "Failed to read body");
    }

    // === Krok 2: Parsowanie JSON ===
    cJSON *json = cJSON_Parse(body);
```

```c
    if (json == NULL) {
        return api_send_error(req, 400, "Invalid JSON");
    }

    // === Krok 3: Pobieranie "direction" ===
    cJSON *dir_json = cJSON_GetObjectItem(json, "direction");
    if (!cJSON_IsString(dir_json)) {
        cJSON_Delete(json);
        return api_send_error(req, 400, "Missing direction");
    }

    int direction;
    if (!api_parse_direction(dir_json->valuestring, &direction)) {
        cJSON_Delete(json);
        return api_send_error(req, 400, "Invalid direction");
    }

    // === Krok 4: Pobieranie "duration" (opcjonalne) ===
    uint32_t duration = 0;  // Domyślnie: ciągły ruch
    cJSON *dur_json = cJSON_GetObjectItem(json, "duration");
    if (cJSON_IsNumber(dur_json)) {
        duration = (uint32_t)dur_json->valuedouble;
    }

    // WAŻNE: Zwolnij pamięć JSON!
    cJSON_Delete(json);

    // === Krok 5: Wywołanie fasady ===
    robot_result_t result = robot_move((robot_direction_t)direction,
duration);

    // === Krok 6: Odpowiedź ===
    if (!result.success) {
        return api_send_error(req, 500, "Move failed");
    }

    // Formatowanie odpowiedzi
    char response[128];
    snprintf(response, sizeof(response),
             "\"action\":\"%s\",\"duration\":%lu",
             robot_action_to_str(result.action),
             (unsigned long)result.duration_ms);
```

```
    return api_send_success(req, response);
}
```

14.3.3 Handler POST /api/v1/turret

```
// Linia 125-176: Handler wieżyczki
esp_err_t api_handle_turret(httpd_req_t *req) {
    char body[API_MAX_BODY_SIZE];

    esp_err_t ret = api_read_body(req, body, sizeof(body));
    if (ret != ESP_OK) {
        return api_send_error(req, 400, "Failed to read body");
    }

    cJSON *json = cJSON_Parse(body);
    if (json == NULL) {
        return api_send_error(req, 400, "Invalid JSON");
    }

    // Parsowanie kierunku
    cJSON *dir_json = cJSON_GetObjectItem(json, "direction");
    if (!cJSON_IsString(dir_json)) {
        cJSON_Delete(json);
        return api_send_error(req, 400, "Missing direction");
    }

    // Wieżyczka obsługuje tylko left/right
    int direction;
    if (strcmp(dir_json->valuestring, "left") == 0) {
        direction = ROBOT_DIR_LEFT;
    } else if (strcmp(dir_json->valuestring, "right") == 0) {
        direction = ROBOT_DIR_RIGHT;
    } else {
        cJSON_Delete(json);
        return api_send_error(req, 400, "Invalid direction (use
left/right)");
    }

    // Duration (opcjonalne)
    uint32_t duration = 0;
    cJSON *dur_json = cJSON_GetObjectItem(json, "duration");
    if (cJSON_IsNumber(dur_json)) {
        duration = (uint32_t)dur_json->valuedouble;
    }
```

```
    cJSON_Delete(json);

    // Wywołanie fasady
    robot_result_t result = robot_turret((robot_direction_t)direction,
duration);

    if (!result.success) {
        return api_send_error(req, 500, "Turret failed");
    }

    char response[128];
    snprintf(response, sizeof(response),
             "\"action\":\"%s\"",
             robot_action_to_str(result.action));

    return api_send_success(req, response);
}
```

14.3.4 Pozostałe handlery

```
// Linia 178-186: Stop
esp_err_t api_handle_stop(httpd_req_t *req) {
    robot_result_t result = robot_stop();

    if (!result.success) {
        return api_send_error(req, 500, "Stop failed");
    }

    return api_send_success(req, "\"action\":\"stop\"");
}

// Linia 188-196: Status
esp_err_t api_handle_status(httpd_req_t *req) {
    robot_status_t status = robot_get_status();

    char response[256];
    snprintf(response, sizeof(response),
             "\"connected\":%s,\"gpio_enabled\":%s",
             status.connected ? "true" : "false",
             status.gpio_enabled ? "true" : "false");

    return api_send_success(req, response);
}
```

```c
// Linia 198-209: Camera URL
esp_err_t api_handle_camera(httpd_req_t *req) {
    if (!camera_stream_is_ready()) {
        return api_send_error(req, 503, "Camera not ready");
    }

    const char *path = camera_stream_get_path();  // "/stream"

    char response[128];
    snprintf(response, sizeof(response), "\"stream_url\":\"%s\"", path);

    return api_send_success(req, response);
}

// Linia 211-217: Health check
esp_err_t api_handle_health(httpd_req_t *req) {
    api_set_cors_headers(req);
    httpd_resp_set_type(req, "application/json");

    const char *response = "{\"status\":\"ok\"}";
    return httpd_resp_send(req, response, strlen(response));
}

// Linia 219-225: Index i Docs
esp_err_t api_handle_index(httpd_req_t *req) {
    return serve_static_file(req, "index.html");
}

esp_err_t api_handle_docs(httpd_req_t *req) {
    return serve_static_file(req, "docs.html");
}
```

14.3.5 Rejestracja routów

```c
// Linia 241-308: Rejestracja wszystkich endpointów
esp_err_t api_handlers_register(httpd_handle_t server, const char
*base_path) {
    if (server == NULL) {
        return ESP_ERR_INVALID_ARG;
    }

    s_base_path = base_path;  // Zapisz ścieżkę SPIFFS
```

```
// === Tablica routów API ===
static const httpd_uri_t routes[] = {
    // POST endpoints
    {.uri = "/api/v1/move",
     .method = HTTP_POST,
     .handler = api_handle_move,
     .user_ctx = NULL},

    {.uri = "/api/v1/turret",
     .method = HTTP_POST,
     .handler = api_handle_turret,
     .user_ctx = NULL},

    {.uri = "/api/v1/stop",
     .method = HTTP_POST,
     .handler = api_handle_stop,
     .user_ctx = NULL},

    // GET endpoints
    {.uri = "/api/v1/status",
     .method = HTTP_GET,
     .handler = api_handle_status,
     .user_ctx = NULL},

    {.uri = "/api/v1/camera",
     .method = HTTP_GET,
     .handler = api_handle_camera,
     .user_ctx = NULL},

    {.uri = "/health",
     .method = HTTP_GET,
     .handler = api_handle_health,
     .user_ctx = NULL},

    // Static files
    {.uri = "/",
     .method = HTTP_GET,
     .handler = api_handle_index,
     .user_ctx = NULL},

    {.uri = "/docs",
     .method = HTTP_GET,
     .handler = api_handle_docs,
```

```
         .user_ctx = NULL},

        // CORS preflight handlers (OPTIONS)
        {.uri = "/api/v1/move",
         .method = HTTP_OPTIONS,
         .handler = handle_options,
         .user_ctx = NULL},
        // ... więcej OPTIONS handlers
    };

    // Rejestracja każdego routu
    for (size_t i = 0; i < sizeof(routes) / sizeof(routes[0]); i++) {
        esp_err_t ret = httpd_register_uri_handler(server, &routes[i]);
        if (ret != ESP_OK) {
            ESP_LOGE(TAG, "Failed to register %s", routes[i].uri);
            return ret;
        }
    }

    ESP_LOGI(TAG, "API handlers registered");

    return ESP_OK;
}
```

14.4 Testowanie API z curl

```
# Health check
curl http://10.42.0.1:4567/health
# {"status":"ok"}

# Status robota
curl http://10.42.0.1:4567/api/v1/status
# {"success":true,"connected":true,"gpio_enabled":true}

# Ruch do przodu przez 1 sekundę
curl -X POST http://10.42.0.1:4567/api/v1/move \
  -H "Content-Type: application/json" \
  -d '{"direction":"forward","duration":1000}'
# {"success":true,"action":"forward","duration":1000}

# Wieżyczka w lewo
curl -X POST http://10.42.0.1:4567/api/v1/turret \
  -H "Content-Type: application/json" \
  -d '{"direction":"left"}'
```

```
# {"success":true,"action":"turret_left"}

# Stop awaryjny
curl -X POST http://10.42.0.1:4567/api/v1/stop
# {"success":true,"action":"stop"}

# URL kamery
curl http://10.42.0.1:4567/api/v1/camera
# {"success":true,"stream_url":"/stream"}
```

CZĘŚĆ IV: TESTOWANIE I DEPLOYMENT

Rozdział 15: Pisanie Testów z Unity Framework

15.1 Wprowadzenie do Unity

Unity to lekki framework do testów jednostkowych w C. ESP-IDF zawiera go jako część komponentu `unity`.

Dlaczego testy?

- Wykrywanie błędów wcześnie
- Dokumentacja przez przykład
- Bezpieczny refaktoring
- CI/CD pipeline

15.2 Struktura testów w ESP-IDF

```
esp32-robot/
├── main/                    # Główna aplikacja
├── components/              # Komponenty
└── test/
    └── main/
        ├── test_main.c           # Test runner
        ├── test_robot.c          # Testy robot_core
        ├── test_motor_control.c  # Testy motor_control
        ├── test_servo_control.c  # Testy servo_control
        └── test_safety_handler.c # Testy safety_handler
```

15.3 Test Runner - test_main.c

```
// Plik: test/main/test_main.c
/**
```

```c
 * @file test_main.c
 * @brief Unity test runner entry point
 */

#include <unity.h>

// === Deklaracje funkcji testowych ===
// Każdy plik test_*.c eksportuje swoje funkcje

// Testy robot_core
extern void test_robot_init(void);
extern void test_robot_move_forward(void);
extern void test_robot_move_backward(void);
extern void test_robot_turn_left(void);
extern void test_robot_turn_right(void);
extern void test_robot_stop(void);

// Testy motor_control
extern void test_motor_control_init(void);
extern void test_motor_direction_forward(void);
extern void test_motor_direction_backward(void);
extern void test_motor_stop(void);

// Testy servo_control
extern void test_servo_init(void);
extern void test_servo_step_left(void);
extern void test_servo_step_right(void);
extern void test_servo_center(void);

// Testy safety_handler
extern void test_safety_validate_duration(void);
extern void test_safety_auto_stop(void);

// === Setup i Teardown ===
void setUp(void) {
    // Wywoływane PRZED każdym testem
    // Można tu zresetować stan, zainicjalizować mock, etc.
}

void tearDown(void) {
    // Wywoływane PO każdym teście
    // Cleanup, zwolnienie zasobów
}
```

```c
// === Entry point ===
void app_main(void) {
    // Rozpoczęcie sesji testów
    UNITY_BEGIN();

    // Uruchomienie testów robot_core
    RUN_TEST(test_robot_init);
    RUN_TEST(test_robot_move_forward);
    RUN_TEST(test_robot_move_backward);
    RUN_TEST(test_robot_turn_left);
    RUN_TEST(test_robot_turn_right);
    RUN_TEST(test_robot_stop);

    // Uruchomienie testów motor_control
    RUN_TEST(test_motor_control_init);
    RUN_TEST(test_motor_direction_forward);
    RUN_TEST(test_motor_direction_backward);
    RUN_TEST(test_motor_stop);

    // Uruchomienie testów servo_control
    RUN_TEST(test_servo_init);
    RUN_TEST(test_servo_step_left);
    RUN_TEST(test_servo_step_right);
    RUN_TEST(test_servo_center);

    // Uruchomienie testów safety_handler
    RUN_TEST(test_safety_validate_duration);
    RUN_TEST(test_safety_auto_stop);

    // Zakończenie i podsumowanie
    UNITY_END();
}
```

15.4 Testy robot_core - test_robot.c

```c
// Plik: test/main/test_robot.c
/**
 * @file test_robot.c
 * @brief Unit tests for robot_core component
 */

#include <unity.h>
#include "robot.h"
```

```c
// Test inicjalizacji
void test_robot_init(void) {
    // Arrange - przygotowanie danych
    robot_config_t config = {
        .movement_timeout_ms = 5000,
        .turret_timeout_ms = 2000,
        .gpio_enabled = false,  // Mock mode!
        .camera_url = "/stream"
    };

    // Act - wykonanie akcji
    esp_err_t ret = robot_init(&config);

    // Assert - sprawdzenie wyniku
    TEST_ASSERT_EQUAL(ESP_OK, ret);

    // Cleanup
    robot_cleanup();
}

// Test ruchu do przodu
void test_robot_move_forward(void) {
    // Arrange
    robot_config_t config = {
        .movement_timeout_ms = 5000,
        .turret_timeout_ms = 2000,
        .gpio_enabled = false,
        .camera_url = "/stream"
    };
    robot_init(&config);

    // Act
    robot_result_t result = robot_move(ROBOT_DIR_FORWARD, 1000);

    // Assert
    TEST_ASSERT_TRUE(result.success);
    TEST_ASSERT_EQUAL(ROBOT_ACTION_FORWARD, result.action);
    TEST_ASSERT_EQUAL(1000, result.duration_ms);

    // Cleanup
    robot_cleanup();
}
```

```c
// Test stopu
void test_robot_stop(void) {
    robot_config_t config = {
        .movement_timeout_ms = 5000,
        .turret_timeout_ms = 2000,
        .gpio_enabled = false,
        .camera_url = "/stream"
    };
    robot_init(&config);

    robot_result_t result = robot_stop();

    TEST_ASSERT_TRUE(result.success);
    TEST_ASSERT_EQUAL(ROBOT_ACTION_STOP_ALL, result.action);

    robot_cleanup();
}
```

15.5 Testy motor_control - test_motor_control.c

```c
// Plik: test/main/test_motor_control.c
#include <unity.h>
#include "motor_control.h"

// Test walidacji NULL
void test_motor_control_init(void) {
    // Przekazanie NULL powinno zwrócić błąd
    esp_err_t ret = motor_control_init(NULL);
    TEST_ASSERT_EQUAL(ESP_ERR_INVALID_ARG, ret);
}

// Test wywołania bez inicjalizacji
void test_motor_direction_forward(void) {
    // Upewnij się że moduł nie jest zainicjalizowany
    motor_control_cleanup();

    // Próba ruchu bez init powinna zwrócić błąd
    esp_err_t ret = motor_move_forward(1000);
    TEST_ASSERT_EQUAL(ESP_ERR_INVALID_STATE, ret);
}

void test_motor_direction_backward(void) {
    motor_control_cleanup();
```

```
    esp_err_t ret = motor_move_backward(1000);
    TEST_ASSERT_EQUAL(ESP_ERR_INVALID_STATE, ret);
}

void test_motor_stop(void) {
    motor_control_cleanup();
    esp_err_t ret = motor_stop_all();
    TEST_ASSERT_EQUAL(ESP_ERR_INVALID_STATE, ret);
}
```

15.6 Asercje Unity

Unity oferuje wiele makr asercji:

Makro	Opis
`TEST_ASSERT_TRUE(x)`	x musi być prawdziwe
`TEST_ASSERT_FALSE(x)`	x musi być fałszywe
`TEST_ASSERT_EQUAL(exp, act)`	Równość (int)
`TEST_ASSERT_EQUAL_STRING(exp, act)`	Równość stringów
`TEST_ASSERT_EQUAL_FLOAT(exp, act, eps)`	Równość float z tolerancją
`TEST_ASSERT_NULL(ptr)`	ptr == NULL
`TEST_ASSERT_NOT_NULL(ptr)`	ptr != NULL
`TEST_FAIL_MESSAGE(msg)`	Natychmiastowa porażka z komunikatem

15.7 Uruchamianie testów

```
# Budowanie testów
cd esp32-robot
idf.py -C test set-target esp32
idf.py -C test build

# Flashowanie i uruchomienie
idf.py -C test -p /dev/ttyUSB0 flash monitor

# Wynik w konsoli:
# test/main/test_main.c:44:test_robot_init:PASS
# test/main/test_main.c:45:test_robot_move_forward:PASS
# ...
# -----------------------
```

```
# 16 Tests 0 Failures 0 Ignored
# OK
```

15.8 Ćwiczenie

Ćwiczenie 15.1: Test walidacji duration

Zadanie: Napisz test sprawdzający że `safety_validate_duration()` poprawnie ogranicza czas.

Rozwiązanie:

```c
// W test_safety_handler.c:
#include <unity.h>
#include "safety_handler.h"

void test_safety_validate_duration(void) {
    // Test: duration <= max → bez zmian
    uint32_t result = safety_validate_duration(1000, 5000);
    TEST_ASSERT_EQUAL(1000, result);

    // Test: duration > max → clamp do max
    result = safety_validate_duration(10000, 5000);
    TEST_ASSERT_EQUAL(5000, result);

    // Test: duration == 0 → bez zmian (ciągły ruch)
    result = safety_validate_duration(0, 5000);
    TEST_ASSERT_EQUAL(0, result);
}
```

Rozdział 16: Budowanie, Flashowanie i Debugging

16.1 System budowania CMake

ESP-IDF używa CMake jako systemu budowania. Każdy komponent ma swój `CMakeLists.txt`:

```
# Plik: components/motor_control/CMakeLists.txt
idf_component_register(
    SRCS "src/motor_control.c" "src/pwm_ramper.c"
    INCLUDE_DIRS "include"
    REQUIRES robot_hal robot_core freertos driver
)
```

Dyrektywa	Opis
`SRCS`	Pliki źródłowe do skompilowania
`INCLUDE_DIRS`	Katalogi z nagłówkami (publiczne)
`REQUIRES`	Zależności od innych komponentów
`PRIV_REQUIRES`	Zależności prywatne (nie eksponowane)

16.2 Konfiguracja Kconfig

Każdy komponent może definiować opcje konfiguracyjne:

```
# Plik: main/Kconfig.projbuild
menu "Robot Controller Configuration"

    menu "WiFi Settings"
        config ROBOT_WIFI_SSID
            string "WiFi SSID"
            default "ESP32"
            help
                SSID sieci WiFi AP

        config ROBOT_WIFI_PASSWORD
            string "WiFi Password"
            default "eEspetrzyjsci2a"
            help
                Hasło sieci WiFi (min 8 znaków dla WPA2)
    endmenu

    menu "Motor Control"
        config ROBOT_MOTOR_LEFT_IN1
            int "Left Motor IN1 GPIO"
            default 12
            range 0 39

        config ROBOT_MOTOR_LEFT_IN2
            int "Left Motor IN2 GPIO"
            default 13
            range 0 39
    endmenu

endmenu
```

Używanie w kodzie:

```
// Dostęp przez makra CONFIG_*
const char *ssid = CONFIG_ROBOT_WIFI_SSID;
const int gpio = CONFIG_ROBOT_MOTOR_LEFT_IN1;
```

16.3 Tablica partycji

```
# Plik: partitions.csv
# Name,   Type, SubType, Offset,  Size, Flags
nvs,      data, nvs,     0x9000,  0x6000,
phy_init, data, phy,     0xf000,  0x1000,
factory,  app,  factory, 0x10000, 0x300000,
spiffs,   data, spiffs,  0x310000, 0xF0000,
```

Partycja	Typ	Rozmiar	Opis
nvs	data	24KB	Non-Volatile Storage
phy_init	data	4KB	Kalibracja PHY WiFi
factory	app	3MB	Aplikacja
spiffs	data	960KB	System plików (Web UI)

16.4 Polecenia idf.py

```
# Ustawienie targetu (raz)
idf.py set-target esp32

# Konfiguracja menuconfig
idf.py menuconfig

# Budowanie
idf.py build

# Flashowanie
idf.py -p /dev/ttyUSB0 flash

# Monitor szeregowy
idf.py -p /dev/ttyUSB0 monitor
# Wyjście: Ctrl+]

# Flash + monitor razem
idf.py -p /dev/ttyUSB0 flash monitor
```

```
# Czyszczenie
idf.py fullclean
```

16.5 Debugowanie

16.5.1 Logowanie ESP_LOG

```
#include <esp_log.h>

static const char *TAG = "my_module";

// Poziomy logowania (od najniższego):
ESP_LOGV(TAG, "Verbose: szczegóły");   // Tylko gdy LOG_LOCAL_LEVEL >=
VERBOSE
ESP_LOGD(TAG, "Debug: wartość=%d", x); // Tylko gdy LOG_LOCAL_LEVEL >= DEBUG
ESP_LOGI(TAG, "Info: normalny log");   // Domyślnie widoczny
ESP_LOGW(TAG, "Warning: uwaga!");      // Żółty
ESP_LOGE(TAG, "Error: błąd!");         // Czerwony
```

16.5.2 Ustawienie poziomu logów

W `menuconfig`:

```
Component config → Log output → Default log verbosity → Debug
```

Lub w kodzie:

```
esp_log_level_set("motor_control", ESP_LOG_DEBUG);
esp_log_level_set("*", ESP_LOG_INFO);  // Wszystkie moduły
```

16.5.3 Przykładowy output

```
I (324) cpu_start: Starting scheduler on PRO CPU.
I (0) cpu_start: Starting scheduler on APP CPU.
I (334) main: ESP32-S3-CAM Robot Controller starting...
I (344) main: Mock mode: disabled
I (354) wifi_manager: WiFi manager initialized
I (364) wifi_manager: AP started - SSID: ESP32, Password: ***, IP: 10.42.0.1
I (374) motor_control: Motor control initialized (freq=1000 Hz, ramp=500 ms)
I (384) servo_control: Servo initialized (pin=16, default=90°)
I (394) http_server: HTTP server started on port 4567
I (404) main: Robot controller initialized successfully
I (414) main: Web UI available at http://10.42.0.1:4567/
```

16.6 Typowe problemy

Problem	Rozwiązanie
"Failed to connect"	GPIO 0 → GND, wciśnij RESET
"No serial output"	Sprawdź TX/RX, baud 115200
"Camera init failed"	Sprawdź PSRAM w menuconfig
"WiFi connect failed"	Sprawdź SSID/hasło, tylko 2.4GHz
"SPIFFS mount failed"	Uruchom `idf.py build` (generuje spiffs.bin)

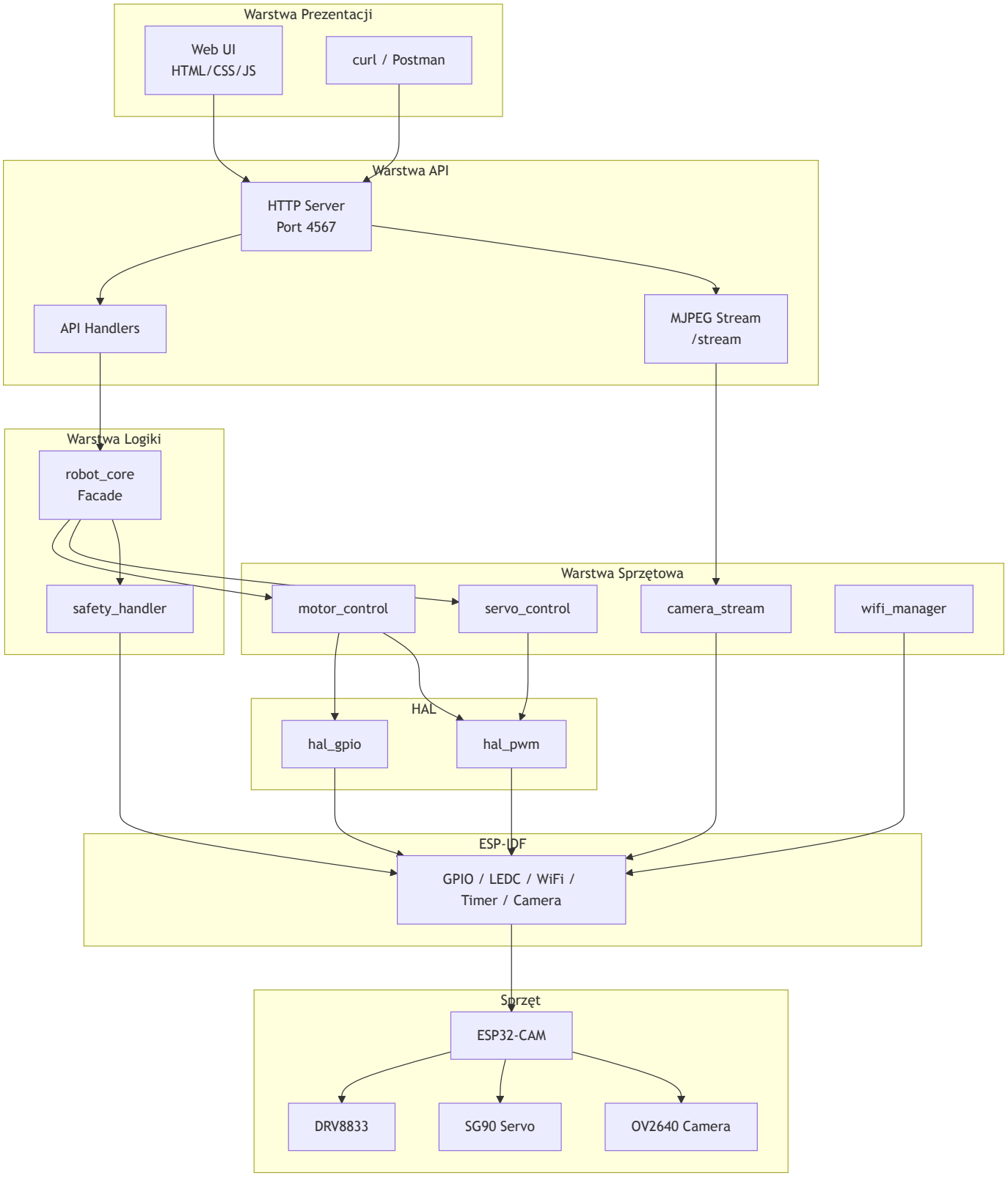
Rozdział 17: Gotowy Projekt i Dalszy Rozwój

17.1 Podsumowanie projektu

Zbudowaliśmy kompletnego robota z ESP32-CAM, który oferuje:

Funkcja	Komponent	Technologia
Sterowanie silnikami	motor_control	DRV8833 + PWM
Sterowanie serwem	servo_control	PWM 50Hz
WiFi Access Point	wifi_manager	ESP WiFi
Streaming kamery	camera_stream	MJPEG + OV2640
REST API	http_server	ESP HTTP Server
Bezpieczeństwo	safety_handler	ESP Timer
Fasada	robot_core	Wzorzec Facade
Web UI	SPIFFS	HTML/CSS/JS

17.2 Architektura końcowa



17.3 Możliwości rozszerzenia

Funkcja	Opis	Komponenty
Czujniki odległości	Unikanie przeszkód	HC-SR04, VL53L0X
Śledzenie linii	Autonomiczna jazda	Czujniki IR
Rozpoznawanie obrazu	AI on the edge	ESP-WHO, TensorFlow Lite

Funkcja	Opis	Komponenty
Sterowanie głosowe	Komendy głosowe	ESP-SR
OTA Updates	Aktualizacja przez WiFi	ESP OTA
MQTT	IoT integration	ESP MQTT
Bluetooth	Sterowanie z telefonu	ESP BLE

17.4 Twoja droga jako Embedded Developer

17.4.1 Umiejętności zdobyte w tym projekcie

- ✅ Zaawansowane C (struktury, wskaźniki, preprocesor)
- ✅ Programowanie mikrokontrolerów (GPIO, PWM, timery)
- ✅ ESP-IDF framework (komponenty, Kconfig, CMake)
- ✅ Protokoły komunikacyjne (WiFi, HTTP, JSON)
- ✅ RTOS (FreeRTOS - taski, semafory, mutexy)
- ✅ Wzorce projektowe (HAL, Facade)
- ✅ Testowanie (Unity framework)
- ✅ Debugowanie embedded

17.4.2 Następne kroki

1. **Praktyka** - Zbuduj więcej projektów
2. **Dokumentacja** - Czytaj ESP-IDF docs
3. **Społeczność** - Dołącz do forum ESP32
4. **Certyfikaty** - Rozważ kursy Embedded Systems
5. **Portfolio** - Publikuj projekty na GitHub

17.5 Podziękowania

Gratulacje! Ukończyłeś kompleksowy kurs budowy robota ESP32-CAM. Masz teraz solidne podstawy do kariery jako Embedded Developer.

Powodzenia! 🤖

Dodatek A: Pełna Lista Referencji do Dokumentacji

ESP-IDF

Temat	Link
GPIO Driver	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/gpio.html
LEDC (PWM)	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/ledc.html
WiFi	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_wifi.html
HTTP Server	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/esp_http_server.html
FreeRTOS	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html
ESP Timer	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_timer.html
NVS	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html
SPIFFS	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/spiffs.html
Event Loop	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_event.html
Error Codes	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_err.html
Logging	https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/log.html

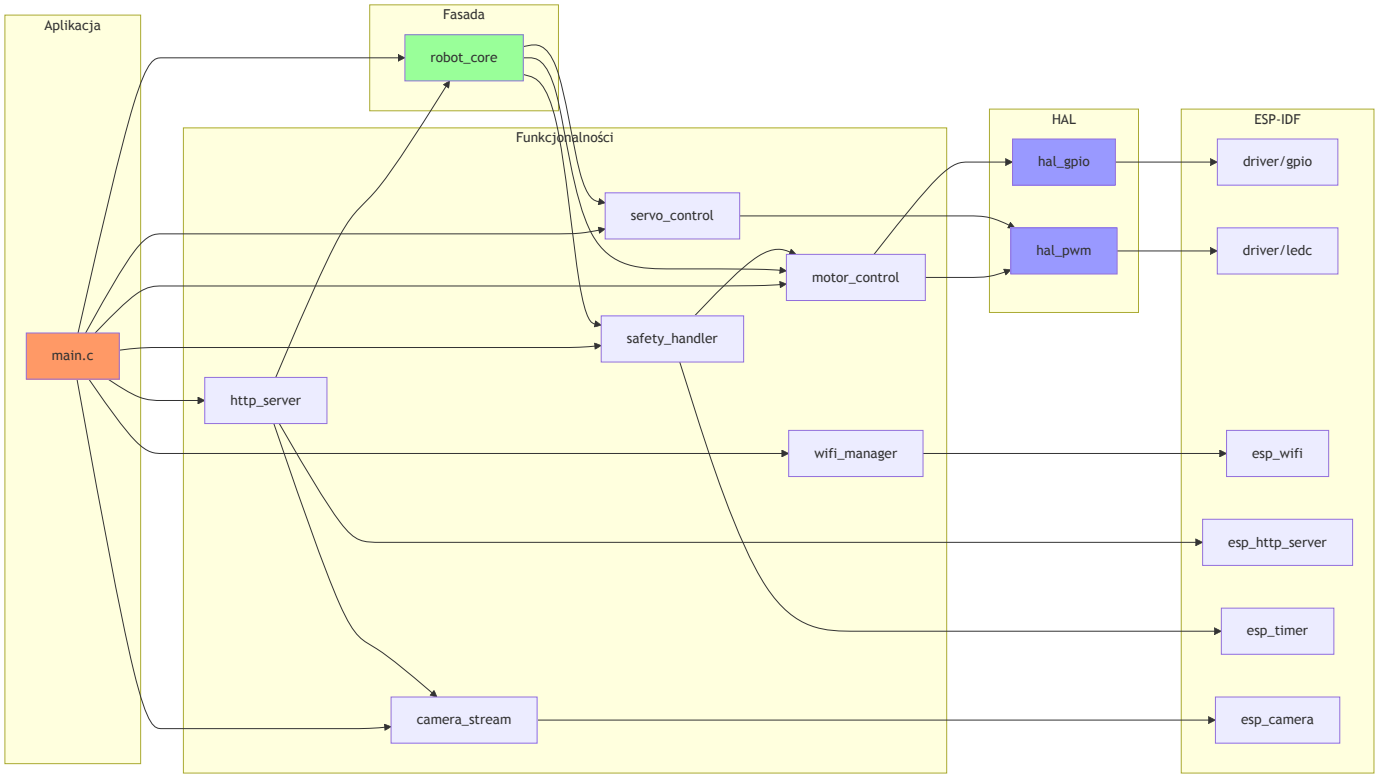
Zewnętrzne biblioteki

Biblioteka	Link
esp32-camera	https://github.com/espressif/esp32-camera
cJSON	https://github.com/DaveGamble/cJSON
Unity	http://www.throwtheswitch.org/unity

Dodatek B: Słownik Polsko-Angielski

Polski	English	Opis
Mikrokontroler	Microcontroller	Komputer w jednym układzie
Pin	Pin	Nóżka układu
Wejście/Wyjście	Input/Output (I/O)	Kierunek przepływu sygnału
Przerwanie	Interrupt	Asynchroniczne zdarzenie
Timer	Timer	Licznik sprzętowy
Watchdog	Watchdog	Strażnik resetujący system
Rampa	Ramp	Stopniowa zmiana wartości
Mostek H	H-Bridge	Układ sterowania silnikiem
Serwo	Servo	Silnik z kontrolą pozycji
Impuls	Pulse	Krótki sygnał
Cykl pracy	Duty Cycle	Stosunek HIGH do okresu
Stos	Stack	Pamięć lokalna funkcji
Sterta	Heap	Pamięć dynamiczna
Task	Task	Wątek w RTOS
Semafor	Semaphore	Mechanizm synchronizacji
Mutex	Mutex	Blokada wzajemnego wykluczenia
Callback	Callback	Funkcja wywoływana zwrotnie
Handler	Handler	Funkcja obsługująca zdarzenie
Buffer	Buffer	Bufor danych
Endpoint	Endpoint	Punkt końcowy API
Stream	Stream	Strumień danych

Dodatek C: Diagram Zależności Komponentów



Koniec E-booka

"Od Podstaw C do Profesjonalnego Developera Embedded"

Wersja 1.0

Dziękujemy za przeczytanie tego e-booka. Mamy nadzieję, że pomógł Ci zrozumieć świat programowania embedded i zbudować Twojego pierwszego robota!

Jeśli masz pytania lub sugestie, otwórz Issue na GitHubie projektu.

Happy coding! 🚀

---

1. `gpio_num_t` - typ zdefiniowany w ESP-IDF reprezentujący numer pinu GPIO. Zobacz: ESP-IDF GPIO Driver ↩
2. `uint32_t`, `uint8_t` - typy o gwarantowanym rozmiarze zdefiniowane w `<stdint.h>`. Zobacz: C Standard Integer Types ↩
3. `esp_event_handler_instance_register()` - zobacz: ESP-IDF Event Loop ↩
4. `WIFI_EVENT` - baza zdarzeń WiFi, zobacz: ESP-IDF WiFi Events ↩
5. `gpio_config_t` - zobacz: ESP-IDF GPIO API ↩ ↩
6. `GPIO_MODE_OUTPUT` - tryb wyjściowy, zobacz: gpio_mode_t ↩
7. `gpio_config()` - konfiguruje GPIO, zobacz: gpio_config() ↩
8. `gpio_set_level()` - ustawia stan pinu, zobacz: gpio_set_level() ↩

9. LEDC Driver - zobacz: ESP-IDF LEDC API ↩
10. `ledc_timer_config()` - zobacz: LEDC Timer Config ↩
11. `ledc_channel_config()` - zobacz: LEDC Channel Config ↩
12. `ledc_set_duty()` - zobacz: ledc_set_duty ↩
13. `ledc_update_duty()` - zobacz: ledc_update_duty ↩
14. FreeRTOS - zobacz: FreeRTOS Documentation oraz ESP-IDF FreeRTOS ↩
15. `xTaskCreate()` - zobacz: xTaskCreate API ↩
16. `vTaskDelay()` - zobacz: vTaskDelay ↩
17. `ulTaskNotifyTake()` - zobacz: Task Notifications ↩
18. `xTaskNotifyGive()` - zobacz: xTaskNotifyGive ↩
19. `xSemaphoreCreateMutex()` - zobacz: Semaphore API ↩
20. ESP Timer - zobacz: ESP-IDF Timer API ↩
21. `esp_timer_create()` - zobacz: esp_timer_create ↩
22. `esp_timer_start_once()` - zobacz: esp_timer_start_once ↩
23. `esp_timer_start_periodic()` - zobacz: esp_timer_start_periodic ↩
24. `esp_timer_stop()` - zobacz: esp_timer_stop ↩
25. `esp_timer_delete()` - zobacz: esp_timer_delete ↩
26. ESP-IDF - zobacz: ESP-IDF Programming Guide ↩
27. NVS Flash - zobacz: ESP-IDF NVS API ↩
28. SPIFFS - zobacz: ESP-IDF SPIFFS API ↩